

时空可组合性：一种编程范式

原文：A Programming Paradigm for Spatiotemporal Composability

Yifan Shi^{1,2}, Wei Zhang¹, Tianyi Cui²

¹北京大学 ²DeepSeek-AI

目录

1	引言	4
1.1	可组合性的维度	4
1.2	动机示例	5
1.2.1	插件系统	5
1.2.2	自演化智能体框架	5
1.2.3	粗粒度变通方案	6
1.3	贡献	6
2	预备知识	7
2.1	效应	7
2.2	余效应	7
2.3	与动态组合的关系	8
3	可逆效应与反应式余效应	9
3.1	可逆效应	9
3.1.1	效应上下文	9
3.1.2	可逆效应函数	12
3.1.3	效应的独立性	15
3.2	反应式余效应	18
3.2.1	余效应上下文	18
3.2.2	规范与通知	19
3.2.3	隔离与拦截	20
3.3	上下文范式 (The Context Paradigm)	22
3.3.1	统一上下文 (Unified Context)	23
3.3.2	观测等价 (Observational Equivalence)	23
3.3.3	上下文范式的定位 (Situating the Context Paradigm)	27

4	动态组合演算	28
4.1	组件与纤程	28
4.2	基础演算	30
4.3	进行中的迁移	33
4.3.1	撤销	34
4.3.2	迭代	35
4.3.3	异步	37
4.3.4	失败	37
4.4	元理论	38
4.4.1	保持性	42
4.4.2	时间可组合性	44
4.4.3	空间可组合性	46
4.4.4	进展性	47
4.4.5	合流性	49
5	实现与案例研究	54
5.1	核心库	54
5.1.1	效应追踪	55
5.1.2	余效应操作	57
5.1.3	组件生命周期	58
5.1.4	上下文访问	61
5.2	组件加载器	61
5.2.1	声明式配置	61
5.2.2	热模块替换	64
5.3	案例研究: Koishi	66
6	讨论	67
6.1	系统边界	68
6.2	服务多路复用	68
6.3	访问控制与沙箱化	69
6.4	语言无关性与选择	70
6.5	相互依赖与组件粒度	71
6.6	依赖类型化与版本管理	71
6.7	与语言和操作系统的协同设计	72
7	相关工作	73

7.1	效应与余效应系统	74
7.1.1	单子效应系统 (Monadic effect systems)	74
7.1.2	作为能力的代数效应 (Algebraic effects as capabilities)	74
7.1.3	可逆效应语义 (Reversible effect semantics)	74
7.1.4	作为效应与余效应统一体的分级类型 (Graded types as unified effects and coeffects)	75
7.2	编程范式	75
7.2.1	面向上下文的编程 (Context-oriented programming)	75
7.2.2	面向切面的编程 (Aspect-oriented programming)	75
7.3	时间可组合性	76
7.3.1	有状态的前向迁移 (Stateful forward migration)	76
7.3.2	开发者编写的恢复 (Developer-authored recovery)	77
7.3.3	静态作用域的逆转 (Statically scoped reversal)	77
7.3.4	拦截式回收 (Interposed reclamation)	77
7.4	空间可组合性	78
7.4.1	初始化时的依赖接线 (Initialization-time dependency wiring)	78
7.4.2	可用性反应式组件模型 (Availability-reactive component models)	78
7.4.3	值级反应性 (Value-level reactivity)	79
8	结论	79
9	参考文献	80

摘要

现代软件——从插件系统到自演化智能体框架——日益要求动态组合 (dynamic composition)，但其形式化基础仍然欠发达。我们识别出该问题的两个正交维度：时间可组合性 (temporal composability)，即在移除组件时完全撤销其副作用的能力；以及空间可组合性 (spatial composability)，即能够声明并以反应式方式管理组件间依赖的能力。我们将经典的效应 (effect) 与余效应 (coeffect) 概念提升为运行时机制来应对这两个维度。具体而言，我们形式化了可逆效应 (reversible effects)，其中每个上下文变换都携带一个由运行时追踪的逆。我们形式化了反应式余效应 (reactive coeffects)，其中上下文的每次变化都会依据组件的余效应规范通知该组件。我们将效应上下文与余效应上下文统一为单一的上下文类型，从而构成一种编程范式。此后，我们将这些机制组合为组件的概念，并给出一个动态组合的演算，其元理论将时空可组合性从单个组件传递到由交错组件构成的整个系统。我们在 Cordis 中实现了这些思想——Cordis 是一个时空可组合性的元框架，提供具备效应追踪与余效应解析的核心库，以及带有配置调和与热模块替换的声明式组件加载器。

1 引言

组合 (composition) ——由较简单的部件组装出复杂系统——是软件工程的基础性原则 [1]。传统上，组合是静态的：函数调用、模块导入和类继承在编译期解析，并在整个执行过程中保持不变。然而，现代软件日益要求动态组合，即在运行时加载、卸载和重新配置组件。插件架构 [2] 与自演化智能体框架都要求系统能够在运行过程中安全地增删功能，但当前的实践仍退而求助于粗粒度的机制 [3]，这类机制只能通过重启完成重新配置，并丢弃运行时状态。尽管动态组合的实际重要性日益增长，与静态组合已有的丰富形式框架相比，其理论基础仍然欠发达。

1.1 可组合性的维度

为刻画动态组合的需求，我们在组合的代数性质这一已被充分研究的方面之外，识别出两个正交的维度：

- 时间可组合性处理时间维度：当组件被移除时，该组件对共享环境所做的修改必须被完全且安全地逆转。这要求追踪组件执行的每一次资源分配、事件注册和状态变更，并保证在移除时有序地回收它们。
- 空间可组合性处理空间维度：组件必须能够以结构化且可验证的方式声明、发现并解析彼此之间的依赖。这要求管理依赖拓扑，并依据依赖的变化协调组件生命周期。

在静态设定下，时间可组合性归结为词法作用域（例如 RAII [4]、括号模式 [5]），空间可组合性归结为模块导入解析 [6]。在动态设定下——组件在运行时到来和离去——两个维度都变得显著困难：时间可组合性必须处理生命周期长、有状态且作用域不受词法约束的效应；空间可组合性必须处理在执行期间出现、消失或改变身份的依赖。

1.2 动机示例

1.2.1 插件系统

插件系统是动态组合的典型实例。我们以 Visual Studio Code (VSCode) ——使用最广泛的、可扩展的集成开发环境之一——作为代表性示例。

时间局限。VSCode 在名为扩展宿主 (extension host) 的共享进程中运行所有扩展。尽管扩展可以动态安装, 该宿主却没有任何机制能在运行时卸载单个扩展的代码。一旦某个扩展的 `activate` 函数执行完毕, 禁用或卸载它就需要重启整个宿主, 从而影响所有已加载的扩展。主题、按键绑定和代码片段等纯声明式扩展不携带代码, 可以随意移除。然而, 在按安装量排名的前 100 个扩展中, 有 87 个包含可执行代码¹, 因此移除时都需要这样的重启。尽管 VSCode 提供了 `deactivate` 钩子, 但它只是在宿主进程终止期间充当优雅关闭的回调, 因而无法实现在线移除。此外, 该钩子将效应释放与效应创建 (在 `activate` 中) 分离开来, 违反了关注点局部性 (locality of concern), 使完整清理难以验证。

空间局限。VSCode 确实提供了用于声明扩展之间依赖的 `extensionDependencies`, 但它很少被使用: 在按安装量排名的前 100 个扩展中, 只有 7 个对非内置扩展声明了 `extensionDependencies`。¹ 这种稀缺反映了扩展 API 的形态: 它暴露的是命令、视图、语言特性等固定的、表层级的扩展点。扩展通过这些扩展点向宿主做出贡献, 而不是彼此依赖, 因此扩展间的依赖很少出现。此外, VSCode 的扩展间交互机制不提供任何结构性的契约: 它通过 `vscode.extensions.getExtension(...).exports` 向其他扩展暴露自身功能, 但返回值是无类型的 (默认是 `any`), 因此依赖方无法依靠经过检查的接口。总之, VSCode 将扩展导向一组固定的、由宿主提供的扩展点, 却没有为扩展之间相互依赖提供安全、结构化的方式。

这两个局限并非 VSCode 独有; 它们普遍存在于各类插件系统中 [2, 7], 只是程度不同。

1.2.2 自演化智能体框架

现代 AI 智能体依赖运行时的智能体框架 [8–10]。这些系统可以组合多样化的工具套件 [11] 与执行环境, 管理权限与沙箱化, 维护会话状态与持久化, 提供上下文管理与记忆系统 [12], 编排子智能体与多智能体工作流 [13], 并向用户和自动化系统暴露接口。未来的框架可能一边持续处理请求, 一边生成并部署对自身组件的修改。模型合成的可复用工具为组件级自我修改提供了一个更狭义的先导 [14]。这样的每一次修改本身都是动态组合的一个实例。

由于这些修改连续不断地发生, 且几乎没有或完全没有人工监督, 动态可组合性变得不可或缺。没有时间可组合性, 每一次自我修改都迫使完全重启, 丢弃所有进程内累积的状态; 在如此频繁的重启下, 累计的不可用时间相当可观, 进行中的任务反复被打断; 更糟的是, 一次有缺陷的自我修改可能使用于恢复的进程本身无法工作。没有空间可组合性, 每个模块都必须自行检测并适应它所依赖的模块在出现、消失或改变身份时的变化, 而且只能借助临时性 (*ad hoc*) 手段; 更糟的是, 朴素的代码替换策略可能悄无声息地破坏依赖方, 或引入只在重载时才暴露出来的循环依赖。

¹数据取自 Visual Studio Code Marketplace, 检索日期为 2026 年 6 月 9 日。

1.2.3 粗粒度变通方案

动态可组合性之所以得到有限的正式关注，一个原因是操作系统和容器编排器已经提供了粗粒度的替代方案。操作系统在进程粒度上提供时间可组合性；容器编排器 [3] 在服务粒度上提供空间可组合性。在实践中，大多数软件通过退而使用这些粗粒度机制来容忍细粒度可组合性的缺失：行为异常的模块靠重启进程来处理，服务依赖则由容器编排器来管理。

然而，这种变通方案代价高昂。在时间上，每次重启都会丢弃所有进程内累积的状态（例如缓存、连接、部分计算结果），而重建这些状态需要数秒到数分钟 [15]；在此期间维持可用性需要冗余副本，以额外的资源开销来弥补无法恢复单个组件的缺陷。在空间上，容器级编排无法表达共享同一地址空间的组件之间的依赖，并且为本可通过本地函数调用完成的交互引入了网络开销。这两种机制都作用于进程和容器的边界，而现代系统越来越多地在更细的层次上进行组合。这种粒度不匹配要求一种组合抽象，能在与组件本身相同的层次上管理效应与依赖。

1.3 贡献

动态可组合性的两个维度分别关注计算如何修改其环境，以及计算如何依赖其环境。这两个方向正是效应系统 [16, 17] 与余效应系统 [18, 19] 所形式化的内容：效应为推理环境修改提供了形式化词汇，余效应则为推理环境需求提供了形式化词汇。然而，现有的表述将推理限制在词法固定作用域上的编译期分析，无法扩展到组件在运行时到来和离去的动态场景。通过将效应提升为可逆的运行时代模型、将余效应提升为反应式的依赖解析机制，我们获得了动态组合的统一形式化基础——它与语言无关，适用于任何需要动态组合的软件架构。我们的贡献如下：

1. 我们形式化了可逆效应（第 3.1 节）：每个上下文变换都携带一个由运行时追踪的显式逆，且追踪与恢复都保持组合性，从而在移除组件时上下文得以恢复。这确立了局部时间可组合性。
2. 我们形式化了反应式余效应（第 3.2 节）：组件将自身所需的余效应声明为一种规范，上下文的每次变化都会依据该规范将组件通知为激活、停用或中性状态。这确立了局部空间可组合性。
3. 我们将效应上下文与余效应上下文统一为单一的上下文类型（第 3.3 节），其中余效应上的观测等价于效应提供了独立性，从而构成时空可组合性的一种编程范式。
4. 我们给出了一个动态组合的演算（第 4 节），它将上述两种机制组合为组件的概念，并为组件的生命周期配备了操作语义。其元理论将时空可组合性从单个组件传递到由交错组件构成的整个系统。
5. 我们在 Cordis（第 5 节）中实现了这些思想——Cordis 是一个时空可组合性的元框架，提供实现该形式模型、具备效应追踪与余效应解析的核心库，以及带有配置调和与热模块替换的声明式组件加载器。

2 预备知识

本节简要概述效应 (effect) 系统与余效应 (coeffect) 系统——支撑我们工作的两大理论支柱。我们假定读者熟悉基础类型论与范畴论；此处的目标是固定记号，并引入第 3 节将作为运行时机制加以实现的关键抽象。

2.1 效应

在简单类型 λ 演算 (STLC) [20, 21] 中，类型判断 $\Gamma \vdash t : T$ 表明项 t 在上下文 Γ 下具有类型 T 。效应系统对类型进行细化，以描述一次计算可能产生哪些副作用，从而得到如下形式的判断：

$$\Gamma \vdash t : T \quad (1)$$

(效应)

这里，结果类型被标注以效应代数的一个元素，该元素描述该计算可能产生的副作用，从而能够对带状态的计算进行组合式推理。这一方法源于 Lucassen 和 Gifford [22]，他们引入了一个带种类的类型系统，区分类型、效应与区域，以发现并行程序中的调度约束。

单子效应。Moggi [16] 首先借助单子 (monad) 在范畴论层面对计算效应进行建模；Wadler [23] 在 Haskell 中推广了这一方法。范畴 $\mathcal{C}$ 上的一个单子 (T, η, μ) 将带效应的计算封装为类型 $T(A)$ 的值，其中 $\eta : A \rightarrow T(A)$ 提升纯值， $\mu : T(T(A)) \rightarrow T(A)$ 将嵌套的计算按顺序组合。经典实例包括 Maybe 单子（对应部分性）、State 单子（对应可变状态）与 IO 单子（对应外部交互）。

代数效应。Plotkin 和 Power [17, 24] 证明了代数运算可以确定单子，从而建立了效应接口与其实现相解耦的框架。效应签名 Σ 声明一组运算（例如，针对状态的 $\text{get} : () \rightarrow S$ 与 $\text{put} : S \rightarrow ()$ ）；程序可以自由地调用这些运算，而不必承诺某种特定的解释。Plotkin 和 Pretnar [25] 随后引入了效应处理器 (effect handler)，通过提供续延 (continuation) 语义来解释这些运算：

$$\text{handle } e \text{ with } \{ \text{op}(v, \kappa) \mapsto \dots \} \quad (2)$$

处理器接收运算参数 v 与定界续延 (delimited continuation) κ ，它可以调用该续延零次、一次或多次，从而在一个统一的框架内支持异常、协程与非确定性 [26]。Koka [27, 28]、Eff [29] 与 OCaml 5 [30] 等语言均已采用代数效应，并在设计上作出了各不相同的取舍。

2.2 余效应

与效应相对偶，余效应系统 [18, 31] 丰富的是上下文而非类型，得到如下形式的判断：

$$\Gamma \vdash t : T \quad (3)$$

(余效应)

这里，上下文被标注以余效应代数的一个元素，该元素描述计算对其环境的要求，例如需要访问的资源、需要持有的权限，或需要依赖的服务。效应为程序对世界的影响建模，而余效应为世界对程序的约束建模。

余单子余效应。使用余单子 (comonad) 来组织上下文相关计算的思想最早由 Uustalu 和 Vene [32] 提出，他们提出了对称 (半) 么半群余单子，作为 Moggi 面向效应的单子框架的对偶，用以刻画数据流与属性求值等概念。Petricek 等人 [18] 在此基础上，将余效应作为对上下文依赖的一种统一静态分析加以提出。一个余单子 (D, ε, δ) 刻画上下文相关的计算： $\varepsilon : D(A) \rightarrow A$ 从上下文中提取当前值， $\delta : D(A) \rightarrow D(D(A))$ 复制上下文以供嵌套访问。环境余单子 (Environment comonad) $D(X) = E \times X$ 为对固定环境 E 的依赖建模；流余单子 (Stream comonad) $D(X) = \mathbb{N} \rightarrow X$ 为对时序数据的依赖建模。

分级余效应。为进行更细粒度的追踪，分级余效应 (graded coeffects) 系统使用一个预序半环 (pre-ordered semiring) $\mathcal{S} = (S, \leq, +, \times, 0, 1)$ 作为余效应代数 [33]，这一学科后来由 Gaboardi 等人 [19] 与分级效应统一起来。 S 的元素标注每一个变量绑定，以量化其使用方式：0 表示未使用，1 表示线性使用， n 表示有界使用， ∞ 表示无限制使用。半环的运算分别以顺序 ($\times$) 与并行 ($+$) 方式组合余效应，从而能够在统一的代数框架 [37] 内实现精确的资源追踪、敏感性分析 [34] 与信息流控制 [35, 36]。

2.3 与动态组合的关系

效应系统与余效应系统沿着两个互补的方向组织关于计算的推理：效应描述计算如何修改其环境，而余效应描述计算如何依赖其环境。这两个方向分别对应第 1 节所识别的动态组合的两个维度：

- **时间可组合性**要求组件对共享环境的修改在卸载时必须是可逆的。相关的效应是那些带状态的效应，它们持久地变换该环境；要撤销这样的变换，该变换必须具有逆。
- **空间可组合性**要求组件间的依赖被声明出来并以反应式方式加以管理。这类依赖正是余效应所捕获的东西，而管理它们就相当于将每一个依赖对照环境所供应的内容加以消解。

然而，经典的效应系统与余效应系统是静态的工具：效应在词法固定的作用域内被追踪，并由编译期的处理器结算；余效应标注则对照执行之前就已确定的上下文进行验证。相比之下，动态组合要求这些保证对在运行时到来与离去的组件仍然成立，而面对的是持续演化的上下文。任何固定的词法作用域都无法界定部署之后才加载的插件；任何编译期的上下文都无法预见由运行时配置产生的依赖。

这促使我们转变视角：与其用更多的标注去扩展静态类型系统，不如将效应与余效应的概念结构具体化 (物化, reify)，使运行时能够直接对它们进行操作，从而以动态的方式建立起这些系统在静态时提供的保证。

3 可逆效应与反应式余效应

本节将第 2 节引入的效应 (effect) 与余效应 (coeffect) 概念提升为运行时机制, 从而构建起动态组合的理论。核心思想是把携带效应与余效应的类型上下文转化为上下文类型 (context type), 即把上下文具体化为一种一等实体、可在运行时操作的类型。对于效应类型, 我们将其建模为配对一个逆的上下文变换, 从而实现局部的时间可组合性。对于余效应上下文, 我们将其建模为携带依赖信息的类型, 从而实现局部的空间可组合性。余效应上的观测等价进而为效应提供独立性。同时携带效应与余效应的统一上下文本身就构成一种编程范式。

3.1 可逆效应

时间可组合性是指在运行时加载和卸载组件的能力, 使得卸载时共享环境被恢复到组合前的状态。这要求组件对环境所做的每一处修改既可追踪又可恢复。因此, 我们把效应建模为类型 $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$ 的函数: 将其作用于当前上下文, 它产生修改后的上下文以及一个显式的逆。提供这个逆使得效应可以被逆转, 而将其交还给运行时则使效应可被追踪。我们称这样的效应为可逆效应 (reversible effect): 通过在执行期间追踪并复合这些逆, 完整的环境恢复成为一种结构性保证。

3.1.1 效应上下文

给定任意非纯函数 $f_{\text{impure}} : X \rightarrow Y$, 我们将其变换为纯形式 $f : \Gamma \times X \rightarrow \Gamma \times Y$, 其中 Γ 是上下文, 所有可能的副作用都可以表示为 Γ 上的变换。对于任意固定的输入 $x : X$, 诱导映射 $\gamma \mapsto \text{pr}_1(f(\gamma, x))$ 独立于返回值地捕获了 f 的副作用。因此, Γ 上的效应生活在复合运算 $\circ$ 之下的变换幺半群 $\Gamma \rightarrow \Gamma$ 中, 其中每一条幺半群公理都可以直接解读为效应的一条性质:

- 封闭性: 两个效应的顺序复合仍然是效应;
- 结合性: 复合效应与括号的添加方式无关;
- 恒等性: id_Γ , 即 Γ 上的恒等函数, 充当复合的单位元。

为了对可撤销的效应建模, 我们把每个变换 f 与另一个撤销 f 的变换 g 配对, 并称 g 为 f 的左逆 (left inverse), 本文通篇简称为逆。撤销是单侧的: 对逆的要求是 $g \circ f$, 而绝不是 $f \circ g$ 。变换对带有它们自己的乘法:

定义 1. 定义上下文变换对的扭曲复合 (twisted composition) 为

$$(f_1, g_1) \circ (f_2, g_2) := (f_1 \circ f_2, g_2 \circ g_1) \quad (4)$$

与 $\circ$ 本身一样, 左操作数在右操作数之后作用, 而逆以相反的顺序累积。这使得 $(\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma)$ 成为一个以 $(\text{id}_\Gamma, \text{id}_\Gamma)$ 为单位元的幺半群, 即变换幺半群与其反幺半群 (对偶) 的积; 我们称之为 Γ 上的扭曲复合幺半群 $\mathfrak{T}_\Gamma$ 。

为了在上下文内部追踪效应，我们引入如下定义：

定义 2. 给定上下文 Γ ，定义其**效应上下文** (effect context) 为：

$$\partial\Gamma := \Gamma \times (\Gamma \rightarrow \Gamma) \quad (5)$$

它可以理解为一对 (γ, φ) ，其中：

- $\gamma : \Gamma$ 是当前上下文状态；
- $\varphi : \Gamma \rightarrow \Gamma$ 是累加器，即迄今为止所执行效应的逆的复合，也就是把上下文恢复到其初始状态的函数。

特别地，初始效应上下文可以表示为 $(\gamma_0, \text{id}_\Gamma)$ 。

我们还写 $\partial^2\Gamma = \partial\Gamma \times (\partial\Gamma \rightarrow \partial\Gamma)$ ，并如此沿塔向上类推。

由于累加器 φ 的存在，在 $\partial\Gamma$ 上执行的所有效应都可以被追踪和恢复。我们现在给出追踪与恢复的具体构造。

定义 3. 在上下文函数对上定义变换 track_Γ ：

$$\begin{aligned} \text{track}_\Gamma : (\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma) &\rightarrow \partial\Gamma \rightarrow \partial\Gamma \\ \text{track}_\Gamma = (f, g) &\mapsto (\gamma, \varphi) \mapsto (f(\gamma), \varphi \circ g) \end{aligned} \quad (6)$$

该变换把一个前向函数 f 连同候选逆 g 一起转换为效应上下文 $\partial\Gamma$ 上的一个变换。将 $\text{track}_\Gamma(f, g)$ 作用于状态 (γ, φ) ，会用 f 变换 γ ，并把逆 g 复合到 φ 上，从而把 f 的效应追踪进上下文中。

定理 4. 对每个 $(f, g) \in (\Gamma \rightarrow \Gamma) \times (\Gamma \rightarrow \Gamma)$ ，下图可交换，即

$$\text{pr}_1 \circ \text{track}_\Gamma(f, g) = f \circ \text{pr}_1 \quad (7)$$

$$\begin{array}{ccc} \Gamma & \xrightarrow{f} & \Gamma \\ \downarrow \text{pr}_1 & \searrow & \downarrow \text{pr}_1 \\ \partial\Gamma & \xrightarrow[\text{track}(f, g)]{} & \partial\Gamma \end{array}$$

证明. 对所有 $(\gamma, \varphi) \in \partial\Gamma$ ：

$$\begin{aligned} (\text{pr}_1 \circ \text{track}_\Gamma(f, g))(\gamma, \varphi) &= \text{pr}_1(f(\gamma), \varphi \circ g) \\ &= f(\gamma) \\ &= (f \circ \text{pr}_1)(\gamma, \varphi) \end{aligned} \quad \square$$

定理 5. track_Γ 是从 $\mathfrak{T}_\Gamma$ 到 $\partial\Gamma \rightarrow \partial\Gamma$ 的么半群同态。即

1. $\text{track}_\Gamma(\text{id}_\Gamma, \text{id}_\Gamma) = \text{id}_{\partial\Gamma}$ ；

2. 对所有 $(f_1, g_1), (f_2, g_2) \in \mathfrak{T}_\Gamma$,

$$\text{track}_\Gamma((f_1, g_1) \circ (f_2, g_2)) = \text{track}_\Gamma(f_1, g_1) \circ \text{track}_\Gamma(f_2, g_2) \quad (8)$$

证明.

1. 单位元被映到单位元, 因为 $\text{track}_\Gamma(\text{id}_\Gamma, \text{id}_\Gamma)(\gamma, \varphi) = (\gamma, \varphi \circ \text{id}_\Gamma) = (\gamma, \varphi)$ 。

2. 对于乘法, 取任意 $(\gamma, \varphi) \in \partial\Gamma$:

$$\begin{aligned} (\text{track}_\Gamma(f_1, g_1) \circ \text{track}_\Gamma(f_2, g_2))(\gamma, \varphi) &= \text{track}_\Gamma(f_1, g_1)(f_2(\gamma), \varphi \circ g_2) \\ &= (f_1(f_2(\gamma)), \varphi \circ g_2 \circ g_1) \\ &= \text{track}_\Gamma(f_1 \circ f_2, g_2 \circ g_1)(\gamma, \varphi) \end{aligned} \quad \square$$

定义 6. 在 $\partial\Gamma$ 上定义变换 recover_Γ :

$$\begin{aligned} \text{recover}_\Gamma : \partial\Gamma &\rightarrow \partial\Gamma \\ \text{recover}_\Gamma = (\gamma, \varphi) &\mapsto (\varphi(\gamma), \text{id}_\Gamma) \end{aligned} \quad (9)$$

该变换把恢复函数 φ 作用于当前状态 γ , 并将 φ 重置为恒等映射。下图说明了一串效应 $\text{track}(f_1, g_1), \dots, \text{track}(f_n, g_n)$ 施加到 $\partial\Gamma$ 之后, recover 如何将上下文恢复到其初始状态:

$$\begin{array}{ccccccc} \Gamma & \xrightarrow{f_1} & \Gamma & \dots & \Gamma & \xrightarrow{f_n} & \Gamma \\ \downarrow \text{track} & & \downarrow \text{track} & & \downarrow \text{track} & & \downarrow \text{track} \\ \partial\Gamma & \xrightarrow{f'_1} & \partial\Gamma & \dots & \partial\Gamma & \xrightarrow{f'_n} & \partial\Gamma \\ & & & & \downarrow \text{recover} & & \\ & & & & \partial\Gamma & & \end{array}$$

该图表明, 被追踪的效应之后接上 recover , 会把初始效应上下文带回其自身。每一个追踪步骤所保持的, 正是恢复本身的结果, 无论从何种状态出发:

定理 7. 对每个 $(\gamma, \varphi) \in \partial\Gamma$ 以及每个满足 $g(f(\gamma)) = \gamma$ 的对 (f, g) ,

$$\text{recover}_\Gamma(\text{track}_\Gamma(f, g)(\gamma, \varphi)) = \text{recover}_\Gamma(\gamma, \varphi) \quad (10)$$

证明.

$$\begin{aligned} \text{recover}_\Gamma(\text{track}_\Gamma(f, g)(\gamma, \varphi)) &= \text{recover}_\Gamma(f(\gamma), \varphi \circ g) \\ &= (\varphi(g(f(\gamma))), \text{id}_\Gamma) \\ &= (\varphi(\gamma), \text{id}_\Gamma) = \text{recover}_\Gamma(\gamma, \varphi) \end{aligned} \quad \square$$

一串对无需单独的论证。设 $(f_1, g_1), \dots, (f_n, g_n)$ 从 (γ, φ) 出发按顺序施加, 并写 $\delta_0 = \gamma$, $\delta_i = f_i(\delta_{i-1})$ 。由定理 5, 复合 $\text{track}_\Gamma(f_n, g_n) \circ \dots \circ \text{track}_\Gamma(f_1, g_1)$ 是扭曲复合 $(f_n \circ \dots \circ f_1, g_1 \circ \dots \circ g_n)$ 。

$\cdots \circ g_n$) 的 track_Γ ; 若对每个 i 都有 $g_i(\delta_i) = \delta_{i-1}$, 则 $(g_1 \circ \cdots \circ g_n)(\delta_n) = \delta_0 = \gamma$ 。因此该对在 γ 处满足定理 7 的前提, 对定理的一次应用给出

$$\text{recover}_\Gamma((\text{track}_\Gamma(f_n, g_n) \circ \cdots \circ \text{track}_\Gamma(f_1, g_1))(\gamma, \varphi)) = \text{recover}_\Gamma(\gamma, \varphi) \quad (11)$$

取 $(\gamma, \varphi) = (\gamma_0, \text{id}_\Gamma)$, 恢复把以这种方式到达的每个状态都带回 $(\gamma_0, \text{id}_\Gamma)$ 。满足 $g \circ f = \text{id}_\Gamma$ 的对在每个状态都满足前提。

恢复通过量 $\varphi(\gamma)$ 来读取状态, 我们把 $\varphi(\gamma) = \gamma_0$ 称为 $\partial\Gamma$ 中一个状态的健全性不变量 (soundness invariant)。

3.1.2 可逆效应函数

上一节的 $\text{track}/\text{recover}$ 模型把逆视为先验给定的: $\text{track}_\Gamma(f, g)$ 在见到任何上下文状态之前就固定了 g , 因此一个 g 必须服务于该效应所施加的每个状态。然而在实践中, 每个效应的逆并非先验已知: 它必须由调用者在效应施加处提供。此外, recover 是全有或全无的: 它不能在保留其他效应的同时选择性地撤销某一个效应。为解决这两个问题, 我们在输入端和输出端同时增强模型:

1. 在输入端, 我们不仅变换 Γ , 还同时返回一个逆函数, 使逆在效应施加处被提供: $\Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)$, 即 $\Gamma \rightarrow \partial\Gamma$;
2. 在输出端, 我们不仅变换 $\partial\Gamma$, 还同时返回一个逆函数, 使一个效应可以在保留其他效应的同时被撤销: $\partial\Gamma \rightarrow \partial\Gamma \times (\partial\Gamma \rightarrow \partial\Gamma)$, 即 $\partial\Gamma \rightarrow \partial^2\Gamma$ 。

这一增强保持了输入与输出之间的结构性一致, 因此我们仍然可以定义保持 track 数学性质的相应理论。由此得到的类型是效应函数 $\mathfrak{E}_\Gamma$ 及其带见证的精细化 $\mathfrak{E}_\Gamma^*$:

定义 8. 定义效应函数 $\mathfrak{E}_\Gamma$ 与带见证的效应函数 $\mathfrak{E}_\Gamma^*$ 为:

$$\begin{aligned} \mathfrak{E}_\Gamma &:= \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \\ \mathfrak{E}_\Gamma^* &:= (e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma)) \\ &\quad \times ((\gamma : \Gamma) \rightarrow ((\delta : \Gamma) \times (g : \Gamma \rightarrow \Gamma) \times ((\delta, g) = e(\gamma) \rightarrow g(\delta) = \gamma))) \end{aligned} \quad (12)$$

其中 $e(\gamma)$ 产生一对 (δ, g) , 表示:

- $\delta : \Gamma$ 是新的上下文;
- $g : \Gamma \rightarrow \Gamma$ 是当前效应的逆函数。

$\mathfrak{E}_\Gamma^*$ 的一个元素按状态选择其逆, 约束 $g(\delta) = \gamma$ 把这个选择限定为在效应施加处撤销该效应, 而在其它所有地方都不约束 g 。单个满足 $g \circ f = \text{id}_\Gamma$ 的 g 一次性地在每个状态都满足该约束, 并通过 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 诱导出 $\mathfrak{E}_\Gamma^*$ 的一个元素, 定理 11 表明这是一个同态。该约束可以用下面的交换图直观表示, 它确保 e 产出的逆确实在 e 被施加的状态处反转该变换:

$$\begin{array}{ccc}
\Gamma & \xrightarrow{f} & \Gamma \\
\downarrow e & \swarrow g & \downarrow \text{pr}_2, \text{pr}_1 \\
& \partial\Gamma &
\end{array}$$

由于效应函数 $\mathfrak{E}_\Gamma$ 不再是上下文上的自同态，它们不能直接复合。因此我们为效应复合定义一个新运算：

定义 9. 给定函数 $f, g \in \mathfrak{E}_\Gamma$ ，定义它们的效应复合 $f \diamond g$ 为：

$$\begin{aligned}
f \diamond g &: \Gamma \rightarrow \partial\Gamma \\
f \diamond g &= \gamma \mapsto \mathbf{let} \ (\delta, s) = g(\gamma) \ \mathbf{in} \\
&\quad \mathbf{let} \ (\varepsilon, t) = f(\delta) \ \mathbf{in} \\
&\quad (\varepsilon, s \circ t)
\end{aligned} \tag{13}$$

定理 10. 效应复合把 $\mathfrak{T}_\Gamma$ 的幺半群结构搬运到 $\mathfrak{E}_\Gamma$ 上。即

1. $(\mathfrak{E}_\Gamma, \diamond)$ 是一个幺半群，单位元为 $\eta_\Gamma := \gamma \mapsto (\gamma, \text{id}_\Gamma)$ ；
2. 赋值 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 是从 $\mathfrak{T}_\Gamma$ 到 $\mathfrak{E}_\Gamma$ 的幺半群同态。

证明.

1. 结合律与单位元律由 $\circ$ 的相应律逐分量地得出。
2. 写 $e_i = \gamma \mapsto (f_i(\gamma), g_i)$ ；则 $(e_1 \diamond e_2)(\gamma) = (f_1(f_2(\gamma)), g_2 \circ g_1)$ ，它是 $(f_1, g_1) \circ (f_2, g_2)$ 的像，且 $(\text{id}_\Gamma, \text{id}_\Gamma)$ 被映到 η_Γ 。□

定理 11. 见证在效应复合下得以保持，且统一的逆在每个状态都提供见证。即

1. $\mathfrak{E}_\Gamma^*$ 是 $\mathfrak{E}_\Gamma$ 的子幺半群；
2. 定理 10 的同态把每个满足 $g \circ f = \text{id}_\Gamma$ 的对都送入 $\mathfrak{E}_\Gamma^*$ 。

证明.

1. 单位元属于 $\mathfrak{E}_\Gamma^*$ ，因为 $\text{id}_\Gamma(\gamma) = \gamma$ 。对于封闭性，取 $f, g \in \mathfrak{E}_\Gamma^*$ 及任意 $\gamma \in \Gamma$ ，设 $(\delta, s) = g(\gamma)$ ， $(\varepsilon, t) = f(\delta)$ ，于是 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ 。则 $s(\delta) = \gamma$ 且 $t(\varepsilon) = \delta$ ，因此 $(s \circ t)(\varepsilon) = s(\delta) = \gamma$ 。
2. $g \circ f = \text{id}_\Gamma$ 在每个 γ 处给出 $g(f(\gamma)) = \gamma$ ，因此这样的对的像在每个状态都被见证。□

正如 track 把 Γ 上的变换对提升到 $\partial\Gamma$ 一样，我们定义 effect 把 $\mathfrak{E}_\Gamma$ 提升到 $\mathfrak{E}_{\partial\Gamma}$ ：

定义 12. 定义效应函数变换 effect_Γ 为:

$$\begin{aligned} \text{effect}_\Gamma : \mathfrak{E}_\Gamma &\rightarrow \partial\Gamma \rightarrow \partial^2\Gamma \\ \text{effect}_\Gamma = e &\mapsto (\gamma, \varphi) \mapsto \text{let } (\delta, g) = e(\gamma) \text{ in} \\ &((\delta, \varphi \circ g), \text{track}_\Gamma(g, \text{pr}_1 \circ e)) \end{aligned} \quad (14)$$

由于 $\text{effect}_\Gamma(e)$ 本身就是 $\mathfrak{E}_{\partial\Gamma}$ ，它所返回的是按定义 8 向上一层解读的逆。这个逆本身是对交换效应两个方向所得之对的 track 。普通的追踪规则再次适用：撤销效应本身也是一种效应，它用 g 变换状态，而撤销它的方式就是再次执行该效应，这正是 $\text{pr}_1 \circ e$ 所做的。因此，该逆复合到交给它的累加器上，正如 track 所规定的那样。

我们现在可以证明与 track 性质类似的 effect 性质。

定理 13. effect_Γ 保持 $\diamond$ 运算。即 $\forall f, g \in \mathfrak{E}_\Gamma$:

$$\text{effect}_\Gamma(f) \diamond \text{effect}_\Gamma(g) = \text{effect}_\Gamma(f \diamond g) \quad (15)$$

证明. 取任意 $(\gamma, \varphi) \in \partial\Gamma$ ，设 $(\delta, s) = g(\gamma)$ ， $(\varepsilon, t) = f(\delta)$ ，于是 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ 且 $\text{pr}_1 \circ (f \diamond g) = (\text{pr}_1 \circ f) \circ (\text{pr}_1 \circ g)$ 。则

$$\begin{aligned} (\text{effect}_\Gamma(f) \diamond \text{effect}_\Gamma(g))(\gamma, \varphi) &= ((\varepsilon, \varphi \circ s \circ t), \text{track}_\Gamma(s, \text{pr}_1 \circ g) \circ \text{track}_\Gamma(t, \text{pr}_1 \circ f)) \\ &= ((\varepsilon, \varphi \circ s \circ t), \text{track}_\Gamma(s \circ t, (\text{pr}_1 \circ f) \circ (\text{pr}_1 \circ g))) \\ &= \text{effect}_\Gamma(f \diamond g)(\gamma, \varphi) \end{aligned}$$

其中第一步在 (γ, φ) 与 $(\delta, \varphi \circ s)$ 处展开定义 12，第二步是定理 5，第三步折叠定义 12。□

两个层级之间的关系如下图所示。其上三角是 e 的见证条件（按定义 8），下三角则是 e' 是否像 e 那样被见证的问题。

$$\begin{array}{ccccc} & & \Gamma & \xrightarrow{f} & \Gamma \\ & \downarrow \text{effect} & \searrow e & \swarrow g & \downarrow \text{pr}_2, \text{pr}_1 \\ & & \partial\Gamma & \xrightarrow{f'} & \partial\Gamma \\ & \downarrow e' & \swarrow g' & \searrow & \downarrow \text{pr}_2, \text{pr}_1 \\ & & \partial^2\Gamma & & \end{array}$$

在层级之间，投影 pr_1 把每个被提升的映射与它所提升的映射联系起来，正如它在定理 4 中对 track_Γ 所做的那样。

定理 14. 设 $e \in \mathfrak{E}_\Gamma$ ，写 $f := \text{pr}_1 \circ e$ ，并设 $e' := \text{effect}_\Gamma(e)$ ，其前向映射为 $f' := \text{pr}_1 \circ e'$ 。则

1. $\text{pr}_1 \circ f' = f \circ \text{pr}_1$;
2. 对每个 $(\gamma, \varphi) \in \partial\Gamma$ ，在此见证的提升逆 $g' := \text{pr}_2(e'(\gamma, \varphi))$ 与逆 $g := \text{pr}_2(e(\gamma))$ 满足 $\text{pr}_1 \circ g' = g \circ \text{pr}_1$ 。

证明.

1. 由定义 12, $f'(\gamma, \varphi) = (f(\gamma), \varphi \circ g)$, 其状态为 $f(\gamma) = (f \circ \text{pr}_1)(\gamma, \varphi)$ 。

2. 这是把定理 4 应用于 $g' = \text{track}_\Gamma(g, f)$ 。 □

下三角是否闭合, 由计算提升逆的返回值来确定:

定理 15. 设 $e \in \mathfrak{E}_\Gamma^*$ 并写 $f := \text{pr}_1 \circ e$ 。固定 $(\gamma, \varphi) \in \partial\Gamma$, 设 $(\delta, g) = e(\gamma)$, 并把 $\text{effect}_\Gamma(e)$ 在 (γ, φ) 处的值记为 (Δ, g') 。则

$$g'(\Delta) = (\gamma, \varphi \circ g \circ f) \quad (16)$$

状态被精确恢复。当且仅当 $g \circ f = \text{id}_\Gamma$ 时, 累加器也同时被恢复, 等价地 $\text{effect}_\Gamma(e) \in \mathfrak{E}_{\partial\Gamma}^*$; 并且在任何情况下都有 $(\varphi \circ g \circ f)(\gamma) = \varphi(\gamma)$, 因此健全性不变量得以保持。

证明. 由定义 12, $\Delta = (\delta, \varphi \circ g)$ 且 $g' = \text{track}_\Gamma(g, f)$, 所以

$$g'(\Delta) = (g(\delta), \varphi \circ g \circ f) = (\gamma, \varphi \circ g \circ f)$$

这里用到了 $g(\delta) = \gamma$ 。属于 $\mathfrak{E}_{\partial\Gamma}^*$ 要求该值在每个输入处都等于 (γ, φ) ; 取 $\varphi = \text{id}_\Gamma$ 可把累加器的相等转化为 $g \circ f = \text{id}_\Gamma$, 而该条件反过来又对每个 φ 给出累加器的相等。最后, $(\varphi \circ g \circ f)(\gamma) = \varphi(g(\delta)) = \varphi(\gamma)$ 。 □

因此, 只有当在 γ 处见证的逆在每个状态都撤销 f 时, 下三角才闭合, 所以 effect_Γ 不把 $\mathfrak{E}_\Gamma^*$ 带入 $\mathfrak{E}_{\partial\Gamma}^*$ 。在任何情况下都成立的是在 γ 处的一致: $\text{recover}_\Gamma(g'(\Delta)) = \text{recover}_\Gamma(\gamma, \varphi)$, 这正是定理 7 对累加器要求的全部内容, 因此撤销不会触碰恢复目标。

按施加顺序的相反顺序撤销效应不需要任何额外的东西, 因为每个逆此时都恰好遇到它自身施加所产生的状态:

定理 16. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 从 $(\gamma_0, \text{id}_\Gamma)$ 出发按顺序施加, 并按相反顺序撤销。则

1. 每次撤销都恢复其施加时所针对的上下文状态;

2. 每个中间状态都满足健全性不变量。

证明. 每一步都是一次施加或一次撤销。施加把 (γ, φ) 带到 $(\delta, \varphi \circ g)$, 其中 $g(\delta) = \gamma$, 因此由定理 7 保持 $\varphi(\gamma)$, 而定理 7 的前提正是 $\mathfrak{E}_\Gamma^*$ 的见证。按相反顺序撤销使每个逆都遇到它自身施加所产生的状态, 因此由定理 15, 该次撤销精确恢复前一状态, 同时也保持 $\varphi(\gamma)$; 这两个结论都不依赖于该逆所接收的累加器。 □

3.1.3 效应的独立性

在效应自身施加所产生的状态处撤销效应, 这是定理 16 所覆盖的; 在其它任何状态处撤销一个效应, 则是本小节所覆盖的。有两种情形需要后者。一种逆可能在后续效应仍然在位时运行, 这正相当于从运行中的系统中撤回一个组件; 一个序列还可能交错多个组件的效应, 每个组件都保存自己的逆, 从而使一个组件的逆被另一个组件的施加所分隔。在这两种

情形中，逆所遇到的都是被外来效应移动过的状态，而它是否仍然撤销它被构造出来要撤销的东西，是一个交换性问题：必须交换的，是一个效应所能执行的每个变换与另一个效应所能执行的每个变换，前向映射与产出的逆都一样。单一的累加器解决不了这两种情形，因为 φ 是一个复合，它按一种顺序并一次性运行它所持有的全部逆。

定义 17. 对于效应函数 $e \in \mathfrak{E}_\Gamma$ ，变换幺半群 $\mathfrak{M}(e)$ 是 $\Gamma \rightarrow \Gamma$ 中由 e 的前向映射连同 e 产出的每个逆所生成的子幺半群， $\mathfrak{M}(e)$ 的生成元就是该生成集的元素：

$$\mathfrak{M}(e) := \langle \{\text{pr}_1 \circ e\} \cup \{\text{pr}_2(e(\gamma)) \mid \gamma \in \Gamma\} \rangle \quad (17)$$

由对 $(f, g) \in \mathfrak{T}_\Gamma$ 诱导的效应以 f 和 g 为生成元，它产出的逆在每个状态都是 g 。

引理 18. 交换性在生成元上即可确定，且 $\diamond$ 不会扩大任何变换幺半群。即

1. 若 $\mathfrak{M}(e_1)$ 的每个生成元都与 $\mathfrak{M}(e_2)$ 的每个生成元交换，则 $\mathfrak{M}(e_1)$ 的每个元素都与 $\mathfrak{M}(e_2)$ 的每个元素交换；
2. $\mathfrak{M}(e_1 \diamond e_2) \subseteq \langle \mathfrak{M}(e_1) \cup \mathfrak{M}(e_2) \rangle$ 。

证明.

1. 与 $\mathfrak{M}(e_2)$ 的每个生成元都可交换的映射构成 $\Gamma \rightarrow \Gamma$ 的一个子幺半群，因为 id_Γ 在其中，且只要 f 与 f' 在其中， $f \circ f'$ 就在其中。由前提，该子幺半群包含 $\mathfrak{M}(e_1)$ 的生成元，因而包含 $\mathfrak{M}(e_1)$ 。固定 $f \in \mathfrak{M}(e_1)$ ，与 f 可交换的映射同样构成一个包含 $\mathfrak{M}(e_2)$ 的生成元、因而包含 $\mathfrak{M}(e_2)$ 的子幺半群。
2. 由定义 9， $e_1 \diamond e_2$ 的前向映射是 $(\text{pr}_1 \circ e_1) \circ (\text{pr}_1 \circ e_2)$ ，它在任意状态产出的逆是 $s \circ t$ ，其中 s 由 e_2 产出， t 由 e_1 产出。因此 $\mathfrak{M}(e_1 \diamond e_2)$ 的每个生成元都是这两者生成元的复合。 $\square$

定义 19. 当以下条件成立时，效应函数 $e_1, e_2 \in \mathfrak{E}_\Gamma$ 称为独立的：

1. 一方的每个变换都与另一方的每个变换交换，

$$\forall f \in \mathfrak{M}(e_1), g \in \mathfrak{M}(e_2). f \circ g = g \circ f \quad (18)$$

2. 任何一方的变换都不干扰另一方产出的逆，

$$\forall g \in \mathfrak{M}(e_2), \gamma \in \Gamma. \text{pr}_2(e_1(g(\gamma))) = \text{pr}_2(e_1(\gamma)) \quad (19)$$

并且交换 e_1 与 e_2 后同样成立。

当对每个 $l \neq l'$ ， e_l 与 $e_{l'}$ 都独立时，族 $(e_l)_{l \in L}$ 称为两两独立的。族可以重复同一个效应函数，而让一个效应与自身独立，就等于要求 $\mathfrak{M}(e)$ 可交换。

对于由对 (f_1, g_1) 和 (f_2, g_2) 诱导的效应，条款 (1) 由引理 18(1) 即四对 $f_1, f_2; g_1, g_2; f_1, g_2; g_1, f_2$ 的交换，而条款 (2) 直接成立，因为诱导效应在每个状态只产生一个逆。在 $\diamond$ 下

的交换是另一个不同的性质。 $e_1 \diamond e_2 = e_2 \diamond e_1$ 所等同的，是两个顺序的复合前向映射彼此相等、两个顺序的复合逆彼此相等，每个逆都在其自身施加所产生的状态进入复合；独立性所关联的则是一个效应的每个变换与另一个效应的每个变换，包括前向映射与外来逆的配对。

在独立性之下，逆可以在后续效应移动过的状态运行，而它在那里撤回的只是自身的贡献，别无其他：

定理 20. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 两两独立，并从 γ_0 出发按顺序施加。写 $f_i := \text{pr}_1 \circ e_i$ ，设 $\delta_i := f_i(\delta_{i-1})$ ，其中 $\delta_0 := \gamma_0$ ，并设 $g_i := \text{pr}_2(e_i(\delta_{i-1}))$ 为 e_i 在施加处产出的逆。固定 j ，并写 $\delta'_i := (f_i \circ \dots \circ f_{j+1})(\delta_{j-1})$ 为去掉 e_j 后序列的状态，于是 $\delta'_j = \delta_{j-1}$ 。则对每个满足 $j \leq u \leq n$ 的 u ，

1. $\delta_u = f_j(\delta'_u)$ 且 $g_j(\delta_u) = \delta'_u$ ；
2. 每个满足 $i > j$ 的 e_i 在 δ'_{i-1} 处产出与它在 δ_{i-1} 处产出的相同的逆 g_i 。

证明.

1. 第一个方程是对 u 的归纳。在 $u = j$ 时它读作 $\delta_j = f_j(\delta_{j-1})$ ，这正是 δ_j 的定义。归纳步中， $\delta_{u+1} = f_{u+1}(\delta_u) = f_{u+1}(f_j(\delta'_u)) = f_j(f_{u+1}(\delta'_u)) = f_j(\delta'_{u+1})$ ，中间的等号是定义 19 的条款 (1) 对 e_{u+1} 与 e_j 的应用，由于 $u+1 > j$ ，它们是该族中不同的效应。对于第二个方程，条款 (1) 把 g_j 穿过在 e_j 之后施加的前向映射移出，使 e_j 的见证得以在它唯一成立的那个状态处使用：

$$g_j(\delta_u) = (g_j \circ f_u \circ \dots \circ f_{j+1})(\delta_j) = (f_u \circ \dots \circ f_{j+1})(g_j(f_j(\delta_{j-1}))) = \delta'_u$$

最后一个等号依赖于 $g_j(f_j(\delta_{j-1})) = \delta_{j-1}$ ，这正是定义 8 要求 e_j 在 δ_{j-1} 处具有的见证。

□

证明 (续). 2. 由第 (1) 款可知状态 δ 为 $f_{i-1}(\delta'_i)$ ，且 $f_{i-1} \in \mathfrak{M}(e_j)$ ，故对 e_j 与 e_i 应用定义 19 的第 (2) 款，得 $\text{pr}_2(e_i(f_j(\delta'_{i-1}))) = \text{pr}_2(e_i(\delta'_{i-1}))$ 。□

第 (1) 款刻画了某个逆所到达的状态：即同一序列在效应从未被应用的情况下本会到达的状态，无论其后应用了哪些效应。第 (2) 款刻画了其效应在该处所持有的逆；二者合在一起，使该定理得以再次应用于更短的序列：

推论 21. 设 $e_1, \dots, e_n \in \mathfrak{E}_\Gamma^*$ 两两独立，并从 γ_0 出发按顺序应用；设 $g_1, \dots, g_n$ 同上。在 δ_n 处按 $\{1, \dots, n\}$ 的任意排列的顺序应用这 n 个逆，均到达 γ_0 。

证明. 对 n 作向下归纳。设该排列以 j 开头。由定理 20 第 (1) 款，在 δ_n 处应用 g_j 到达 δ'_n ——即删去 e_j 后该序列所到达的状态；由定理 20 第 (2) 款，其余效应在该处产生的逆正是手中的 g_i 。该序列两两独立（它是子族），因此归纳假设适用于它以及排列的其余部分；空序列到达 γ_0 。□

LIFO 顺序正是这样的一个排列，且定理 16 在其中无需任何假设即可逆回。独立性换来的是所有其他顺序，以及随之而来的、交错多个组件的序列——第 4.4.2 节将这一点带到整个系统的迹上。

这些构造合在一起构成了可逆效应： $\mathcal{E}_T^*$ 中的每个效应函数都显式提供自身的逆，effect 在效应上下文 $\partial\Gamma$ 上追踪这些逆，而 $\diamond$ 运算在保持可逆性的同时将它们复合。它们带来的是局部时间可组合性——所谓“局部”，是指该保证只针对单个组件自身的效应来解读。我们将此视为如下判据：对组件所应用的每一序列效应函数，累加器都能恢复其开始时所在的上下文（定理 7），且逆回该序列会使每个逆获得其自身应用所作用的状态（定理 16）。加载组件就是应用这样的序列并把其逆累加到 φ 中；卸载组件就是应用 φ 。

该判据遗漏了两点，而这两点都只有在多个组件同时运行之后才会出现：脱离累加器所施加的顺序进行逆回，以及交错他人效应的序列。独立性提供了这两点（推论 21）；它是关于效应本身的条件，而非构造的性质——第 3.3.2 节指出了满足该条件的方法纪律，第 4.4.2 节则在整系统迹的层面上解读这一保证。当独立性不成立时，顺序就不得不由别处承担：在单个组件内部由累加器承担，无论效应如何它都按 LIFO 顺序逆回（第 4.3.2 节）；在组件之间则由声明的余效应承担，它使一次激活相对于另一次激活有序（第 4.3.1 节）。

3.2 反应式余效应

空间可组合性是指组件能够相互声明依赖、系统能够在运行时解析、提供并撤销这些依赖的能力。这要求每当共享上下文发生变化时都重新评估依赖的满足情况，从而使组件在其依赖可用时激活、在其依赖被撤销时停用。因此，我们把组件的依赖建模为一个规范，并针对该规范把上下文的每次变化分类为激活、停用或中性。针对规范进行分类，正是检测满足状态变化之处；对这一分类作出响应，正是驱动激活与停用之处。我们称这类余效应为反应式（reactive）的：通过对上下文变化进行分类并据此驱动激活与停用，正确的余效应排序成为结构性保证。

3.2.1 余效应上下文

传统的控制反转（IoC）容器 [38] 通常把依赖建模为简单的键值映射。本节把 IoC 形式化为一种余效应上下文，它与可逆效应协同，为动态组合提供数学基础。

定义 22. 给定类型族 $\mathcal{V} : K \rightarrow \text{Type}$ ，把余效应上下文定义为如下依赖偏函数类型：

$$\Sigma := (k : K) \multimap \mathcal{V}_k \quad (20)$$

其中 $\sigma : \Sigma$ 是一个有限偏函数，为每个 $k \in \text{dom}(\sigma) \subseteq K$ 指派一个类型为 $\mathcal{V}_k$ 的值。我们记：

- $\sigma(k)$ 表示应用（当 $k \in \text{dom}(\sigma)$ 时有定义）；
- $\sigma[k \mapsto v]$ 表示在 k 处绑定 v 、其余各处与 σ 一致的表；
- $\sigma \setminus k$ 表示限制（当 $k \in \text{dom}(\sigma)$ 时有定义）；
- $k \in \text{dom}(\sigma)$ 表示成员关系。

使用类型族 $\mathcal{V}$ 可保证每个依赖键 k 关联一个特定的值类型 $\mathcal{V}_k$ ，从而为依赖访问提供静态类型安全。扩展与限制都带有前提条件，由下述运算强制：依赖不能重复提供（扩展要求

$k \notin \text{dom}(\sigma)$), 也不能在缺席时被撤销 (限制要求 $k \in \text{dom}(\sigma)$)。违反前提条件会以错误形式发出信号且不产生任何迁移, 因此描述实际发生的迁移的效应代数可以原封不动地应用于这些运算。偏好将失败内化的读者可以把下文中每个 $\Sigma \rightarrow \Sigma$ 读作 $\Sigma \rightarrow \text{Maybe}(\Sigma)$, 并在 Maybe 单子中复合 (第 2.1 节), 代价是把每个恒等映射替换为运算定义域上的偏恒等映射。基于此上下文结构, 我们定义两个核心运算:

定义 23. Σ 上的 get 与 set 运算定义如下:

$$\begin{aligned} \text{get} &: (k : K) \rightarrow \Sigma \rightarrow \mathcal{V}_k \\ \text{get} &= k \mapsto \sigma \mapsto \sigma(k) \\ \text{set} &: (k : K) \times \mathcal{V}_k \rightarrow \Sigma \rightarrow \Sigma \times (\Sigma \rightarrow \Sigma) \\ \text{set} &= (k, v) \mapsto \sigma \mapsto (\sigma[k \mapsto v], \lambda \sigma'. \sigma' \setminus k) \end{aligned} \quad (21)$$

其中 $\text{get}(k)$ 以 $k \in \text{dom}(\sigma)$ 为前提条件, $\text{set}(k, v)$ 以 $k \notin \text{dom}(\sigma)$ 为前提条件。

值得注意的是, $\text{set}(k, v)$ 的类型为 $\mathfrak{E}_{\Sigma}^*$, 恰好是余效应上下文上的效应函数。因此我们可以直接套用第 3.1 节的效应机制: effect_{Σ} 自动提供依赖注册的追踪与恢复。这正是反应式余效应与可逆效应之间的协同: 余效应运算就是效应, 而效应是可逆的。

get 交给组件的只是一个值, 而组件能用该值做什么, 取决于该键处的余效应提供了什么。因此, 一个键所携带的不仅仅是值类型:

定义 24. 键 k 处的余效应是一个三元组 $(\mathcal{V}_k, \simeq_k, \mathcal{A}_k)$, 其中 $\mathcal{V}_k$ 是定义 22 中的值类型, $\simeq_k$ 是 $\mathcal{V}_k$ 上的一个等价关系, k 处的值即按此关系进行比较 (第 3.3.2 节), 而 $\mathcal{A}_k$ 是一组余效应运算, 即绑定在 k 处的值向持有它的组件所提供的运算。运算 $a \in \mathcal{A}_k$ 带有参数类型 X_a 与结果类型 B_a , 并且只作用于该值本身:

$$a : X_a \rightarrow \mathcal{V}_k \rightarrow \mathcal{V}_k \times (\mathcal{V}_k \rightarrow \mathcal{V}_k) \times B_a \quad (22)$$

其前两个分量构成 $\mathcal{V}_k$ 上的效应函数 (按定义 8 的要求得到见证), 第三个分量则是一个结果。每个运算都要求尊重 $\simeq_k$: 在 $\simeq_k$ 相关的值上, 它要么二者皆有定义, 要么二者皆无定义; 在有定义处, 它产生 $\simeq_k$ 相关的后继、把 $\simeq_k$ 相关的值再次映到 $\simeq_k$ 相关的值的逆, 以及相等的结果。运算通过其提升作用于余效应上下文:

$$a_{\Sigma}(x)(\sigma) := \text{let } (v, g, b) = a(x)(\sigma(k)) \text{ in } (\sigma[k \mapsto v], \lambda \sigma'. \sigma'[k \mapsto g(\sigma'(k))], b) \quad (23)$$

该式在 $k \in \text{dom}(\sigma)$ 时有定义, 其前两个分量是 Σ 上的效应函数。

把 k 的一个运算类型化为作用在 $\mathcal{V}_k$ 上, 正是为了把它限制在 k 处的绑定上: 该提升只读写这一绑定, 而让其他每个键保持原样, 因此无需额外的边条件来说明这一点。在隔离生效之处, 它所能到达的绑定正是域所解析到的那个绑定 (定义 28), 共享同一域的两个键共享同一个绑定。行为依赖于另一个键的运算, 会把该键的值读入其参数 X_a ; 下一小节的反应式纪律会保证, 只要读取了该值的组件仍在运行, 该值就保持不变 (定理 63)。

3.2.2 规范与通知

前面的定义描述了单个依赖如何被注册和访问。然而, 访问一个缺席的依赖是运行时失败。因此, 组件应当在其声明的全部依赖都就绪之后才激活, 而不是乐观地访问它们、在某

个依赖缺失时失败。这引出两个问题：组件的声明依赖是否被联合满足，以及当该状态发生变化时系统应如何响应。余效应上下文 Σ 带有一种自然的观测结构，使这两个问题都可处理：对任意余效应规范 $d \subseteq K$ ，定义满足谓词：

$$\sigma \models d := \forall k \in d. k \in \text{dom}(\sigma) \quad (24)$$

该谓词是可判定的（因为 $\text{dom}(\sigma)$ 是有限的）。由于对 σ 的所有修改都经由效应函数（其逆会恢复先前的定义域），满足状态的变化在每个效应边界处都是可检测的。这就是反应性的代数基础：效应系统保证每一个余效应变化都被观测到。

定义 25. 余效应规范即：

$$\mathcal{D}_\Sigma := \text{Set}(K) \quad (25)$$

它表示组件向环境声明的依赖集合。

使这一规范具有反应性的，是它对状态迁移的分类方式。任何把 σ 变换为 σ' 的效应，都可以由规范 $d \in \mathcal{D}_\Sigma$ 根据 d 的满足状态是否改变来分类：

定义 26. 给定余效应规范 $d \subseteq K$ 与状态 $\sigma, \sigma' \in \Sigma$ ，定义：

$$\text{notify}_d(\sigma, \sigma') := \begin{cases} \text{activating}, & \text{if } \sigma \not\models d \wedge \sigma' \models d \\ \text{deactivating}, & \text{if } \sigma \models d \wedge \sigma' \not\models d \\ \text{neutral}, & \text{otherwise} \end{cases} \quad (26)$$

这是良定义的，因为 $\sigma \models d$ 是可判定的，且所有状态迁移都由效应函数中介。反应性不变量是：激活型迁移触发组件效应的执行（带完整的效应追踪），而停用型迁移则通过应用累加器触发恢复。这些迁移的精确操作语义取决于它们与控制流的相互作用，将在第 4 节中展开。

set 与 **notify** 共同带来的是局部空间可组合性——“局部”的含义与前面相同，即该保证只针对单个组件自身的余效应来解读。我们将此视为如下判据：组件只在满足其规范的状态下激活，因此它永远不会读取缺席的绑定；上下文的每个变化都会针对该规范分类，因此满足状态的丧失会在其发生处被检测到并驱动一次停用。这两半都直接来自上述定义：满足性是在组件将要激活之处检查的前提条件，而 notify_d 在每个迁移处都有定义。

该判据覆盖了余效应排序的一个方向，而非另一个方向。若组件 A 提供键 k ，组件 B 声明 $k \in d_B$ ，那么 B 只能在 A 激活并提供 k 之后激活，因为 $\sigma \models d_B$ 要求 $k \in \text{dom}(\sigma)$ 。反方向则不成立：卸载 A 会把 k 从 $\text{dom}(\sigma)$ 中移除，从而破坏 B 的满足状态；但单凭一个通知，既不能使 k 在 B 自身的拆除需要它的期间保持可读，也不能把 A 的恢复推迟到 B 完成之后。把一次撤销排在其所导致的各次停用之后，是对其他组件而非行动者本身的条件，因此它属于该保证的全局形态；第 4.3.1 节提供了实现它所需的手段。

3.2.3 隔离与拦截

基本的余效应上下文 Σ 建模了一张扁平的依赖表。然而在实践中，系统可能需要对不同组件把不同的值绑定到同一个逻辑依赖上。本节用两种机制扩展余效应上下文：余效应隔离（同一个键在不同上下文中解析出不同的值）与余效应拦截（依赖访问上的横切行为）。

实现 (Realization) . 这两种机制与 `get`、`set` 的不同之处在于它们作用的对象。一次提供写入每个组件都读取的共享表，因此它是该表上的一个效应，并携带撤销它的逆。隔离与拦截则调整一个键在某个上下文下对组件如何解析，而让表本身保持原样。把运算类型化为效应，固定了其指称——一个后继状态配上逆——但并不固定其实现；实现决定了该逆如何被执行。

定义 27. 上下文上的效应函数有两种实现：

- **就地实现** (in-place realization) 原地修改上下文并返回一个非平凡逆；后继与输入别名，恢复时运行该逆以撤销修改。
- **派生实现** (derived realization) 保持输入不变，返回一个从它派生出的新上下文，以恒等映射作为其逆；恢复时丢弃该派生上下文。从另一个上下文派生出的上下文，正是定义 32 的递归结构所携带的。

在纯函数式设定下，两者重合；命令式宿主可以按运算任选其一，第 5.1.2 节实现了两者。隔离与拦截被直接赋予派生实现：它们各自产生一个新上下文，其自身的表与继承来的表不同，因此下文把两者类型化为上下文到上下文的映射，而非效应函数。共享表中没有任何变化，所以没有需要追踪的逆，定义 12 也无从提升；恢复则把派生上下文连同它所携带的调整一并丢弃。对派生表的赋值会覆盖继承表在该键处持有的任何内容，这正是两个运算都不携带前提条件的原因。

余效应隔离. 通过引入隔离域，余效应隔离允许同一个依赖在不同上下文中绑定到不同的值。这在多租户系统、测试环境和组件沙箱中有广泛的应用。

定义 28. 定义带隔离的余效应上下文为：

$$\Sigma_{\text{iso}} := (K \rightarrow R) \times ((r : R) \rightarrow \mathcal{V}_r) \quad (27)$$

它可以表示为一对 (ρ, σ) ，其中：

- $\rho : K \rightarrow R$ 是隔离域表，为每个被隔离的键指派一个域标识符； $\text{dom}(\rho)$ 之外的键解析到其自身的域，因此那里我们记 $\rho(k) = k$ ($R \supseteq K$)；
- $\sigma : (r : R) \rightarrow \mathcal{V}_r$ 是依赖表，一个从域标识符到类型化值的依赖偏函数。

这种两层映射结构把逻辑层与存储层解耦，使依赖访问具有上下文感知性。访问键 k 时，系统先解析 $\rho(k)$ 得到域标识符 r ，再访问 $\sigma(r)$ 获取实际值。

定义 29. Σ_{iso} 上的 `get`、`set` 与 `isolate` 运算如下：

$$\begin{aligned} \text{get} &: (k : K) \rightarrow \Sigma_{\text{iso}} \rightarrow \mathcal{V}_{\rho(k)} \\ \text{get} &= k \mapsto (\rho, \sigma) \mapsto \sigma(\rho(k)) \\ \text{set} &: (k : K) \times \mathcal{V}_{\rho(k)} \rightarrow \Sigma_{\text{iso}} \rightarrow \Sigma_{\text{iso}} \times (\Sigma_{\text{iso}} \rightarrow \Sigma_{\text{iso}}) \\ \text{set} &= (k, v) \mapsto (\rho, \sigma) \mapsto ((\rho, \sigma[\rho(k) \mapsto v]), \lambda(\rho', \sigma'). (\rho', \sigma' \setminus \rho'(k))) \\ \text{isolate} &: K \times R \rightarrow \Sigma_{\text{iso}} \rightarrow \Sigma_{\text{iso}} \\ \text{isolate} &= (k, r) \mapsto (\rho, \sigma) \mapsto (\rho[k \mapsto r], \sigma) \end{aligned} \quad (28)$$

其中 `get` 与 `set` 携带定义 23 的前提条件沿 ρ 迁移后的版本, 即 $\rho(k) \in \text{dom}(\sigma)$ 与 $\rho(k) \notin \text{dom}(\sigma)$ 。`isolate`(k, r) 派生的上下文把域 r 指派给 k , 并原样继承依赖表, 因此已被隔离的键会被重新指派而不是被拒绝。

余效应隔离机制本质上实现了一个运行时即席多态 (ad-hoc polymorphism) 系统。通过隔离域标识符, 同一个依赖键可以在不同上下文中解析出完全不同的值, 而且这种多态可以在运行时动态调整。与传统的依赖注入相比, 余效应隔离提供了更细粒度的控制, 能够为特定组件定制隔离; `set` 仍然是效应函数 ($\mathfrak{E}_{\Sigma_{\text{iso}}}^*$), 因此继承了可逆性, 而 `isolate` 不需要任何逆——它派生一个上下文, 而非写入共享表。

余效应拦截. 第二种机制——余效应拦截——把横切元数据附加到依赖访问上, 在不修改依赖值的前提下增加行为。这种元数据既可以由上下文携带, 也可以由组件声明, 因此我们同时扩展余效应上下文与余效应规范:

定义 30. 定义带拦截的余效应上下文与规范 (specification) 为:

$$\begin{aligned}\Sigma_{\text{inter}} &:= ((k : K) \rightarrow \mathcal{M}_k) \times ((k : K) \rightarrow (\mathcal{M}_k \rightarrow \mathcal{V}_k)) \\ \mathfrak{D}_{\text{inter}} &:= (k : K) \rightarrow \mathcal{M}_k\end{aligned}\tag{29}$$

上下文 Σ_{inter} 是一个二元组 (ι, σ) : ι 是安装在上下文本身的上下文携带元数据, 默认情况下为空 ϵ_k ; σ 把每个键 k 映射到从元数据 $\mathcal{M}_k$ 到值 $\mathcal{V}_k$ 的提供者函数。规范 $d \in \mathfrak{D}_{\text{inter}}$ 携带组件声明的元数据, 为每个键指定其元数据 $d(k)$, 其中 $\text{dom}(d)$ 充当依赖集合。每个键都为其元数据配备一个幺半群 $(\mathcal{M}_k, \oplus_k, \epsilon_k)$: 合并 $\oplus_k$ 以 ϵ_k (空元数据) 为单位元, 且满足结合律。

定义 31. Σ_{inter} 上的 `get`、`set` 与 `intercept` 操作定义为:

$$\begin{aligned}\text{get} &: (k : K) \times \mathcal{M}_k \rightarrow \Sigma_{\text{inter}} \rightarrow \mathcal{V}_k \\ \text{get} &= (k, \mu) \mapsto (\iota, \sigma) \mapsto \sigma(k)(\mu \oplus_k \iota(k)) \\ \text{set} &: (k : K) \times (\mathcal{M}_k \rightarrow \mathcal{V}_k) \rightarrow \Sigma_{\text{inter}} \rightarrow \Sigma_{\text{inter}} \times (\Sigma_{\text{inter}} \rightarrow \Sigma_{\text{inter}}) \\ \text{set} &= (k, \psi) \mapsto (\iota, \sigma) \mapsto ((\iota, \sigma[k \mapsto \psi]), \lambda(\iota', \sigma'). (\iota', \sigma' \setminus k)) \\ \text{intercept} &: (k : K) \times \mathcal{M}_k \rightarrow \Sigma_{\text{inter}} \rightarrow \Sigma_{\text{inter}} \\ \text{intercept} &= (k, \nu) \mapsto (\iota, \sigma) \mapsto (\iota[k \mapsto \iota(k) \oplus_k \nu], \sigma)\end{aligned}\tag{30}$$

其中 `get` 与 `set` 携带定义 23 对提供者表的前置条件, 即 $k \in \text{dom}(\sigma)$ 与 $k \notin \text{dom}(\sigma)$ 。`intercept`(k, ν) 所派生的上下文把 ν 合并到在 k 处继承的元数据上, 并原样继承提供者表。

当携带规范 d 的组件访问键 k 时, 系统求值 $\sigma(k)(d(k) \oplus_k \iota(k))$: 组件声明的元数据与上下文携带的元数据 ι 合并, 再将提供者函数作用于合并结果。这种合并遵循各键自身的语义 (例如标量字段被覆盖、集合值字段取并集), 并且是右偏的 (right-biased), 因此 $\iota(k)$ 具有优先权, 可以覆盖组件的声明, 从而让外围上下文无需修改组件即可约束组件使用余效应 (例如第 6.3 节)。

3.3 上下文范式 (The Context Paradigm)

第 3.1 节与第 3.2 节各自作用于一个上下文: 前者将其作为效应的载体, 后者将其作为余效应的载体, 而一个同时承载两者的单一上下文应当是什么样, 则尚付阙如。本节为该统

一给出一个具体的构造，从余效应中组装出观测等价，以补足第 3.1.3 节所遗留的效应独立性，并论证由此得到的上下文类型本身构成一种编程范式。

3.3.1 统一上下文 (Unified Context)

对上下文 Γ 而言，效应上下文 $\partial\Gamma$ (第 3.1 节) 提供了一个更高层的抽象，它承载上一层级的上下文与该层级的累加器 (定义 2)。把这一结构递归化，并与余效应上下文 Σ 相结合，便得到如下类型：

定义 32. 上下文类型 Γ_∞ 定义为：

$$\Gamma_\infty := \mu\Gamma. \Gamma \times (\Gamma \rightarrow \Gamma) \times \Sigma \quad (31)$$

其中三个投影分别是：

- Γ ：当前上下文状态 (递归的)；
- $\Gamma \rightarrow \Gamma$ ：累加器，它恢复该层级的效应；
- Σ ：携带依赖信息的余效应上下文。

在此定义下，效应映射 $\mathfrak{E}_{\Gamma_\infty}$ 将上下文映到自身，从而把 ∂ -塔 (∂ -tower) 统一为单一的自相似类型。余效应上下文 Σ 在结构上被整合进来：依赖操作 (set、get) 作用于 Σ ，而累加器追踪它们的逆转。由于 Σ 底层的类型族 $\mathcal{V}$ 不受约束，系统需要在组件间共享的任何状态都可以编码为带有适当值类型的依赖—— Σ 涵盖所有共享的可变状态，而不仅是组件间的依赖。组件与其环境之间的每一次交互都经过这单一实体。

分层组合。 Γ_∞ 的递归结构支持分层控制：父上下文聚合多个子层级的效应，形成树形的控制结构，在保持模块化的同时支持统一的跨层级管理。这一效应变换实现了一种字面意义上的“即插即用” (plug-in) 隐喻：

- 加载组件对应于执行其效应 (插入)；
- 卸载组件对应于恢复其效应 (拔出，且不影响其他运行中的组件)；
- 层级中不同层级的组件可独立加载与卸载；父上下文聚合并管理其全部子级的效应，从而支持任意深度的嵌套组合。

3.3.2 观测等价 (Observational Equivalence)

第 3.1 节的恢复保证所断言的是状态相等 (定理 7)，这是一种理想化，因为物理状态无法恢复成它原本的样子。例如，free 将一个内存块交还给分配器，却并未恢复堆在 malloc 之前的布局；生成式名称也不会被丢弃它的逆所恢复，因为下一次创建会抽取一个全新的名称 [39]。因此，第 3 节的相等应理解为模等价 $\simeq$ 的相等，而我们取 $\simeq$ 为观测等价：当没有任何观察者能够区分两个状态时，它们相关。比较行为而非表示，是通向程序等价的既定路

径 [40]，而这种比较所产生的关系取决于观察者被赋予何种可供其使用的东西 [41]。上下文观察者被赋予的，是上下文所携带的余效应；每个余效应都自带一个等价关系（定义 24），因此上下文上的关系是由这些等价关系组装而成的。组装它是本小节的任务，而按它取商，正是获得第 3.1.3 节所要求的独立性之所在。

定义 33. 当两个余效应上下文把相同的键绑定到相关的值时，它们相关；当两个上下文状态的余效应投影相关时，这两个状态相关：

$$\begin{aligned}\sigma \simeq \sigma' &:= \text{dom}(\sigma) = \text{dom}(\sigma') \wedge \forall k \in \text{dom}(\sigma). \sigma(k) \simeq_k \sigma'(k) \\ \gamma \simeq \gamma' &:= \sigma_\gamma \simeq \sigma_{\gamma'}\end{aligned}\tag{32}$$

其中 σ_γ 记 γ 的余效应投影（定义 32）。

状态中未被任何键绑定的部分由此被遗忘，而正是这种遗忘使定理 7 得以在模 $\simeq$ 的意义下成立：除非某个键绑定它们，否则上述例子中的堆布局与生成式名称都位于该关系之外。第 3.2.2 节对 $\simeq$ 所要求的性质是随之而来的，而非预先假设的。相关的状态具有相同的定义域，因此它们在满足谓词 $\sigma \models d$ 与定义 26 的分类 notify_d 上一致，并且反应性是 $\Sigma/\simeq$ 的一个性质。

称该关系为观测的，是对每个 $\simeq_k$ 的一个断言：它所区分的至多不超过 k 的操作所能分辨的。值的观察者运行这些操作并读取其结果。

定义 34. 设 V 携带一簇按定义 24 的意义的操作 $\mathcal{A}$ ，并以 $\mathfrak{M}(a)$ 记效应函数 $a(x)$ 在一切参数 $x : X_a$ 上的变换幺半群（定义 17）。 $\mathcal{A}$ 上的一个测试 (test) 是幺半群 $\mathfrak{M}(a)$ ($a \in \mathcal{A}$) 的生成元上的一个有限字，其中每个字母都作用于它之前的字母所留下的值；其结果是那些作为操作前向映射的字母沿途产出的结果，而在前置条件失败处无定义。当 $\mathcal{A}$ 上的每个测试在 v 与 v' 两处要么均有定义、要么均无定义，并在两处产生相同的结果时，称值 $v, v' : V$ 不可区分，记为 $v \approx_{\mathcal{A}} v'$ 。

引理 35. 不可区分性是操作所尊重的最粗关系。即：

1. $\mathcal{A}$ 的每个操作都按定义 24 的意义尊重 $\approx_{\mathcal{A}}$ ；
2. 被 $\mathcal{A}$ 的每个操作所尊重的每个等价关系都包含于 $\approx_{\mathcal{A}}$ 。

因此， $\simeq_k$ 的每个可容许选择都包含于 $\approx_{\mathcal{A}_k}$ ，且 $\approx_{\mathcal{A}_k}$ 本身是可容许的。

证明.

1. 设 $v \approx_{\mathcal{A}} v'$ ，并设 $a \in \mathcal{A}$ 被施加于某一参数。在一个测试前添加一个字母仍然是一个测试，因此前向映射所到达的值不可区分，任何一个产出的逆从不可区分的参数出发所到达的值也同样不可区分；单字母测试给出在两者处均有定义或均无定义，且结果相等。
2. 设 R 是这样一个等价关系，且 $v R v'$ 。测试的每个字母都是某操作的前向映射或产出的逆，尊重性把 R 沿两者中的任何一个传递下去，使所到达的值保持相关、每个字母处的结果保持相等。因此每个测试在 v 与 v' 处都一致。 $\square$

把 $=$ 全部替换为 $\simeq$ 本身并不足够，因为效应函数除返回状态外还返回一个逆，而被 $\simeq$ 视为同一的两个状态还必须产出同样被 $\simeq$ 视为同一的逆。

定义 36. 当

$$\forall \gamma, \gamma' \in \Gamma. \gamma \simeq \gamma' \Rightarrow f(\gamma) \simeq f(\gamma') \quad (33)$$

时，称映射 $f : \Gamma \rightarrow \Gamma$ 尊重 $\simeq$ 。当两个映射在每个状态处都一致时，它们相关；当 $\partial\Gamma$ 中的两个二元组在两个分量上都相关时，这两个二元组相关：

$$\begin{aligned} f \simeq g &:= \forall \gamma \in \Gamma. f(\gamma) \simeq g(\gamma) \\ (\delta, g) \simeq (\delta', g') &:= \delta \simeq \delta' \wedge g \simeq g' \end{aligned} \quad (34)$$

尊重 $\simeq$ 的映射就是能下降到 $\Gamma/\simeq$ 的映射，而由 $\simeq$ 关联的两个映射就是在那里下降到同一映射的两个映射。效应函数两者都需要：前者使它所计算的状态在商空间上被确定，后者使它返回的逆在商空间上被确定。

定义 37. 把定义 8 按模 $\simeq$ 来读： $e \in \mathfrak{E}_\Gamma$ 属于 $\mathfrak{E}_\Gamma^*$ ，当 e 作为映射 $\Gamma \rightarrow \partial\Gamma$ 尊重 $\simeq$ ，并且，记 $(\delta, g) = e(\gamma)$ ，对每个 $\gamma \in \Gamma$ 有：

1. $g(\delta) \simeq \gamma$;
2. g 尊重 $\simeq$ 。

把 $\simeq$ 取为 Γ 上的相等，即恢复定义 8。

引理 38. 若 $\mathfrak{E}_\Gamma^*$ 按定义 37 来读，则第 3.1 节断言的所有状态相等在把 $=$ 替换为 $\simeq$ 后仍然成立，并且从 $(\gamma_0, \text{id}_\Gamma)$ 出发可达的每个状态的累加器都尊重 $\simeq$ 。

证明. 累加器是逆的复合，其中每个逆按定义 37(2) 尊重 $\simeq$ ，而尊重 $\simeq$ 的映射的复合仍然尊重 $\simeq$ ，基例为 id_Γ 。于是第 3.1 节的证明原样成立——尊重性正是把关系穿过逆传递的性质：由 $g_2(\delta_2) \simeq \delta_1$ 与 $g_1(\delta_1) \simeq \gamma$ ，尊重性给出 $(g_1 \circ g_2)(\delta_2) \simeq \gamma$ ，这正是每次逆复合所采取的步骤，而定理 7 的健全性不变量据此步骤读作 $\varphi(\gamma_0) \simeq \gamma_0$ 。□

定义 19 所要求的交换性也由同一引理按模 $\simeq$ 来读，而正是这样读才使它成为可达到的：两个操作可能留下被 $\simeq_k$ 视为同一的值，却仍然算作交换。比起它们的提升所诱导的效应函数，它对两个操作还多要求一样东西——操作还会产生一个结果。

定义 39. 当操作 a 与 a' 的提升作为效应函数（定义 19）在每一对参数处都独立，且其中任何一个的变换都不干扰另一个所产出的结果时，称 a 与 a' 独立：

$$\forall x : X_a, g \in \mathfrak{M}(a'_\Sigma), \sigma \in \Sigma. \text{pr}_3(a_\Sigma(x)(g(\sigma))) = \text{pr}_3(a_\Sigma(x)(\sigma)) \quad (35)$$

交换 a 与 a' 后同样成立，其中 $\mathfrak{M}(a_\Sigma)$ 记提升 $a_\Sigma(x)$ 在一切参数上的变换幺半群，正如定义 34 以 $\mathfrak{M}(a)$ 记操作本身的变换幺半群一样。当 $\mathcal{A}_k$ 中任意两个操作都独立时——操作与其自身也被视为独立——称键 k 是可交换的 (commutative)。

在不同键之间，该条件无条件成立。

定理 40. 位于不同键上的操作是独立的。

证明. 设 $a \in \mathcal{A}_k$, $a' \in \mathcal{A}_{k'}$, 且 $k \neq k'$. 按定义 24, $\mathfrak{M}(a_\Sigma)$ 的每个生成元都具有 $\sigma \mapsto \sigma[k \mapsto u(\sigma(k))]$ 的形式, 其中 u 是 $\mathcal{V}_k$ 上的映射——它或者是前向映射的提升, 或者是产出的逆的提升; a' 在 k' 处同理。这样的两个映射交换, 因为各自只读写一个键、且两个键互不相同, 而引理 18(1) 把交换性从生成元扩展到两个幺半群。对于第二个条件, a_Σ 在 σ 处所产出的——逆与结果皆然——由 $\sigma(k)$ 决定, 而 $\mathfrak{M}(a'_\Sigma)$ 的每个生成元都不动 $\sigma(k)$ 。□

值是从中独立添加、移除条目的表的键是可交换的, 路由或事件监听器的注册是代表性情形: 两个注册无论以何种顺序进行, 都会留下一个对每个测试都给出相同回答的表, 且其中任何一个注册都可以在另一个仍然有效时被撤销。值为有序链的键则不然, 因为插在另一个中间件之前的中间件会看到不同的请求, 而且两种顺序中的任何一种都不能在不干扰另一种的情况下被撤销。开头示例中的分配器按其接口所公布的内容而分野: 当它所发放的句柄不被该键的任何操作比较时, $\simeq_k$ 可以把两个相差一个句柄重命名的堆关联起来——这正是 CompCert 关联一个程序及其翻译的内存状态的方式 [42]——此时分配是可交换的; 而当地址是按相等比较的结果时, 没有任何可容许的 $\simeq_k$ 能使两种分配顺序一致, 此时该键不是可交换的。

组件执行的是一个操作序列, 其中每个操作都可能依赖于它之前的操作所产出的结果, 而下面这个定理所谈论的正是这种形状的效应函数。

定义 41. 余效应介导的效应函数构成最小集合 $\mathfrak{E}_\Sigma^A \subseteq \mathfrak{E}_\Sigma$, 它包含单位 η_Σ , 并在如下构造下封闭: 对键 k 、操作 $a \in \mathcal{A}_k$ 、参数 $x : X_a$ 以及成员族 $(e_b)_{b \in B}$,

$$\sigma \mapsto \mathbf{let} (\delta, s, b) = a_\Sigma(x)(\sigma) \mathbf{in} \mathbf{let} (\varepsilon, t) = e_b(\delta) \mathbf{in} (\varepsilon, s \circ t) \quad (36)$$

仍是成员。每个阶段执行一个操作, 并根据结果选择其后的内容, 因此参数可以依赖于已经获得的结果。一个成员中出现的操作, 就是其各阶段在每一种结果选择下所执行的操作。

定理 42. 设 $e_1, e_2 \in \mathfrak{E}_\Sigma^A$, 并设每一个同时出现两者操作的键都是可交换的 (定义 39)。则 e_1 与 e_2 独立 (定义 19)。

证明. 对定义 41 的构造归纳可知, $\mathfrak{M}(e_i)$ 位于由 e_i 中出现的操作的生成元所生成的子幺半群中: 单位生成平凡幺半群, 而一个阶段是 $a_\Sigma(x)$ 与某个成员的 $\diamond$ -复合, 对此可应用引理 18(2)。

对于定义 19 的第 (1) 款, 由引理 18(1), 只需 e_1 中出现的某个操作的生成元与 e_2 中出现的某个操作的生成元交换即可。当两个操作位于不同的键时, 这是定理 40; 当它们位于同一个键时, 该键同时承载两个操作, 按假设是可交换的。

对于第 (2) 款, 取 $g \in \mathfrak{M}(e_2)$ ——它是 e_2 中出现的操作的生成元的复合——并对 e_1 的构造进行归纳。单位在每个状态处产出 id_Σ 。在一个阶段, 设 $(\delta, s, b) = a_\Sigma(x)(\sigma)$, $(\varepsilon, t) = e_b(\delta)$, 则该阶段在 σ 处产出 $s \circ t$ 。操作独立性每次作用于 g 的一个生成元, 在 $g(\sigma)$ 处再次产出 s 与 b , 因此选择相同的续延 e_b ; 第 (1) 款把它所运行的状态置于 $g(\delta)$, 而归纳假设在那里再次产出 t 。因此该阶段在 $g(\sigma)$ 处产出 $s \circ t$ 。□

组件与其环境之间的每一次交互都经过上下文, 且类型族 $\mathcal{V}$ 不受约束, 因此系统可以把它在组件间共享的每个位置都绑定到自己的某个键上 (第 3.3.1 节)。于是组件的效应函数就是某个余效应介导的效应函数沿余效应投影的提升, 独立性随之传递到该提升——它的变换只移动投影。第 3.1.3 节遗留的假设由此得到满足, 随之而来的是整个组件系统的时

间可组合性。

这一分解所划分的，是计算的可交换部分与对顺序敏感的部分。可交换部分由效应承载：组件按任务要求的任何顺序执行它们，而推论 21 按系统认为方便的任何顺序逆转它们，任意两个组件互不约束。对顺序敏感的部分由余效应承载：一个操作不交换的键，其顺序必须从效应外部施加，而施加顺序有两个位置可用。在单个组件内部，由累加器施加它，以 LIFO 顺序逆转效应的一切（定理 16）。在组件之间，由声明的余效应施加它：一个组件提供另一个组件所声明的依赖，且提供先于声明的满足（第 3.2.2 节）。于是可组合性在组件的粒度上获得，而非在单个效应的粒度上获得——这正是第 4 节工作的尺度。

该定理有两个局限值得指出。把每个共享位置绑定到键上，是范式的纪律而非构造的性质，因此系统无法具体化为余效应的位置位于第 6.1 节的边界之外，也随之位于定理之外。键的可交换性是键所公布的接口的性质，因此满足它是提供该键的组件的义务，而非消费它的组件的义务。

3.3.3 上下文范式的定位 (Situating the Context Paradigm)

编程范式在处理副作用的方式上存在根本差异。谱系的两端由两个既定的极点界定：

显式状态线程化（函数式）。为保持引用透明性（referential transparency），纯函数式语言把副作用建模为状态上的显式变换。状态单子（State monad） $S \rightarrow (A, S)$ [23] 将环境穿过每一次计算。这种方法带来强的组合性保证：效应在类型中可见，且适于等式推理。然而，它也带来显著的易用性代价：调用链中的每个函数都必须接受并返回状态参数，即使它只是原样传递状态。随着效应维度（日志、配置、I/O）的增多，单子堆叠或效应处理器样板代码会不断增殖。

隐式变更（命令式/面向对象）。主流命令式语言允许组件修改共享状态，并在调用点不经显式声明地访问依赖。在效应一侧，一个代表性例子是 React 的 `useEffect` 钩子：它在组件的内部纤程上注册一个持久的副作用，但效应目标与注册机制都不以显式参数出现——识别依赖于隐藏运行时状态中的调用顺序位置。在余效应一侧，Java 的服务定位器模式（例如 Spring 的 `ApplicationContext.getBean(...)`）在运行时从进程级注册表取回依赖，在每个调用点都要求空值检查与类型转换；依赖关系是隐式的，且散落在整个代码库中。更一般地说，要理解 `f()` 如何修改系统或依赖系统，需要传递性地阅读其实现。重构因此变得脆弱：移动或删除一个调用可能会悄然破坏远处的不变量。

上下文范式结合了函数式方法的可追溯性与命令式方法的易用性。效应与余效应都通过一个显式的上下文参数来中介。因此，每个操作都可归因于它被调用时所处的特定上下文，进而归因于该上下文所属的组件。

在结合两个极点的优点之外，上下文范式还让开发者逐一处理每个效应与依赖，并自动地把它们组合进系统的行为之中。对于可逆效应，开发者提供每个原子操作的逆，而任何复合操作的逆都随之由复合给出（第 3.1 节），因此组件的拆卸（teardown）由其加载派生而来，而非与加载并列地另行编写。对于反应式余效应，组件只声明它需要的依赖，运行时自动解析并重新接线这些依赖（第 3.2 节），在提供者被添加、移除或替换时使它们保持一致地接线。在这两个方向上，本需依靠开发者纪律的正确性变成了该范式的结构性性质。

4 动态组合演算

第 3 节仅以局部形式确立了空间可组合性与时间可组合性。要把它们推广到整个系统，需要把系统分解为组件（components），每个组件将一条余效应规范与一个见证效应函数（witnessed effect function）配对，使得与共享环境的每一次交互都可归因于其中之一。以下各节为该分解赋予操作语义，并以全局形式确立空间可组合性与时间可组合性。第 4.1 节和第 4.2 节给出能够为生命周期立规则的最小演算——该演算把每条迁移都视为原子的、即时的且不会失败的；第 4.3 节去掉这三个假设，针对迁移可能运行的每个方向各去掉一次原子性，容纳运行时在迁移开始与结束之间插入的各种控制流形态，从而得到真实运行时实现的演算；第 4.4 节确立该演算的元理论，即保持性、全局时间与空间可组合性、进展性与合流性。

4.1 组件与纤程

本节固定规则所作用的对象：组件；纤程（fiber）——组件的实例化，携带其自身的生命周期状态；以及注册表（registry）——保存状态所携带的纤程，并从中读出余效应上下文。

组件。一个组件以三元组给出，其余效应侧分为它从环境中读取的内容与它向环境提供的内容。

定义 43. 在携带效应与余效应的上下文 Γ （定义 32）上的组件定义为：

$$\mathfrak{C}_\Gamma := \mathfrak{D}_\Gamma \times \mathfrak{P}_\Gamma \times \mathfrak{E}_\Gamma^* \quad (37)$$

表示三元组 (d, p, e) ，其中：

- $d : \mathfrak{D}_\Gamma$ 是定义 25 的余效应规范，声明需要从环境中获得的依赖；
- $p : \mathfrak{P}_\Gamma := \text{Set}(K)$ 是提供，声明该组件可能提供的余效应键，并且 p 之外的任何键都不是其效应函数会写入的键；
- $e : \mathfrak{E}_\Gamma^*$ 是定义 8 的见证效应函数，定义组件处于激活状态时贡献的效应，以及撤销这些效应的逆。

这两条声明是同一接口的两个方向： d 是组件从环境中读取的内容， p 是组件写入环境的内容；第 4.2 节不允许同一注册表中的两个纤程提供彼此相交的提供集。全文的下标都取在 Γ 上，余效应上下文是 Γ 的投影之一（定义 32），因此定义 25 的 $\mathfrak{D}_\Sigma$ 在此写成 $\mathfrak{D}_\Gamma$ 。

提供的互不相交是本章与第 3.2.3 节分道扬镳之处。定义 28 的隔离允许一个键通过领域表（realm table）解析，因此两个纤程可以在不同的领域中提供相同的键；携带领域的演算会把不相交性放宽为领域内的不相交性，并依据声明键的纤程所在的领域来解析所声明的键。我们在此不引入领域，而是在一个共享领域中读取所有键——这正是使上面的不相交性成为正确条件、并使每个键的提供者唯一（定义 45）的原因。它所限制的是组件可被实

例化的次数：提供集非空的组件在同一时刻只有一个线程，因此下文中的大量实例化都是针对不提供任何内容的组件——即只消费、或只注册其他组件的组件的常见情形。

运行系统中的组件会随时间被激活和停用，因此它携带一个生命周期状态，迁移 (transition) 则是把它从一个生命周期状态移动到另一个生命周期状态的东西：一次激活执行 e ，在上下文上累积副作用；一次停用应用累加器以恢复上下文。其最简单的形式是图 1 的两状态模型，第 4.2 节为它给出规则；第 4.3 节在允许每种控制流特性时对它进行细化。

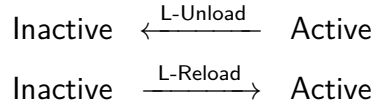


图 1 | 基础组件生命周期

线程。一个组件可以被多次实例化，每次实例化都携带其自身的生命周期状态。我们把这样的实例化称为线程。一个线程记录：产生它的组件、它在哪个线程之下被实例化、它提供的余效应，以及它处于生命周期的哪个阶段。

定义 44. 固定一个线程名集合 $\mathfrak{N}$ 。实例化组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 的线程是元组 $\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$ ，其中

- $d : \mathfrak{D}_\Gamma$ 、 $p : \mathfrak{P}_\Gamma$ 与 $e : \mathfrak{C}_\Gamma^*$ 是定义 43 的余效应规范、提供与效应函数；
- $\pi : \mathfrak{N} \cup \{\text{root}\}$ 是父线程，即该线程在其下被实例化的那个线程，或是根标记 root ；
- $\sigma : \Sigma$ 是线程自身的余效应表（定义 22），在它激活之前为空，并由其效应在运行时写入；
- $\tau : \{\perp, \top\}$ 是退役标志，在新线程中为 $\perp$ ，一旦编排器已使该线程退役即为 $\top$ ；
- $\theta : \Theta_\Gamma$ 是生命周期状态，在第 4.2 节的两状态模型中为

$$\Theta_\Gamma := \text{Inactive} \mid \text{Active}(g, \omega) \quad (38)$$

其中 $g : \Gamma \rightarrow \Gamma$ 是累加器， $\omega : d \rightarrow \mathfrak{N}$ 是已提交视图 (committed view)。

已提交视图 ω 把线程声明的每个键都映射到迁移提交时提供该键的线程的名称。第 4.3 节用进行中的迁移所需的扩展替换 Θ_Γ ；定义 44 的其余部分对两者只给出一次，只是 e 要在第 4.3 节各层引入的更丰富的效应类型下读取。

注册表。一个状态按名称持有其线程，线程的标识与第 3.2 节的余效应上下文都从这种安排中读出。

定义 45. 记 $\mathfrak{F}_\Gamma$ 为 Γ 上的线程集合。状态 $\gamma \in \Gamma$ 携带一个注册表

$$F_\gamma : \mathfrak{N} \rightarrow \mathfrak{F}_\Gamma \quad (39)$$

这是一个有限偏函数，其父指针构成一棵以 root 为根的树，连同 Γ 中没有任何线程的 σ 所指名的其余部分。我们用 $\gamma(n)$ 表示 $F_\gamma(n)$ ，并在状态明确时通过给 $\gamma(n)$ 的字段加下标 n 来

缩写它, 因此 $d_n, p_n, e_n, \pi_n, \sigma_n, \tau_n, \theta_n$ 是定义 44 的各个字段, g_n, ω_n 是 θ_n 所携带的累加器与已提交视图; $\gamma[\theta_n \mapsto \theta'_n]$ 、 $\gamma[n \mapsto \langle \dots \rangle]$ 与 $\gamma \setminus n$ 分别是与 γ 在一个字段、一个线程以及一个线程的存在性上不同的状态。

线程的名称赋予它一种在其自身变更后依然存续的标识: 下面的每条规则都重写一个线程的生命周期状态而保持其他线程不动, 因此规则必须说明是哪一个; 并且有两个字段引用线程而非描述线程, 即父线程 π 与已提交视图 ω 。名称是原子: 没有规则会计算出一个名称、检查它的结构、或用相等以外的任何方式关联两个名称, 而引入一个线程只是抽取一个尚未使用的名称。这正是动态创建的局部名称 [39] 的纪律, 此处用于线程标识。

每个线程都拥有一张表意味着余效应上下文是被推导出来的而非被存储的: 它就是激活的线程共同提供的内容。

$$\sigma_\gamma := \bigcup \{ \sigma_m \mid m \in \text{dom}(F_\gamma), \theta_m = \text{Active}(-, -) \} \quad (40)$$

该并集是良定义的, 因为一个线程只写它声明的键, $\text{dom}(\sigma_n) \subseteq p_n$, 且不同线程的提供互不相交 (定义 43), 因此每个 $k \in \text{dom}(\sigma_\gamma)$ 都恰好位于一个 **Active** 线程的表中, 该线程的名称我们记作 $\text{provider}_k(\gamma) \in \mathfrak{N}$, 并称之为 k 的提供者。因此每个键都恰好有一个可能的提供者, 它由提供集决定而非由状态决定。

没有规则直接写 σ_n : 线程的提供是其自身效应函数执行的集合操作, 这些操作落在 σ_n 中, 因此已经属于 e_n 返回的状态的一部分, 并随累加器再次离开。效应中只有余效应部分以这种方式被记录, 因为只有余效应部分才是其他线程声明所针对的内容; 变更 γ 中其他位置状态的效应与其他效应一样由 g 追踪, 但没有任何线程能在规范中指名它们, 因此它们不贡献任何排序约束。

第 3.2.2 节的满足关系随后原样适用, 其中 $\gamma \models d$ 是 $\sigma_\gamma \models d$ 的缩写。一个键位于 $\text{dom}(\sigma_\gamma)$ 中, 当且仅当某个 **Active** 线程已安装它; 提供是它可能安装的键而非它已安装的键, 因此 $\gamma \models d$ 已经要求每个声明的键都有一个 **Active** 提供者。仅对 **Active** 线程取并集, 正是让一个线程能够在撤销任何东西之前就停止提供的原因, 第 4.3.1 节把这转化为排序纪律。

4.2 基础演算

本节给出图 1 的两状态生命周期的演算, 仅此而已: 每个线程与之比较的目标 (target), 以及移动它的五条规则。

目标视图。规则把每个线程与一个目标比较, 即它是否应当运行、以及应当依据对其依赖的何种解析来运行。目标并非线程单独的性质, 因为线程声明的键是针对整个状态解析的, 所以目标是该状态上的一个谓词。

定义 46. n 在 γ 处的目标视图把每个声明的键映射到它的提供者, 因此它是全映射 $d_n \rightarrow \mathfrak{N}$, 当 n 根本不应运行时为 $\perp$:

$$\text{target}_n(\gamma) := \begin{cases} \perp & \text{if } \tau_n \vee \neg(\gamma \models d_n) \\ (k \in d_n) \mapsto \text{provider}_k(\gamma) & \text{otherwise} \end{cases} \quad (41)$$

当每个线程都达到其目标视图时，状态是静默的：

$$\text{quiet}(\gamma) := \forall n \in \text{dom}(F_\gamma). \begin{cases} \text{target}_n(\gamma) = \perp & \text{if } \theta_n = \text{Inactive} \\ \text{target}_n(\gamma) = \omega_n & \text{if } \theta_n = \text{Active}(-, \omega_n) \end{cases} \quad (42)$$

目标只对两件事作出应答，别无其他：退役（通过 τ_n ）与余效应解析（通过 $\gamma \models d_n$ 和 provider_k ），每个声明的键都在定义 43 的那个共享领域中从 σ_γ 读出。

定义 44 的已提交视图与目标视图具有相同的类型，生命周期正是通过比较二者来驱动的： ω_n 是 n 据以激活的解析， $\text{target}_n(\gamma)$ 是它应当据以运行的解析，下面每条规则都在二者一致或不一致时触发。记录提供者而非值，才使这种比较可用，因为否则一个提供相等值的不同线程也会比较为相等。组件读取的值是通过该视图到达的，因为提供者的表持有那个值；实现把该映射保存在 `fiber.committed` 中，并把它的一枚哈希保存在 `fiber.target` 中（第 5.1.3 节）。

规则。基础演算把每条迁移都视为原子的、即时的且不会失败的：一次激活在一个步骤中应用其效应函数，一次停用在一个步骤中应用累加器，并且二者都能成功完成。第 4.3 节去掉这三者。

五条规则生成两个关系。编排规则（以 O- 为前缀，写作 $\gamma \Rightarrow \delta$ ）是编排器可以执行的一个动作；它的前提说明该动作何时合法，而非何时发生。生命周期规则（以 L- 为前缀，写作 $\gamma \longrightarrow \delta$ ）是系统在其前提成立时主动采取的一步。步骤序列把两者交错，下文中的 $\longrightarrow^*$ 仅指生命周期步骤。

$$\begin{array}{c} \frac{n \notin \text{dom}(F_\gamma) \quad \pi \in \text{dom}(F_\gamma) \cup \{\text{root}\} \quad (d, p, e) \in \mathfrak{C}_\Gamma \quad \forall m \in \text{dom}(F_\gamma). p \cap p_m = \emptyset}{\gamma \Rightarrow \gamma[n \mapsto \langle d, p, e, \pi, \emptyset, \perp, \text{Inactive} \rangle]} \quad \text{O-Insert} \\[10pt] \frac{n \in \text{dom}(F_\gamma)}{\gamma \Rightarrow \gamma[\tau_n \mapsto \top]} \quad \text{O-Retire} \\[10pt] \frac{\tau_n = \top \quad \theta_n = \text{Inactive} \quad \forall m. \pi_m \neq n}{\gamma \Rightarrow \gamma \setminus n} \quad \text{O-Remove} \end{array}$$

插入与退役是仅有的外部输入：编排器请求一个线程存在或不再存在，而从不直接设置它的生命周期状态。O-Retire 对线程的状态是无条件的，因为退役是一个请求，而生命周期规则才是执行它的东西。出于同样的原因，退役与移除是分离的：一个已退役但仍处于 Active 的线程必须首先被停用，若更早地移除它，则会丢弃累加器并造成泄漏。前提 $\forall m. \pi_m \neq n$ 通过先于父线程移除子线程来保持树的良好性。O-Insert 的最后一条前提正是施加单一来源纪律之处：一个键只有一个可能的提供者，因为编排器不得接纳第二个声明它的组件。

$$\begin{array}{c} \frac{\theta_n = \text{Inactive} \quad \omega = \text{target}_n(\gamma) \neq \perp \quad e_n(\gamma) = (\delta, g)}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Active}(g, \omega)]} \quad \text{L-Reload} \\[10pt] \frac{\theta_n = \text{Active}(g, \omega) \quad \text{target}_n(\gamma) \neq \omega \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Inactive}]} \quad \text{L-Unload} \end{array}$$

L-Reload 在安装逆的同时安装已提交视图；L-Unload 应用逆并丢弃已提交视图。两者都由同一种比较驱动：当线程不持有已提交视图且其目标视图不为 $\perp$ 时 L-Reload 触发，当它所持有的已提交视图不是其目标视图时 L-Unload 触发。这正是第 3.2 节的反应式纪律，从对退役与余效应都作出应答的目标中读出：每当目标视图改变时，就会发起一次迁移，而不管两者中是哪一方使它改变的。

实例化。一个组件在安装其效应时可能实例化另一个组件——这正是插件宿主在某个插件加载它自己的插件时所做的。到目前为止的规则把注册表完全留给编排规则，因此这样的实例化无处发生。一个原语给了它一个发生之处。

定义 47. 一次对 e_n 的应用（或在第 4.3.2 节适用处的某次迭代）可以注册一个组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 。它不是产生一个状态映射，而是取该组件的 O-Insert（其中 $\pi = n$ ），并以所注册线程的 O-Retire 作为其逆产出。该规则抽取一个名称（受 O-Insert 的新鲜性前提约束），并把它交给效应函数。

逆选择退役而非移除，原因在于逆必须在到达它的任何地方都能应用。O-Remove 携带前提，因此由它构造的逆可能无法应用：其子线程仍处于 Active 的父线程无法运行它的累加器，而且由于定义 46 不读取线程树，没有规则会移动那个子线程。O-Retire 以 $n \in \text{dom}(F_\gamma)$ 作为其唯一前提。它在注册被采取的那个状态留下的条目是已退役的 $\text{Inactive}(\perp)$ ，并持有一张空表——这正是引理 57 的退化条目（vestigial entry）：它与线程不存在只差在控制字段上，而且没有规则能区分二者。

使一个子线程退役会设置 τ_n ，从而把它的目标视图置为 $\perp$ ，此后普通规则把它带回 Inactive。父线程不必等待，因为 O-Retire 是无条件的，所以无论子线程是否已经离开，L-Unload 都适用于父线程。孙线程一级一级地到达：子线程自身的累加器使子线程注册的东西退役。定理 66 一起覆盖了这一级联以及第 4.3.1 节沿余效应施加的那一种级联。

限定。有了这唯一一个例外在手，就可以给出效应函数所受的纪律。它限定一次应用写什么，从而使应用它的规则能够说明所有其他变化；也限定一次应用读什么，从而使一个线程只能看到它声明的余效应，以及注册表中不多于这些的内容。对写入的限定正是使第 4.4 节能把表 1 当作它们的完整清单来读的原因。

定义 48. 映射 $f : \Gamma \rightarrow \Gamma$ 被限定于 n ，当对每个满足 $n \in \text{dom}(F_\gamma)$ 的 $\gamma \in \Gamma$ ，记 $\delta = f(\gamma)$ ，有

1. (写入) $\text{dom}(F_\delta) = \text{dom}(F_\gamma)$ ，对每个满足 $m \neq n$ 的 $m \in \text{dom}(F_\gamma)$ 有 $\delta(m) = \gamma(m)$ ，且 $\delta(n)$ 与 $\gamma(n)$ 仅在 σ_n 上不同；
2. (读取) 在 σ_n 上、在限制 $\sigma_m|_{d_n}$ （对每个 $m \in \text{dom}(F_\gamma)$ ）上、以及在没有任何线程的表所指名的状态部分上一致的两个状态，会被 f 送到在这三者上一致的状态。

效应函数 e_n 被限定于 n ，当它的每次应用（以及第 4.3.2 节适用处的它的每次迭代）要么注册一个组件（定义 47），要么使其状态映射 $\text{pr}_1 \circ e_n$ 与它所产出的逆都被限定于 n 。每个线程的效应函数都被要求限定于该线程。

一次注册写出 O-Insert 所写出的那个条目，位置在其抽取的那一个名称处，别无其他；它作为逆产出的 O-Retire 写出该名称的 τ ，别无其他。因此，任何一种应用都不会写入已

存在纤程的任何控制字段（除那一个 τ 外），并且完全不读取任何控制字段。

第 (2) 款正是组件可以读取它所声明的值得原因：这些值位于其提供者的表中，因此一个除了 σ_n 外不读取任何表的效应函数将无法使用它自己的余效应。它不得读取的是 d_n 之外的任何表，或任何控制字段——这使组件不能依据它未声明的纤程的生命周期状态进行分支。

这些规则是非确定性的：多个纤程可能持有与其目标视图不同的已提交视图，而该关系不规定它们之间的任何顺序。它们也是反应式的——只不过，没有任何规则提及调度器：步骤是规则应用的任意序列，因此对所有此类序列证明的定理，对运行时可能采用的每一种调度策略都成立。

4.3 进行中的迁移

本节在四种设置下扩展基础演算。第一种提供第 3.2 节需要而第 4.2 节无法表达的东西：一种跨越其依赖者可能占据的时间区间的停用；另外三种则去掉“迁移是原子的、即时的、不会失败的”这一理想化——真实运行时中的迁移，这三者一个都不具备。被去掉的是“整个迁移是一个步骤”这一假设，而不是“一个步骤是一条规则的一次应用”；四种设置共享一个结构后果，在此一并交代：不是步骤的迁移，在进行期间需要一个可供其占据的状态，其可能运行的每个方向各一个。

定义 49. 本节的各个生命周期状态把 Θ 替换为

$$\Theta_\Gamma := \text{Inactive}(\zeta) \mid \text{Reloading}(i, g, \omega) \mid \text{Active}(g, \omega) \mid \text{Unloading}(g, \omega, \zeta) \quad (43)$$

其中 $i : \mathfrak{C}_\Gamma^{\text{iter}*}$ 是剩余的效应迭代器（见下面的定义 51）， $g : \Gamma \rightarrow \Gamma$ 是迄今构建的累加器， $\omega : d \rightarrow \mathfrak{N}$ 是已承诺视图， $\zeta : \{\perp\} \cup \Xi$ 是结局（outcome）：Unloading 携带它作为其停用所指向的结局，Inactive 携带它作为其所达到的结局，它要么是 $\perp$ ，要么是第 4.3.4 节提供的错误集合 Ξ 中的某个错误。

当一个纤程处于携带累加器和已承诺视图的三种状态之一时，它是已安装的（installed）；当它携带错误结局时，它是已失败的（failed）：

$$\text{installed}_n(\gamma) := \theta_n \neq \text{Inactive}(-), \quad \text{failed}_n(\gamma) := \exists \xi \in \Xi. \theta_n = \text{Inactive}(\xi) \quad (44)$$

已安装的纤程 n 在 $\omega_n(k) = m$ 时把 k 解析为 m 。定义 46 的静默态（quiescence）在更宽的状态空间上解读为

$$\text{quiet}_n(\gamma) := \begin{cases} \zeta \neq \perp \vee \text{target}_n(\gamma) = \perp & \text{若 } \theta_n = \text{Inactive}(\zeta) \\ \forall n \in \text{dom}(F_\gamma). \text{target}_n(\gamma) = \omega_n & \text{若 } \theta_n = \text{Active}(-, \omega_n) \\ \perp & \text{否则} \end{cases} \quad (45)$$

第 4.1 节的定义沿用至这一状态空间，但需确定两处解读。第一，第 4.2 节的 Inactive 在 O-Insert 的结论中解读为 Inactive($\perp$)，在 O-Remove 的前提中解读为 Inactive($-$)。第二， σ_γ 仍然只对 Active 纤程的表取并集，因此迁移在任一方向上进行中的纤程，通过它所持有的

ω 读取余效应，而不提供任何自己的余效应；于是其迁移已经写入的键，尚不能成为依赖者可据以激活的键。在两状态演算中这一区分是空洞的，因为在那里每个已安装的纤程都是 Active。

图 2 画出这些状态形成的生命周期，下面四个小节给出其各条边上的规则。

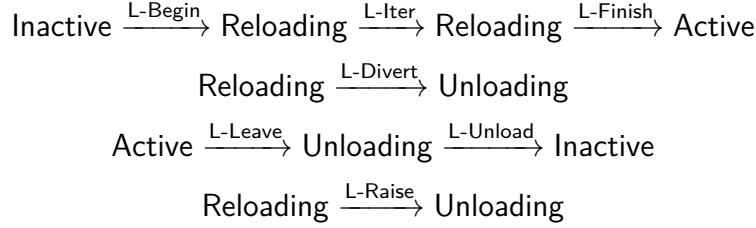


图 2：进行中的迁移的生命周期；两个迁移状态以轮廓线标出。

4.3.1 撤销

第 3.2 节要求依赖者在它们的依赖之后激活，依赖者只有在其依赖者 (dependents) 已停用之后才能撤销自己的提供。前半部分在基础演算中已经成立：激活要求 $\gamma \models d_n$ ，因此声明 k 的纤程只有在某个纤程积极提供 k 之后才能激活。后半部分才是实质性的，而且它必须提供的不仅仅是状态变化的先后次序。一个组件因提供者即将消失而被拆除时，正在运行自己的拆除 (teardown) 代码，该代码可能恰好需要正在被撤销的那个余效应；关闭连接池通常意味着把连接交还给提供它们的一方。后半部分必须保证的是：消费者在自身的整个停用过程中仍然能读取 k ，而提供者对 k 的撤销只能在此之后才生效。基础演算完全无法提供这一点：其 L-Unload 把移除提供与运行逆合为一步，二者之间没有留下任何可供消费者的拆除代码占据的时间区间。

这一层把那一步拆成两步，并用下面的条件守卫后半部分。

定义 50. 当某个其他已安装纤程把一个键解析到纤程 n 时，称 n 在 γ 处被依赖 (relied upon)：

$$\text{relied}_n(\gamma) := \exists m \in \text{dom}(F_\gamma), k \in d_m. m \neq n \wedge \text{installed}_m(\gamma) \wedge \omega_m(k) = n \quad (46)$$

$$\frac{\theta_n = \text{Active}(g, \omega) \quad \text{target}_n(\gamma) \neq \omega}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Unloading}(g, \omega, \perp)]} \text{L-Leave}$$

$$\frac{\theta_n = \text{Unloading}(g, \omega, \zeta) \quad \neg \text{relied}_n(\gamma) \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Inactive}(\zeta)]} \text{L-Unload}$$

L-Leave 记录停用的决定而不付诸行动：它让纤程停止提供余效应，同时保持其自身的已承诺视图以及所有其他人的视图不变。L-Unload 应用累加器，丢弃已承诺视图，并让纤程以它携带的结局停留在 Inactive；在 4.3.4 节提供另一种情形之前，该结局为 $\perp$ 。它是该演算中唯一应用累加器的规则。

这一先后次序的两半于是由形式的不同部分承载：可见性这一半由已承诺视图承载——L-Unload 作为其最后一个动作将其丢弃；次序这一半由前提 $\neg \text{relied}_n(\gamma)$ 承载，我们称之为守卫 (guard)，它把 k 的撤销一直按住，直到每一个把它解析到 n 的消费者都已离开为止。定理 63 将确立这两点。

守卫是按绑定而非按纤程施加的： $\text{relied}_n(\gamma)$ 检验是否有某个已承诺视图指名 n ，因此一个不声明 n 的任何键的纤程不构成阻碍，在另一个界域 (realm, 见 3.2.3 节) 中解析了 n 的某个键的纤程同样不构成阻碍。在第 4.2 节的单一来源纪律下，按绑定的解读与较粗略的检验 $\exists m \neq n, k \in d_m. \text{installed}_m(\gamma) \wedge k \in p_n$ 一致，因为在那里一个键只有一个可能的提供者。

这类守卫通常会死锁。使它免于死锁的，是 Unloading 连同 σ_γ 只对 Active 纤程取并集这一点：一旦 L-Leave 标记了 n ， n 的表就离开 σ_γ ，因此不再有任何目标视图能指名 n ，而每一个承诺于 n 的消费者自己也已在离场途中。定理 66 将这一点转化为“守卫总会解除”的断言。

守卫沿着余效应而非沿着纤程树对停用排序：父纤程可以在其某个子纤程仍处于 Unloading 时运行自己的逆，因为 relied 只谈论已承诺视图。因此父与子之间的排序弱于定理 63 对提供者与其消费者的排序；而效应在环境状态中相遇的父与子，则改由定义 60 的独立性假设支配。

4.3.2 迭代

一次激活可以依次执行多个效应，而停用必须恢复它们。我们用效应迭代器 (effect iterator) 对这样的激活建模，其每一次迭代产生修改后的上下文、一个逆和一个续体 (continuation)：

定义 51. 定义效应迭代器 $\mathfrak{E}_\Gamma^{\text{iter}}$ 和带见证的效应迭代器 (witnessed effect iterator) $\mathfrak{E}_\Gamma^{\text{iter}*}$ 为如下递归类型：

$$\begin{aligned}\mathfrak{E}_\Gamma^{\text{iter}} &:= \mu \mathfrak{J}. \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J}) \\ \mathfrak{E}_\Gamma^{\text{iter}*} &:= \mu \mathfrak{J}. (e : \Gamma \rightarrow \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J})) \\ &\quad \times ((\gamma : \Gamma) \rightarrow (\text{let } (\delta, g, o) = e(\gamma) \text{ in } g(\delta) \simeq \gamma))\end{aligned}\tag{47}$$

其中 $e(\gamma)$ 产生三元组 (δ, g, o) ，分别表示：

- δ 是新的上下文；
- g 是当前效应的逆函数；
- o 指示续体：
 - Nothing 表示迭代终止；
 - Just(i) 提供下一次迭代。

见证 (witness) 按定义 33 的 $\simeq$ 解读, 正如定义 37 解读 $\mathfrak{C}^*$ 的见证一样: 当 i 尊重 $\simeq$, 且它产生的每个 g 都尊重 $\simeq$ 并满足上述条款时, $i \in \mathfrak{C}_\Gamma^{\text{iter}}$ 就位于 $\mathfrak{C}_\Gamma^{\text{iter}*}$ 中。三元组按分量比较: **Nothing** 只与 **Nothing** 比较, **Just**(i) 与 **Just**(i') 在 $i \simeq i'$ 时比较; 迭代器上的 $\simeq$ 是满足这些条款的最大关系。把 $\simeq$ 取为 Γ 上的相等, 便恰好恢复这一解读。

效应迭代器变换 effectiter_Γ 通过递归调用把 effect_Γ 扩展到迭代器结构上:

定义 52. 定义效应迭代器变换 effectiter_Γ 为:

$$\begin{aligned} \text{effectiter}_\Gamma : \mathfrak{C}_\Gamma^{\text{iter}} &\rightarrow \partial\Gamma \rightarrow \partial^2\Gamma \\ \text{effectiter}_\Gamma = i &\mapsto (\gamma, \varphi) \mapsto \mathbf{let} \ (\delta, g, o) = i(\gamma) \ \mathbf{in} \\ &\quad \mathbf{let} \ t = \text{track}_\Gamma(g, \text{pr}_1 \circ i) \ \mathbf{in} \\ &\quad \mathbf{match} \ o \\ &\quad | \ \mathbf{Nothing} \Rightarrow ((\delta, \varphi \circ g), t) \\ &\quad | \ \mathbf{Just}(i') \Rightarrow \mathbf{let} \ (s, r) = \text{effectiter}_\Gamma(i')(\delta, \varphi \circ g) \ \mathbf{in} \ (s, t \circ r) \end{aligned} \quad (48)$$

在每一次迭代中, 逆 g 按应用顺序复合到 φ 上, 因此累加器 $\varphi \circ g_1 \circ \dots \circ g_k$ 在应用时自然按后进先出 (LIFO) 顺序恢复效应。由于 effectiter_Γ 与 effect_Γ 落在同一个 $\partial\Gamma \rightarrow \partial^2\Gamma$ 中, 迭代器本身就是一个效应, 凡效应可用之处它都可用。组件的整个激活就是这样的一个用途——本节其余部分将对此加以形式化——而且实现 (implementation) 在每一个变更点 (mutation site) 都接受一个迭代器 (见 5.1.1 节)。Maybe($\mathfrak{C}_\Gamma^{\text{iter}}$) 续体在任意两个连续迭代之间提供一个边界, 在该边界处上下文就是迄今各次迭代所形成的那个上下文, 而累加器恰好恢复这些迭代而不恢复更多。在这个意义上, 效应迭代器是具体化的定界续体 (delimited continuation) ——主流语言通过 `yield` 运算符 [43] 暴露的结构——因此该模型直接对应到它们已经提供的生成器 (generator) 上。

在演算中, 定义 44 的 e_n 从此以后按 $\mathfrak{C}_\Gamma^{\text{iter}*}$ 解读; 用迭代器替换原子效应函数, 把基础的 L-Reload 分裂为一个轨迹 (trace) 经过的”已开始”状态, 并给线程提供第二条离开该状态的途径。

$$\begin{aligned} &\frac{\theta_n = \text{Inactive}(\perp) \quad \omega = \text{target}_n(\gamma) \neq \perp}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Reloading}(e_n, \text{id}_\Gamma, \omega)]} \text{L-Begin} \\ &\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) \neq \omega \quad (\delta, h) = (\gamma, \text{id}_\Gamma) \vee i(\gamma) = (\delta, h, -)}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Unloading}(g \circ h, \omega, \perp)]} \text{L-Divert} \\ &\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \text{Just}(i'))}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Reloading}(i', g \circ h, \omega)]} \text{L-Iter} \\ &\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad \text{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \text{Nothing})}{\gamma \longrightarrow \delta[\theta_n \mapsto \text{Active}(g \circ h, \omega)]} \text{L-Finish} \end{aligned}$$

每一次迭代都按定义 52 把新产生的逆以 $g \circ h$ 的方式复合到累加器上, 从而使累加器按后进先出顺序应用各逆。在任意两个连续迭代之间, 如果目标视图已经改变, 系统可以改

道 (divert) 该迁移, 应用迄今累积的逆来恢复上下文。L-Divert 与所有其他停用一样经由 Unloading 绕行, 而不是就地应用累加器; 它在那里遇到的守卫是空泛的 (vacuous), 因为从未 Active 过的纤程不提供任何东西, 也不出现在任何已承诺视图中。其两个分支中的第一个中止纤程当前持有的迭代——只有迭代边界才能使这成为可能, 因此改道能够落下的粒度就是迭代器的粒度; 第二个分支让该迭代落地, 而这正是 4.3.3 节所需要的地方。

普通的效应函数 ($\mathfrak{E}_\Gamma$) 是退化情形: 其第一次迭代就已经产生 Nothing。这样的迁移仍然经过 Reloading, L-Divert 在那里仍然适用, 但累加器是 id_Γ , 且没有运行任何迭代, 因此什么也不恢复, 该迁移要么安装其全部效应, 要么一个也不安装。

4.3.3 异步

迄今的各层允许环境在相邻两次迭代之间移动, 并假设每一次迭代本身瞬时完成, 其启动与落地同为一个步骤。我们对非即时性进行抽象建模: 一次迭代产生一个类型为 $\text{Future}(A)$ 的值, 其中 Future 是一个不透明类型构造子, 其定义性属性是: 在提交 (submission) 与解析 (resolution) 之间, 外部状态可能改变。

在此模型下, 一次迭代在一个状态启动而在另一个状态落地, 纤程在飞行 (in flight) 期间处于 Reloading。该层添加的是惯性 (inertia): 一旦启动, 迭代必然落地, 且其落地不可拒绝。因此, 飞行期间转向的目标视图不能通过中止迭代来应对, 只剩下 L-Divert 中让迭代落地的那一个分支可用: 迭代落地, 纤程随后停用。

因此该层不添加任何规则, 也不添加规则可匹配的类型; 在 Γ 的粒度上, 惯性就是其全部内容, 其形式是对宿主可以采取 L-Divert 的哪个分支的限制。

那个分支正是基础演算无法表达的东西。在那里, 目标视图已经转向的迁移在发现它的同一步骤中被撤销; 在这里, 飞行中的迭代必须先落地, 因此纤程在其逆运行期间需要有一个去处, 而唯一健全的去处是持有该迭代所产生的逆的 Unloading。改经 Active 绕行会让纤程在一个步骤的时长内提供其余效应, 并迫使它的依赖者针对一个已经离开的组件进行激活。这就是实现中 reload 与 unload 的相互链接 (mutual chaining)。

停用也可以直接链接回一次激活, 不过是通过复合 (composite) 而非规则。L-Unload 对目标视图不携带任何前提, 因此无论纤程停用期间目标视图变成了什么, 累加器都会运行, 纤程变成 Inactive, 而 L-Begin 可以立即从该状态开始一次新的迁移。

4.3.4 失败

迄今的每条规则都假设它所运行的效应成功, 而运行时做不到这一点。组件安装的效应所触及的范围在追踪它们的上下文之外, 而被触及的对象可能拒绝: 已被绑定的端口、不存在的文件、不应答的对端。失败的迁移仍必须让纤程的效应处于已恢复而非搁浅 (stranded) 的状态。

设 Ξ 为错误集合, 并细化定义 51 的效应迭代器, 使一次迭代可以抛出 (raise) 错误以

代替产生三元组：

$$\begin{aligned}\mathfrak{E}_\Gamma^{\text{fail}} &:= \mu\mathfrak{J}. \Gamma \rightarrow \text{Either}(\Xi, \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J})) \\ \mathfrak{E}_\Gamma^{\text{fail}*} &:= \mu\mathfrak{J}. (e : \Gamma \rightarrow \text{Either}(\Xi, \Gamma \times (\Gamma \rightarrow \Gamma) \times \text{Maybe}(\mathfrak{J}))) \\ &\quad \times ((\gamma : \Gamma) \rightarrow (\text{let Right}(\delta, g, o) = e(\gamma) \text{ in } g(\delta) \simeq \gamma))\end{aligned}\quad (49)$$

见证只约束 **Right** 情形，在模式不匹配处是空泛的，因为抛出没有需要撤销的东西；**Reloading** 携带的 i 从此以后按 $\mathfrak{E}_\Gamma^{\text{fail}*}$ 解读。定义 52 的提升 (lift) 随之沿用，只是把抛出像三元组一样传播，因此会抛出的迭代器与普通迭代器一样，凡效应可用之处它都可用。该层添加一条规则，并把定义 49 的第二种结局投入使用；O-Remove 无需加宽 (widening) 即可接纳它。L-Iter、L-Finish 和 L-Divert 的前提按“**Right** 包裹它们所匹配的三元组”来解读。抛出是迭代所做的事情，因此这条规则是离开 **Reloading** 的一条出口。

$$\frac{\theta_n = \text{Reloading}(i, g, \omega) \quad i(\gamma) = \text{Left}(\xi)}{\gamma \longrightarrow \gamma[\theta_n \mapsto \text{Unloading}(g, \omega, \xi)]} \text{ L-Raise}$$

L-Raise 先恢复后记录。线程带着作为其结局的错误进入 **Unloading**，累积到失败迭代为止的累加器在那里被应用，线程抵达 **Inactive**(ξ) 而未安装任何东西，其所处的状态与一次中止性 L-Divert 所产生的状态相比，仅差在线程所携带的结局上。把失败像所有其他停用一样绕行，正是使每一个结局都只能通过 L-Unload 达到的原因，这是定理 59 所凭借的单一事实。L-Begin 以 **Inactive**($\perp$) 为前提，因此生命周期不会从错误结局重新进入；这正是该结局的实质：它扣住一个效应函数已在它所运行于其上的状态中表明自身不健全的线程，而不是在一个未改变的环境中重试它。失败的线程也不妨碍任何东西：它是 **Inactive**，因此不携带任何已承诺视图，也无法使 **relied** 成立。

失败记录在线程上而不是传播给其父线程，因此迁移失败的组件会让它的兄弟继续运行——这正是插件宿主想要的行为，也是结局按线程而非作为整个状态的一种属性的原因。

4.4 元理论

第 4.3 节提供了十条规则：第 4.2 节的三条编排规则；用于激活的 L-Begin、L-Iter 和 L-Finish；用于激活可能提前结束的两种方式的 L-Divert 和 L-Raise；以及用于停用的 L-Leave 和 L-Unload。本节以这些规则的全局形式读出可组合性的两个维度——一个线程的保证无论其他线程在期间做什么都成立——并补充只有整个系统才能被要求的東西：它总是达到其目标 (targets) 所要求的配置，而且该配置正是静态装配 (static assembly) 会产生的那个配置。下面的每一条性质都是步骤序列的性质，因此我们为步骤编号，并依据该编号读出状态的各个字段。

两个约定把第 3.3.2 节带入本节。下面状态之间的每一个等式都按定义 33 的观测等价 $\simeq$ 解读，正如引理 38 解读第 3.1 节的那些等式一样；效应函数被要求满足的见证条件正是定义 37 给出的那个，对迭代器按定义 51 给出的方式解读，对注册性迭代 (registering iteration) 按下面的 $\approx$ 解读。

定义 53. 用 t 为步骤编号, 使 γ_t 是前 t 个步骤所达到的状态, 并记

$$\text{step}_t := r(n) \quad (50)$$

为在 γ_t 处采取的步骤: r 是它所应用的那条规则 (十条之一), $n \in \mathfrak{N}$ 是它应用该规则所针对的名字。序列从 $\text{dom}(F_0) = \emptyset$ 的 γ_0 开始, 因此每个纤程都经由一次 O-Insert (无论是编排者的还是某次迭代采取的, 见定义 47) 而存在。 γ_t 的字段以上标携带该编号, 因此 θ_n^t 、 ω_n^t 、 σ_n^t 、 g_n^t 和 i_n^t 分别是 n 在 γ_t 处的生命周期状态、已承诺视图、表、累加器和剩余迭代器, 而 F_t 和 σ_t 是 γ_t 自身的注册表和余效应上下文, 即定义 45 的 F_γ 和 σ_γ 在那里取的值。谓词以状态为参数、其余一切为下标, 因此 installed_n^t 、 target_n^t 、 relied_n^t 和 quiet_n^t 是定义 46、定义 49 和定义 50 的谓词在 γ_t 处的取值。 n 的一个片段 (episode) 是索引的一个极大区间 $[b, u]$, 在整个区间上 installed_n^t 成立。它在 b 处开启, 其中 $b > 0$ 且 $\neg \text{installed}_n^{b-1}$ (空的 F_0 使初始时没有任何已安装纤程); 它在 u 处闭合, 此时 installed_n^u 成立而 installed_n^{u+1} 不成立, 最后的片段无需闭合。

第 4.3 节的每条规则都以 $\gamma \longrightarrow \delta[\dots]$ 的形状作结论, 其中前提从 γ 计算出 δ , 在什么也不计算的地方把它留作 γ ; 方括号则编辑注册表的指定字段。

这两半分别命名, 而且两者都是定义在整个 Γ 上的映射。一条作用于 n 的规则在 γ_t 处采取的步骤, 其状态映射 (state map) 为

$$\Psi_t := \begin{cases} \text{pr}_1 \circ i & \text{在 L-Iter、L-Finish 以及落地型 L-Divert 处} \\ g & \text{在 L-Unload 处} \\ \text{id}_\Gamma & \text{在所有其他规则处} \end{cases} \quad (51)$$

其中 i 和 g 是 θ_n^t 携带的迭代器和累加器, 而编辑 $\text{edit}_t : \Gamma \rightarrow \Gamma$ 是把方括号当作函数来读: 把它指名的字段赋值为前提在 γ_t 处计算出的值。因此两者都由 step_t 连同 γ_t 确定, 并在每个状态处有定义, 这正是定理 61 和引理 71 得以在远离 γ_t 处求值它们的原因。每个步骤分解为

$$\gamma_{t+1} = \text{edit}_t(\Psi_t(\gamma_t)) \quad (52)$$

例如, 在 L-Unload 处, edit_t 是 $[\theta_n \mapsto \text{Inactive}(\zeta)]$, 在 O-Remove 处则是移除 $\backslash n$ ——这正是后半部分称为编辑 (edit) 而非赋值 (assignment) 的原因。

字段沿同一条接缝划分: 表 σ_m ——一旦创建 m 的 O-Insert 已把它置空, 就没有任何 edit_t 会写它——以及控制字段 θ_m 、 τ_m 、 π_m 、 d_m 、 p_m 、 e_m 连同 $\text{dom}(F_\gamma)$ ——除经由定义 47 的原语外, 没有任何 Ψ_t 会写它们。当两个状态除控制字段外在一切上都一致时, 记 $\gamma \approx \delta$ 。

关系 $\approx$ 不是定义 33 的 $\simeq$, 二者互不细化对方, 因为各自遗忘的东西正是对方必须保留的东西。恢复精确性是关于效应的断言, 因此 $\approx$ 精确地比较各表和环境状态, 只遗忘注册表中关于哪个纤程安装了它们的记录。规则要读取控制字段来决定自身是否适用, 因此 $\simeq$ 必须保留它们; 本节把它解读为定义 33 与下列各项的合取: 注册表的定义域一致, 且每个纤程的每一个控制字段一致:

$$\gamma \simeq \delta := \sigma_\gamma \simeq \sigma_\delta \wedge \text{dom}(F_\gamma) = \text{dom}(F_\delta) \wedge \forall n, c \in \{\theta, \tau, \pi, d, p, e\}. c(\gamma(n)) \simeq c(\delta(n)) \quad (53)$$

函数类型的字段——如 e_n 和 θ_n 内部的 g ——按定义 36 比较映射的方式比较，迭代器按定义 51 比较两个迭代器的方式比较，任何其他类型的字段按相等比较。下面的结果对两个关系都成立，一个关系对应状态的一半，引理 55 一劳永逸地为全部十条规则确立 $\simeq$ 这一半。

表 1 把第 4.3 节的十条规则读作这样的写操作。累加器、已承诺视图和剩余迭代器是 θ_n 的组成部分，因此第三列也记录对它们的写操作；那里的 h 指名第四列的迭代所产生的逆，当 L-Divert 中止该迭代时为 id_Γ 。当一个由迭代器构建的 Ψ_t 注册一个线程时（定义 47），该注册在它所抽取的名字处携带 O-Insert 行的写操作；而累加器使某个线程退役的 L-Unload 则携带 O-Retire 行的写操作。下面的每一个情形分析都是对表的查表，其中有五个查表出现得足够频繁，值得命名。

规则	θ_n^t	θ_n^{t+1}	Ψ_t	编辑的控制字段
O-Insert	未定义	Inactive($\perp$)	id_Γ	$\text{dom}(F_\gamma)$
O-Retire	不受约束	不变	id_Γ	τ_n
O-Remove	Inactive($-$)	未定义	id_Γ	$\text{dom}(F_\gamma)$
L-Begin	Inactive($\perp$)	Reloading($e_n, \text{id}_\Gamma, \omega$)	id_Γ	θ_n
L-Iter	Reloading(i, g, ω)	Reloading($i', g \circ h, \omega$)	$\text{pr}_1 \circ i$	θ_n
L-Finish	Reloading(i, g, ω)	Active($g \circ h, \omega$)	$\text{pr}_1 \circ i$	θ_n
L-Divert	Reloading(i, g, ω)	Unloading($g \circ h, \omega, \perp$)	id_Γ 或 $\text{pr}_1 \circ i$	θ_n
L-Raise	Reloading(i, g, ω)	Unloading(g, ω, ξ)	id_Γ	θ_n
L-Leave	Active(g, ω)	Unloading($g, \omega, \perp$)	id_Γ	θ_n
L-Unload	Unloading(g, ω, ζ)	Inactive(ζ)	g	θ_n

表 1: 把规则看作对它们所作用的线程 n 的写操作，其中 step_t 即该规则在 n 处的应用。

引理 54. 结合定义 48 阅读表 1: 对每一个步骤 t 和 γ_t 处存在的所有线程 m, n :

1. $\sigma_m^{t+1} \neq \sigma_m^t$ 仅当步骤 t 作用于 m ，且该写操作位于 Ψ_t 内部；
2. ω_n 只在 $\text{step}_t = \text{L-Begin}(n)$ 处产生，只在 $\text{step}_t = \text{L-Unload}(n)$ 处消失，因此 ω_n^t 在 n 的一个片段内为常数；
3. $\Psi_t = g_n^t$ 仅当 $\text{step}_t = \text{L-Unload}(n)$ ，且没有其他步骤把 g 应用于状态；
4. $\neg \text{installed}_n^t \wedge \text{installed}_n^{t+1} \Rightarrow \text{step}_t = \text{L-Begin}(n)$ ，且 $\text{installed}_n^t \wedge \neg \text{installed}_n^{t+1} \Rightarrow \text{step}_t = \text{L-Unload}(n)$ ；
5. π_n, d_n, p_n 和 e_n 随 n 的条目一起产生，此后不再被写； τ_n 是单调的，只在 $\top$ 处且只由 O-Retire 写入。

证明. 设步骤 t 在 n 处应用 r 。由定义 53 它分解为 $\text{edit}_t \circ \Psi_t$ ，其中 edit_t 只写表 1 第五列指名的字段而不写别的， Ψ_t 是 id_Γ 、 n 的某次迭代的一次应用，或累加器 g_n^t ——即这些迭代所产生的各逆的复合。三者中的每一个都由定义 48 限定于 n ，因此 Ψ_t 不写 γ_t 处存

在任何纤程的字段，只写 σ_n 、注册所添加的条目以及其逆所写的 τ 。于是两半划分了全部写操作，而每一条款就是把这一划分读到一个字段上。第二列和第三列有一种读法被使用了两次：Inactive 是唯一不携带已承诺视图的生命周期状态，L-Begin 是唯一离开它的规则，L-Unload 是唯一进入它的规则，而其余每一行都把它前提中的 ω 原样带入其结论。

(1) edit_t 不写任何表（第五列没有指名任何表）， Ψ_t 对存在的 $m \neq n$ 不写任何 σ_m 。因此 σ_m 只能在 $m = n$ 处、且只能在 Ψ_t 内部移动。

(2) ω_n 是 θ_n 的组成部分，只有 edit_t 写 θ_n ，而且只写步骤所作用的那个纤程的 θ_n ，因此由上面的读法， ω_n 在 n 的一次 L-Begin 处产生，在 n 的一次 L-Unload 处消失。 n 的一个片段是 installed_n 成立的一个区间，因而在整个区间上 ω_n 有定义，所以两条规则都不落在其内部。

(3) 第四列中，累加器只出现在 L-Unload 处：其他规则取前向映射 $\text{pr}_1 \circ i$ 或 id_Γ ，而且没有任何 edit_t 把映射应用于状态。

(4) installed_n 即 $\theta_n \neq \text{Inactive}(-)$ ，由上面的读法，L-Begin 和 L-Unload 是仅有的其前提与结论在 θ_n 是否为 Inactive 上不同的规则。作用于某个 $m \neq n$ 的步骤不写 θ_n ，而注册所添加的条目位于 γ_t 处不存在的名字上。

(5) 第五列没有任何一行指名 π 、 d 、 p 或 e ；它们随 O-Insert 添加的条目一起产生，该条目的结论写它们，注册所采取的 O-Insert 亦然。只有 O-Retire 写 τ ，且在 $\top$ 处写，无论是编排者采取的还是在注册的逆中采取的（定义 47）；O-Insert 在一个尚未存在的名字处把 τ 置为 $\perp$ ，因此没有任何步骤把 τ 送回 $\perp$ 。□

还有三个查表说明规则看不到什么。第一个是：规则只通过上述观测读取状态，因此整个演算可以下降到 $\Gamma/\simeq$ 。

引理 55. ($\simeq$ -不变性。) 设 $\gamma \simeq \gamma'$ 按上述读法。则第 4.3 节的一条规则在 γ 处作用于 n 适用，当且仅当它在 γ' 处作用于 n 适用，而且两次应用所达到的状态再次由 $\simeq$ 相关联。

证明. 第 4.3 节的每一个前提都属于四种类型之一，每种类型都读取该关系所保留的一个组成部分。把 θ_n 或 τ_n 与模式匹配的前提，以及 O-Remove 的前提 $\forall m. \pi_m \neq n$ ，读取控制字段。O-Insert 的前提 $(d, p, e) \in \mathfrak{C}_\Gamma$ 和 $\forall m. p_m \cap p = \emptyset$ 读取 d 、 p 和 e 。提及 target_n 或 relied_n 的前提读取 τ_m 、 θ_m 内部的已承诺视图以及 $\text{dom}(\sigma_\gamma)$ （定义 45 从 θ_m 和 $\text{dom}(\sigma_m)$ 计算出它），而定义 33 只在两个余效应上下文的定义域一致时才关联它们。其余前提读取 $\text{dom}(F_\gamma)$ 。没有哪个前提以超出 $\simeq$ 的方式读取某个值 $\sigma_\gamma(k)$ ，因此没有哪个前提区分两个 $\simeq$ -相关的状态。

对于结论，由定义 53 有 $\gamma_{t+1} = \text{edit}_t(\Psi_t(\gamma_t))$ 。 edit_t 所赋的值是它所匹配的前提的组成部分，由上一段和定义 51 在两个状态处相关联——定义 51 关联迭代器在 $\simeq$ -相关状态处产生的三元组。而 Ψ_t 尊重 $\simeq$ ：它是 id_Γ ，或是 e_n 的一次迭代（定义 51 要求它尊重 $\simeq$ ），或是 θ_n 内部的累加器——由同一定义，它是各自尊重 $\simeq$ 的各逆的复合。□

状态携带的名字由上述观测中的两个读取： $\text{dom}(F_\gamma)$ 和控制字段的编号；而抽取名字的规则抽取任何尚未使用的名字（定义 47）。因此，按 $\simeq$ 解读下面的结果，也要求按重命名解读它们，这正是第 4.1 节的纪律的兑现。

引理 56. (等变性。) 设 $\chi : \mathfrak{N} \rightarrow \mathfrak{N}$ 是一个双射， $\chi \cdot \gamma$ 是携带注册表 $F_\gamma \circ \chi^{-1}$ 的状态，

其中出现在某个 π_m 或 ω_m 中的每一个名字都被其像替换。则 $\chi \cdot \gamma$ 是一个状态，在 γ 良构处良构；且 $\text{step}_t = r(n)$ 把 γ_t 带到 γ_{t+1} ，当且仅当 $r(\chi(n))$ 把 $\chi \cdot \gamma_t$ 带到 $\chi \cdot \gamma_{t+1}$ 。

证明. 前提只通过把名字与另一个名字比较来读取名字——无论是直接的，如 O-Insert 的新鲜性 $n \notin \text{dom}(F_\gamma)$ 和 O-Remove 的 $\forall m. \pi_m \neq n$ ，还是通过名字表，如 target_n 和 relied_n 读取 π_m 和 ω_m 。双射保持每一次这样的比较。规则写入的名字只有 O-Insert 所设置的 π 和 L-Begin 所设置的 ω ，两者都取自其前提所读取的东西，因此这些写操作与 χ 交换；效应函数根本不写任何名字，只通过定义 47 的原语抽取一个名字，而定义 48 把该原语限定于它所添加的条目。良构性（定义 58）是四个把名字与名字比较的条件。□

因此，一个序列与其重命名以相同的顺序采取相同的规则，并达到仅相差 χ 的状态。除各自注册所抽取的名字外一致的两个序列因此被等同看待，下面的结果按把二者等同起来的重命名解读。

第二个查表是：剥去名字以外一切的条目对规则是不可见的——这使定义 47 能在其恢复的状态中没有相应纤程的地方令一个纤程退役，也使引理 72 能移除被删除片段所做的注册。

引理 57. (残留条目。) 当 $\tau_n = \top$ 、 $\theta_n = \text{Inactive}(\perp)$ 、 $\sigma_n = \emptyset$ 且没有 m 具有 $\pi_m = n$ 时，称 n 在 γ 处是残留的 (vestigial)；一个残留条目满足 $\gamma \approx \gamma \setminus n$ 。若 n 在 γ 处是残留的，则对每条规则和每个 $m \neq n$ ：

1. 一条在 γ 处作用于 m 的规则在 $\gamma \setminus n$ 处作用于 m 同样适用，两次应用所达到的状态只在 n 处的条目上不同，且该条目保持残留；
2. 反之，一条在 $\gamma \setminus n$ 处作用于 m 的规则在 γ 处适用，除非它是一次抽取名字 n 或认领 p_n 的某个键的 O-Insert。

证明. 残留的 n 对作用于 $m \neq n$ 的规则的前提所读取的任何观测都没有贡献。它不是 Active，因此 σ_n 不进入 σ_γ ， n 也不是任何键的提供者，使 $\gamma \models d_m$ 和 target_m 不受影响； installed_n 不成立，因此 n 不对 relied_m 贡献任何析取项；没有 π 指名 n ，因此 m 的 O-Remove 的前提 $\forall m'. \pi_{m'} \neq m$ 不受影响；而 θ_n 、 τ_n 和 π_n 只被作用于 n 本身的规则读取。条款 (2) 除外的两个前提正是移除所放宽的两个前提：不存在的名字是新鲜的，不存在的提供满足其他一切。由引理 54，作用于 $m \neq n$ 的规则不写 n 的任何字段，因此该条目存续；且由定义 48，该步骤的状态映射被限定于 m ，因此它使 σ_n 保持为空。□

简化生命周期状态以及匹配它们的规则，会得到一个子演算，但并非每个结果都能在简化后存活。去掉第 4.3.1 节才是要紧的情形——这正是第 4.3 节开头所做的划分，从元理论一侧来看：其守卫正是确立定义 58 的条款 (3) 和 (4) 的东西，而定理 63 依赖于守卫所创造的时间区间，因此没有它这三者都会失效。其他三个小节所添加的内容可以在不干扰下面结果的情况下被简化掉，因为它们各自只是向定义 49 固定的那一个状态空间添加规则。

4.4.1 保持性

定义 45 固定了注册表的形状，在下面的结果可以在此基础上添加内容之前，必须对照它检查各条规则。本小节识别规则所保持的不变量，其第一款就是那种形状，其余条款则

是那些结果所假设的东西。

定义 58. 当对所有的 $m, n \in \text{dom}(F_\gamma)$ 和所有 $k \in K$ 都有:

1. $\pi_n \in \text{dom}(F_\gamma) \cup \{\text{root}\}$;
2. $m \neq n \Rightarrow p_m \cap p_n = \emptyset$;
3. $\text{installed}_n(\gamma) \Rightarrow \omega_n$ 在 d_n 上全定义且取值于 $\text{dom}(F_\gamma)$;
4. $\text{installed}_n(\gamma) \wedge k \in d_n \wedge \omega_n(k) = m \Rightarrow \text{installed}_m(\gamma)$ 。

时, 称注册表 F_γ 是良构的 (well formed)。

条款 (1) 是定义 45 的树逐边读取, 保持父指针落在注册表内。该定义还要求的无环性 (acyclicity) 不需要条款, 因为指针指名的纤程先于指名它的纤程注册。

定理 59. (保持性。) 若 F_t 是良构的, 则无论步骤 t 应用哪条规则, F_{t+1} 也是良构的。每一条款都在 γ_{t+1} 处由 γ_t 处的全部四条确立。

证明. 设步骤 t 作用于 n 。

(1) 由表 1, 只有 O-Insert 和 O-Remove 写 π 或 $\text{dom}(F_\gamma)$ 。O-Insert 以 $\pi_n \in \text{dom}(F_t) \cup \{\text{root}\}$ 为前提, 这正是它添加的纤程的条款, 而且它让所有其他 π 保持不变……在扩大 $\text{dom}(F_\gamma)$ 的同时。O-Remove 以 $\forall m. \pi_m \neq n$ 为前提, 所以没有幸存的 π_m 指名它所取走的纤程。

(2) O-Insert 的最后一条前提是 $\forall m. p_n \cap p_m = \emptyset$, 这是针对它所添加纤程的条款; 而由表 1, 没有任何其他规则写入 p 或扩大 $\text{dom}(F_\gamma)$ 。下面用到两个推论: 由定义 43, $\text{dom}(\sigma_m) \subseteq p_m$, 所以不同的表互不相交且 σ_γ 是一个函数; 又 $k \in p_m \cap p_{m'}$ 迫使 $m = m'$, 所以 k 至多有一个可能的提供者。

(3) 由引理 54(2), 唯一写入某个 ω 的规则是 L-Begin, 其前提 $\omega_n = \text{target}_n^t \neq \perp$ 使它 (所写的 ω) 在 d_n 上完全且取值于 $\text{dom}(F_\gamma^t)$, target 指名提供者。由表 1, 唯一缩小 $\text{dom}(F_\gamma)$ 的规则是 O-Remove, 其前提 $\theta_n^t = \text{Inactive}(-)$ 给出 $\neg \text{installed}_n^t$; 从而由 γ_t 处的条款 (4), 在 installed_m^t 成立时, 没有 m 对某个 $k \in d_m$ 满足 $\omega_m^t(k) = n$; 并且 n 自身不携带任何 ω 。

(4) 由引理 54(2) 和 (4), 该条款在 γ_{t+1} 处失败只可能发生在: 某个已安装的 (纤程) 跌落、某个 ω 被写入、或某个 ω 所指名的纤程离开了 $\text{dom}(F_\gamma)$ 。最后一种情况是 O-Remove: 其被移除的纤程未安装, 从而由 γ_t 处的条款 (4), 它不被任何已安装 m 的 ω_m^t 所指名。第一种情况是 n 的 L-Unload, 其前提 $\neg \text{relied}_n^t$ 读作

$$\forall m \neq n, k \in d_m. \text{installed}_m^t \Rightarrow \omega_m^t(k) \neq n$$

且它对 $m \neq n$ 不写入任何 ω , 并留下 $\neg \text{installed}_n^{t+1}$, 所以该条款对 n 也成立。第二种情况是 n 的 L-Begin, 写入 target_n^t , 其值为 d_n 的键的提供者, 从而在 γ_t 处为 Active; 该步骤不改变其他纤程的 θ , 所以它们在 γ_{t+1} 处也处于已安装状态。□

L-Unload 上的护栏正是承载条款 (3) 和 (4) 的东西。O-Remove 的前提 $\forall m. \pi_m \neq n$ 只谈论父指针; 使已提交视图不会指名被移除纤程的, 是那个护栏——它在若干步骤之前、出

于另一个原因被施加。由于失败也经由 Unloading 路由，该论证不必为错误结局重复一遍。由此得出基础演算不具备的两件事：O-Remove 释放的名字可以被 O-Insert 重新签发，因为没有过期的已提交视图会指名它；并且线程一旦 Inactive 即可被移除，无需另行检查是否有人依赖它。

4.4.2 时间可组合性

局部时间可组合性用一个累加器撤销一个效应序列（第 3.1.3 节）。注册表为每个线程持有一个累加器，而线程相互交错：在 n 把一个逆复合到 g_n 上的时刻与 g_n 运行的时刻之间，其他线程已经移动了状态。 g_n 在那里是否仍然撤销它被构造出来所要撤销的东西，正是该保证的全局形式所断言的；而它所依赖的条件是，中间的步骤与 g_n 交换。

定义 60. 对 $i \in \mathfrak{C}^{\text{iter}*}$ ，设 $\text{reach}(i)$ 为包含 i 且在续延下封闭的最小迭代器集合；并按如下方式在迭代器处解读定义 17 的变换幺半群 $\mathfrak{M}$ ：以其生成元取 $\text{reach}(i)$ 中每个迭代器的前向映射与所产生的逆：

$$\text{reach}(i) := \bigcap \{S \mid i \in S \wedge \forall i' \in S, \gamma \in \Gamma. i'(\gamma) = (-, -, \text{Just}(i'')) \Rightarrow i'' \in S\} \quad (54)$$

$$\mathfrak{M}(i) := \langle \{\text{pr}_1 \circ i' \mid i' \in \text{reach}(i)\} \cup \{\text{pr}_2(i'(\gamma)) \mid i' \in \text{reach}(i), \gamma \in \Gamma\} \rangle$$

在适用第 4.3.4 节之处，对三元组取 Right 读法；并把 $\text{len}(i)$ 记为续延所排序的链 $C \subseteq \text{reach}(i)$ 上 $|C|$ 的上确界。两个迭代器 i, j 按定义 19 的意义独立——用这些变换幺半群来解读，并把迭代的产出理解为它的逆连同其续延：

$$\forall f \in \mathfrak{M}(i), g \in \mathfrak{M}(j). f \circ g \simeq g \circ f \quad (55)$$

$$\forall i' \in \text{reach}(i), g \in \mathfrak{M}(j), \gamma \in \Gamma. \text{pr}_{2,3}(i'(g(\gamma))) \simeq \text{pr}_{2,3}(i'(\gamma))$$

并且关于 j 对称： $\simeq$ 在映射上按定义 36 解读，在续延上按定义 51 解读，在注册性迭代（定义 47）上按其所指名组件的一致来解读。当对每个 $l \neq l'$ ， i_l 与 $i_{l'}$ 都独立时，称迭代器族 $(i_l)_{l \in L}$ 两两独立；当 $(e_n)_{n \in N}$ 两两独立时，称一个步骤序列两两独立，其中 N 是序列所曾持有的名字集合——编排器每插入一个线程、迭代每注册一个线程，各对应一个名字。

这种意义上的独立性正是迹理论所取为原始概念的东西：交换的动作在序列上生成一个等价关系，在该等价关系下，重排两个相邻的独立动作保持端点不变 [44]；引理 71 就是对这些规则的重排。之所以用族而不用集合，是为了把同一组件的两个名字保持在作用域内：于是该条件要求该组件的效应函数与自身独立，也就是要求 $\mathfrak{M}(i)$ 可交换。第一个条件正是定理 61 所用的，第二个则是定理 73 额外需要的：重排两个线程的步骤，会在另一个线程移动过的状态处对迭代器求值；而映射的交换本身并不说明迭代器在那里产出相同的逆与相同的续延。检查第一个条件所需的不过就是迭代本身，因为引理 18(1) 把交换性从生成元传递到它们所生成的幺半群。

在这些条件下，定理 7 的单累加器不变量在交错中存活下来，其形式正是时间可组合性的内容所在：运行一个逆会撤销该线程的贡献，此外别无其他。

定理 61. (恢复精确性) 设步骤序列两两独立, 设 n 的一个时段在 b 处开启, 设 $u \geq b$ 位于其中, 并设 $t_1 < \dots < t_l$ 是 $[b, u]$ 中作用线程不是 n 的那些下标。则

$$g_n^u(\gamma_u) \approx (\Psi_{t_l} \circ \dots \circ \Psi_{t_1})(\gamma_b) \quad (56)$$

也就是说, 在 γ_u 处应用 n 的累加器, 在不计控制字段的意义下, 得到的就是同样的那些步骤从 γ_b 出发会产生的状态。若把右端读作“假若 n 从未开始”所达到的状态, 则还需假定 n 所注册的线程在 $[b, u]$ 中不采取任何步骤, 因为 n 所注册的线程在那种情形下本不会在场去采取步骤。

证明. 对 u 归纳, 遍及使 $u+1$ 位于时段中的那些下标 u 。在 $u=b$ 处, $b-1$ 处的步骤是 L-Begin (时段按定义 53 开启), 所以由表 1, $g_n^b = \text{id}_\Gamma$, 下标集为空, 断言即 $\gamma_b \approx \gamma_b$ 。每一步都用两个事实。由于 edit_t 只写控制字段,

$$\gamma_{t+1} \approx \Psi_t(\gamma_t)$$

又由于 $\mathfrak{M}(e_n)$ 中每个映射除了注册所添加的字段外不写任何控制字段 (由定义 48 连同定义 47), 这样的每个映射都把 $\approx$ -相等的状态带到 $\approx$ -相等的状态。

设步骤 u 作用于 n 。由于时段在 u 和 $u+1$ 处都是开启的, 引理 54(4) 排除 n 的 L-Begin 和 L-Unload, 而 O-Insert 与 O-Remove 读取一个被 installed_n^u 否认的 θ_n , 于是只剩下两种情况。当规则是 L-Iter、L-Finish 或落地的 L-Divert 时, 表 1 给出 $\Psi_u = \text{pr}_1 \circ i_n^u$, 且对迭代所产出的逆 h 有 $g_n^{u+1} = g_n^u \circ h$ 。定义 51 的见证条件读作 $h(\Psi_u(\gamma_u)) = \gamma_u$ ——在迭代注册线程之处按 $\approx$ 理解 (引理 57)——而由上面的等式, g_n^u 保持 $\approx$, 所以

$$g_n^{u+1}(\gamma_{u+1}) \approx (g_n^u \circ h)(\Psi_u(\gamma_u)) = g_n^u(\gamma_u)$$

当规则是 L-Leave、L-Raise、中止的 L-Divert 或 n 的 O-Retire 时, 表 1 给出 $\Psi_u = \text{id}_\Gamma$ 且 $g_n^{u+1} = g_n^u$, 所以同样的等式在 $h = \text{id}_\Gamma$ 下成立。无论哪种情形, 归纳假设都在下标集不变的情况下延续——这正是定理 7 的逐步骤计算。

设步骤 u 作用于 $m \neq n$ 。则由表 1, $g_n^{u+1} = g_n^u$, 且 $\Psi_u \in \mathfrak{M}(e_m)$; 若规则是编排规则, 则 $\Psi_u = \text{id}_\Gamma$ 。于是独立性给出

$$g_n^u(\gamma_{u+1}) \approx g_n^u(\Psi_u(\gamma_u)) = \Psi_u(g_n^u(\gamma_u))$$

这正是附加了 Ψ_u 的归纳假设。 □

推论 62. (终端恢复) 设步骤序列两两独立, 并设 n 的一个时段在 b 处开启、在 u 处闭合, 无论 n 到达何种结局。则按定理 61 中的 $t_1 < \dots < t_l$,

$$\gamma_{u+1} \approx (\Psi_{t_l} \circ \dots \circ \Psi_{t_1})(\gamma_b) \quad (57)$$

被 O-Remove 移除的线程也同样不留下任何东西, 因为其前提只允许 $\theta_n = \text{Inactive}(-)$ 。

证明. 由引理 54(4), 步骤 u 是 n 的 L-Unload; 由引理 54(3), 其 Ψ_u 即 g_n^u , 所以 $\gamma_{u+1} \approx g_n^u(\gamma_u)$, 定理 61 适用。无论是结论还是 $\approx$ 都不提及 ζ , 而由表 1, ζ 正是 L-Divert 与 L-Raise 所导致的状态唯一不同的字段。 □

上述结果把两两独立性作为对组件的前提假设, 而第 3.3.2 节正是解除这一假设的地方: 当组件执行的每个效应都是某个键的操作且每个键都可交换时, 由这些操作构造出的任意两个效应函数都独立 (定理 42)。把该结果从效应函数带到迭代器不需要任何新的东西: 经由余效应中介的效应函数 (定义 41) 已经按每个阶段产出的结果来选择该阶段之后发生的事情, 而这正是迭代器在其续延中所携带的。第 3.2 节的余效应操作是根本不需要任何假设的情形: 组件在那里贡献的映射是集合运算与相应限制的复合; 两个这样的映射只要触及不相交的键就交换; 而定义 58 的条款 (2) 使不同纤维的提供互不相交。

4.4.3 空间可组合性

局部空间可组合性把组件约束在它自己的规范之内: 只在它的依赖已被提供之处激活它, 并针对这些依赖对每个上下文变化进行分类 (第 3.2.2 节)。全局形式添加了对其他纤维量化的内容: 提供者只有在其解析了它的每个依赖者都已停用之后才撤销绑定; 并且迁移据以安装其效应的解析不会在它之下移动。余效应侧的这两个性质分别交付这两条, 而它们是一起证明的, 因为它们是同一个不变量的两半, 即引理 54(2) 所确立的 ω_n 在一个时段上的固定性。排序定理 (ordering theorem) 是这种固定性在时段中 n 处于 Active 随后 Unloading 的那一部分所买到的东西; 协调定理 (coherence theorem) 则是它在 n 安装其效应的那一部分所买到的东西。

定理 63. (排序) 一个纤维只在它的依赖已被提供之处才开始迁移:

$$\text{step}_t = \text{L-Begin}(m) \Rightarrow \gamma_t \models d_m \quad (58)$$

再设 $[b', u']$ 是 m 的一个时段, 满足对某个 $m \neq n$ 和 $k \in d_m$ 有 $\omega_m^{b'}(k) = n$; 设 $[b, u]$ 是包含 b' 的 n 的时段; 并让 t 遍取 $[b', u']$ 。则

1. $\omega_m^t(k) = n$;
2. $b < b'$, 且若 $[b, u]$ 闭合则 $u' < u$;
3. $k \in \text{dom}(\sigma_n^t)$ 且 $\sigma_n^t(k) = \sigma_n^{b'}(k)$ 。

证明. 第一个断言就是 L-Begin 的前提 $\text{target}_m^t \neq \perp$, 由定义 46 它给出 $\gamma_t \models d_m$ 。

(1) 即引理 54(2)。

对于 (2): $b' - 1$ 处的 L-Begin 写入 $\omega_m^{b'} = \text{target}_m^{b'-1}$, 其值是提供者, 所以 $\theta_n^{b'} = \text{Active}(-, -)$; $b - 1$ 处的 L-Begin 留下 $\theta_n^b = \text{Reloading}(-, -, -)$, 所以 $b \neq b'$, 从而 $b < b'$ (两个时段都按定义 53 开启)。设 $[b, u]$ 闭合, 并假设 $u \leq u'$ 。则 $u \in [b', u']$, 所以 installed_m^u 成立; 又由 (1), $\omega_m^u(k) = n$ ——这正是 relied_n^u , 而 u 处的 L-Unload 否认它。因此 $u' < u$ 。

对于 (3): n 是 k 在 $\gamma_{b'}$ 处的提供者, 所以 $k \in \text{dom}(\sigma_n^{b'})$ 。 $[b', u']$ 中没有 n 的 L-Unload: 当 $[b, u]$ 闭合时, 由 (2) 它落在 $u > u'$ 处; 当它不闭合时, 引理 54(4) 使 n 根本没有 L-Unload。由于 $\theta_n^{b'} = \text{Active}(-, -)$, 表 1 于是在 $[b', u']$ 内只留下 L-Leave 这一条可作用于 n 的规则, 而它的 Ψ_t 是 id_Γ ; 由引理 54(1), σ_n 在那里恒定。 $\square$

一个横跨多个步骤的迁移，否则可能会安装针对一个在其下已发生变化的解析而计算的效应；两条前提阻止了这一点。L-Iter 和 L-Finish 携带 $\text{target}_n(\gamma) = \omega$ ，所以迁移只有在它的已提交视图仍是其目标视图时才继续；L-Divert 携带其否定，所以目标视图的任何变化都会使线程离开迁移。L-Raise 根本不以目标视图为条件——raise 是迭代自己做的事情，而不是环境所请求的——并且它在任何情况下都会退出迁移。变化的两个方向不加区分：依赖已经消失的组件与依赖已被替换的组件走同一条路线离开，因为已经变成 $\perp$ 的目标视图与已经变成另一个线程的目标视图，与 ω 的不同程度是一样的。惯性 (inertia) 正是使这不成其为关于每一步的保证的东西。当目标视图转动时，一个已经在飞行中的迭代无论如何都会落地（经由 L-Divert），而那一次落地安装的效应是针对一个不再成立的解析计算的。因此规则所交付的是一个析取，而第二个分支正是使第一个分支安全的原因。

定理 64. (解析协调) 设 n 的一个时段 $[b, u]$ 在 b 处开启，且 $\omega_n^b = \omega$ 。则 θ_n 在该时段的初始区间 $[b, r]$ 上为 $\text{Reloading}(-, -, -)$ ，且迁移的每个迭代都针对同一个解析 ω 运行：

$$\forall t \in [b, r]. \text{step}_t \in \{\text{L-Iter}(n), \text{L-Finish}(n)\} \Rightarrow \text{target}_n^t = \omega \quad (59)$$

当线程离开该区间（即 $r < u$ ）时，下列情形恰有一个成立：

1. $\text{step}_r = \text{L-Finish}(n)$ ，且 $\theta_n^{r+1} = \text{Active}(-, \omega)$ ；
2. $\text{step}_r \in \{\text{L-Divert}(n), \text{L-Raise}(n)\}$ ，且时段在某个 $u > r$ 处闭合，满足 $\gamma_{u+1} \approx (\Psi_{t_l} \circ \dots \circ \Psi_{t_1})(\gamma_b)$ ，如推论 62 所述。

证明. $b - 1$ 处的 L-Begin 写入 Reloading ，且由表 1，它是唯一进入该生命周期状态的规则；其前提 $\theta_n = \text{Inactive}(\perp)$ 与引理 54(4) 使对它的任何第二次应用都落在时段之外。所以 Reloading 占据 $[b, u]$ 的一个初始区间 $[b, r]$ ，且不会被重新进入。

于是第一个断言就是表 1 赋予 L-Iter 与 L-Finish 的前提 $\text{target}_n(\gamma) = \omega'$ ，再结合引理 54(2) 给出的 $\omega' = \omega$ 。

对于这个二分： step_r 是一条前提含有 $\theta_n = \text{Reloading}(-, -, -)$ 而结论不含的规则；表 1 提供的有 L-Finish、L-Divert 和 L-Raise。第一个落入 $\text{Active}(-, \omega)$ ，另外两个落入 $\text{Unloading}(-, \omega, -)$ ；从后者出发，引理 54(4) 使 L-Unload 成为唯一出口，而推论 62 提供那个等式。落地的 L-Divert 所贡献的迭代是 n 自己的迭代之一，因而属于那个累加器所撤销的映射。若相反 $r = u$ ，则序列以迁移仍在飞行中结束，第一个断言就是全部所断言的内容。
□

4.4.4 进展性

一个把提供者的撤销推迟到其依赖者消失为止的护栏，只有当它最终释放时才交付定理 63。注册表线程上的一个关系承载着这一论证。

定义 65. 注册表名字上的优先关系为

$$n \prec m := p_n \cap d_m \neq \emptyset \quad (60)$$

于是 n 可能提供 m 所声明的一个键。它只读 d 和 p ，而由引理 54(5)，这两者在纤程的条目出现时一并产生，且之后永不再被写入。

定理 66 与定理 73 是在 $\prec$ 无环这一假设上建立的；这是一个假设，而不是定义本身所交付的东西——对一个声明了它自己提供的键的组件， $n \prec n$ 成立。 $\prec$ 所排序的是两个纤程的激活，而不是它们的寿命： $n \prec m$ 说的是 n 必须先变成 Active， m 才能；而提供者比其消费者活得更久这一点则是定理 63(2)，一个关于带护栏演算的定理。

纤程的目标视图既回应于创建它的纤程，也回应于它的提供者。创建者通过定义 47 的原语写入的是 τ_n ，而由引理 54(5)， τ_n 是单调的。因此创建者在它孩子的整个存在期间至多只能转动其目标视图一次。

进展性断言某个规则可以应用，所以它是就主机必须提供的规则来表述的：L-Begin、L-Leave、L-Unload、落地规则 L-Iter、L-Finish 与 L-Raise，以及 L-Divert。它任何地方都不诉诸 L-Divert 的中止选项，所以受第 4.3.3 节惯性约束的主机也被覆盖。

定理 66. (进展性) 假设 $\prec$ 无环、对每个 n 有 $\text{len}(e_n) \leq K$ 、定义 60 的名字集合 N 有限；并设每个步骤都应用生命周期规则。记 $S(n)$ 为作用于 n 的步骤数，

$$V(n) := |\{t : \text{target}_n^t \neq \text{target}_n^{t+1}\}| \quad (61)$$

为其目标视图转动的次数。则

1. (无死锁) $\neg \text{quiet}_t$ 蕴含在 γ_t 处某个生命周期规则可应用；
2. (终止性) $S(n) \leq (K + 4)(V(n) + 1)$ ，且 $V(n)$ 与 $\sum_n S(n)$ 都有限。

因此每个极大的生命周期步骤序列都以静默态结束。

证明. 无死锁. 设 $\neg \text{quiet}_t$ ，则某个纤程 n 不满足定义 49 的静默的任何一条条款。对照表 1 读它可能属于的四种情形：

- $\theta_n^t = \text{Inactive}(\perp)$ 且 $\text{target}_n^t \neq \perp$ ：L-Begin 可应用；
- $\theta_n^t = \text{Reloading}(-, -, \omega_n)$ 且 $\text{target}_n^t = \omega_n$ ：若 $i_n^t(\gamma_t)$ 的值所选出的 L-Iter、L-Finish、L-Raise 三者之一可应用；
- $\theta_n^t = \text{Reloading}(-, -, \omega_n)$ 且 $\text{target}_n^t \neq \omega_n$ ：若 $i_n^t(\gamma_t)$ raise 则 L-Raise 可应用，否则 L-Divert 可应用，使该迭代落地而非中止它；
- $\theta_n^t = \text{Active}(-, \omega_n)$ 且 $\text{target}_n^t \neq \omega_n$ ：L-Leave 可应用。

设没有任何纤程属于这些种类，那么剩下某个 m_0 满足 $\theta_{m_0}^t = \text{Unloading}(-, -, -)$ 。如下构造 $m_0, m_1, \dots$ ：给定处于 Unloading 的 m_j ，要么 $\neg \text{relied}_{m_j}^t$ ，此时 L-Unload 可应用于 m_j ，构造停止；要么存在 $m_{j+1} \neq m_j$ 与 k_j 满足 $\text{installed}_{m_{j+1}}^t$ 且 $\omega_{m_{j+1}}^t(k_j) = m_j$ 。在后一种情形中

$$k_j \in d_{m_{j+1}} \cap \text{dom}(\sigma_{m_j}^t) \subseteq d_{m_{j+1}} \cap p_{m_j}$$

第二个成员关系是定理 63(3) 应用于 t 所在的 m_{j+1} 时段的结果, 从而 $m_j \prec m_{j+1}$ 。再者, $\text{target}_{m_{j+1}}^t \neq \omega_{m_{j+1}}^t$: 处于 Unloading 的线程在定义 σ_γ 的并集之外, 所以 k_j 在 γ_t 处未被提供, 或由 m_j 之外的线程提供。若 m_{j+1} 处于 Active 或 Reloading, 则它就会属于已被排除的四种情形之一, 所以它处于 Unloading, 构造继续。这些 m_j 是 $\prec$ -递增的, 从而由无环性互不相同; 而 $\text{dom}(F_\gamma^t)$ 有限, 所以构造停止。

终止性. 两条断言界定 $S(n)$ 。

(A) 在 target_n^t 恒为 ω^* 的一个极大区间上, 至多有 $K+4$ 个步骤作用于 n 。读表 1 的 θ 列: 从 $\omega \neq \omega^*$ 的 Active($-, \omega$) 出发, 线程经历一次 L-Leave 和一次 L-Unload, 然后若 $\omega^* \neq \perp$, 还经历一次 L-Begin 和至多 $\text{len}(e_n) \leq K$ 次落地, 若最后一次落地是 L-Raise 则再加上第二次 L-Unload; 从针对 $\omega \neq \omega^*$ 的 Reloading 出发, 它用 L-Divert 取代 L-Leave; 从任何其他状态出发, 它经历该序列的一个后缀。区间内不再有 L-Divert 或 L-Leave——L-Begin 写入的 ω 就是 $\text{target}_n^t = \omega^*$ 本身——而在 Active($-, \omega^*$)、在 $\omega^* = \perp$ 的 Inactive($\perp$)、以及在 Inactive(ξ) 处, 没有任何规则可应用。

(B) 若 $\text{target}_n^t \neq \text{target}_n^{t+1}$ 且步骤 t 作用于 m , 则要么 $m \prec n$, 要么步骤 t 写入 τ_n 。由定义 46, target_n^t 的值是 τ_n 与 d_n 各键的提供者的表的函数; 提供者满足 $k \in \text{dom}(\sigma_m) \cap d_n$, 从而 $m \prec n$; 而由引理 54(1), 表只在作用于其自身线程的步骤处改变。无环性在第一种情形给出 $m \neq n$; 引理 54(5) 的单调性允许第二种情形在每个线程至多一个 t 处发生。

由 (A), 区间计数把 $S(n)$ 界定为 $S(n) \leq (K+4)(V(n)+1)$; 由 (B), target_n^t 的每次转动要么消耗某个严格 $\prec$ -低于 n 的线程的一个步骤, 要么就是 τ_n 所容许的那一次转动, 所以 $V(n) \leq 1 + \sum_{m \prec n} S(m)$ 。由于 $\prec$ 无环且 N 有限, 递归

$$B(n) := (K+4) \left(2 + \sum_{m \prec n} B(m) \right)$$

是良基的, 并定义出满足 $S(n) \leq B(n)$ 的 B ; 因此 $V(n)$ 有限, 且 $\sum_n S(n) \leq \sum_n B(n)$ 。由 (1), 不可扩展的序列是静默的。□

N 的有限性是假设出来的而非推导出来的; 组件上的一个条件即可交付它。主机所持有的组件是在任何东西运行之前给定的有限多个程序, 所以如果没有任何组件能够 (无论多么间接地) 注册某个“注册了自身实例的组件”的线程, 那么注册构成一棵深度有界的树, 而 $\text{len}(e_n) \leq K$ 界定了它的分支。该假设所排除的, 是一个无界地注册自身实例的组件。

target 记录的是提供线程而不是一个布尔值; 在第 4.2 节的单源纪律下, 两者驱动相同的迁移, 因为在那里一个键只有一个可能的提供者。该视图买到的是上述结果的词汇: 定理 63 与定理 64 都谈论线程据以激活的解析; 也正是它使这些结果在第 3.2.3 节的有作用域解析下存活——在该解析下, 一个键在不同领域 (realm) 中解析到不同的提供者, 提供不再强制该视图。实现携带了这一作用域, 并把视图保存在 `fiber.committed` 中 (第 5.1.3 节)。

4.4.5 合流性

到目前为止的结果都是关于单个线程的。刻画系统整体的性质是: 它的动态历史不留下任何痕迹——无论一个运行中的系统经历了怎样的激活与停用序列, 它所静默于其中的状

态，正是同样的插入与退役在”每个最终处于激活状态的组件都按依赖顺序被加载一次、且从未被卸载”的情况下会产生那个状态。生命周期关系是合流的，而它所收敛到的范式正是静态装配的那一个。对于动态组合而言，这正是变更传播为增量计算所确立的”与从头求值的一致性”的对应物 [45]。

该断言只关乎 $\rightarrow$ 本身。编排步骤是输入；给定不同输入的两个序列落在不同的地方，并无有趣的原因；问题在于：生命周期规则——它在”下一个由哪个线程执行步骤”以及”一个 Reloading 线程走哪个出口”上都是非确定的——能否被弄得不一致。

首先需要三个引理。第一个在不参照任何步骤序列的情况下确定最终处于 Active 的线程集合——这正是使它成为输入的函数而非调度的函数的东西。

定义 67. 当线程未被退役、注册它的线程是受支撑的、且它声明的每个键都由某个受支撑的线程提供时，称该线程在 γ 处受支撑 (supported)。dom(F_γ) 上的支撑关系是这些条款所读出的两个关系的并，

$$m \triangleleft n := m \prec n \vee \pi_n = m \quad (62)$$

在它良基之处 (引理 68)，我们把支撑集记作 A ，即在 γ 处受支撑的线程：

$$n \in A := \neg \tau_n \wedge (\pi_n = \text{root} \vee \pi_n \in A) \wedge \forall k \in d_n. \exists m \in A. k \in p_m \quad (63)$$

其中 $\pi_n = \text{root}$ 标记编排器插入的线程，否则 π_n 是其激活注册 n 的那个线程。这些条款除 τ, π, d, p 外不读任何字段。两半都把线程关联到紧挨在它下面的一个线程：是父而不是祖先，是直接提供者而不是传递提供者，因为条款读的正是这些；下面各结果需要序的地方取传递闭包，其极小元、极大元与线性化与 $\triangleleft$ 的相同。

这些条款引用 A 自身，所以该定义是沿 $\triangleleft$ 的递归；而正是下面这条使它成为有解的递归。

引理 68. (支撑是良基的) 设 $\prec$ 无环，且 γ 由某个步骤序列达到。则 $\triangleleft$ 是良基的，且 A 是定义 67 的唯一解，是仅关于 τ, π, d, p 的函数。

证明. 按注册每个名字的步骤的下标对 dom(F_γ) 的名字排序——定义 53 通过让序列从空注册表开始提供这一排序。 $\triangleleft$ 的父半部沿该下标下降：O-Insert 以 $\pi \in \text{dom}(F_\gamma)$ 为前提，所以父指针指名一个更早注册的线程；迭代它可在有限多步内到达一个名字的整个祖先链。因此环必须使用 $\prec$ ；而由于 $\prec$ 无环，它必须混合这两者——这需要某个 m 声明一个键，而 m 自己的子树中的某个线程可能提供该键。这样的线程由 m 或 m 的某个后代的激活注册，因而发生在 m 的 L-Begin 之后的某一步；那个 L-Begin 以 $\gamma \models d_m$ 为前提，所以提供该键的线程在它之前就已 Active，而定义 58 的条款 (2) 使该键没有第二个可能的提供者。因此那个会闭合环的线程永远不会被注册，这条边不出现在 dom(F_γ) 中。良基递归有唯一解，而这些条款只读那四个字段。 $\square$

最后一条条款读的是 p ——组件可能提供的键；而 target 读的是 dom(σ_γ)——其线程已安装的键；定义 43 仅以 dom(σ_n) $\subseteq p_n$ 关联两者。因此支撑集一般地过度逼近 Active 线程，而弥合这一差距的条件如下。

定义 69. 当组件 (d, p, e) 的一次完成的激活已安装 p 的每个键——使得在每个实例化它的 Active 线程处 dom(σ_n) = p_n ——时，称该组件在其提供上是完全的 (total)。

与独立性（定义 60）一样，这是只关于组件的条件，不提及任何生命周期状态或步骤；而独立性已经界定了它可能失败到何种程度：如果一个组件只在另一个组件的效应所能达到的上下文状态处安装某个键，那么它的前向映射就不会与那个组件的（前向映射）交换，所以线程安装的键由它的组件决定，而非由调度决定。完全性所额外给出的，是这个被固定的集合是全部 p ，而不是它的一个真子集。

引理 70.（静默处的支撑）设 $\prec$ 无环，设 $\text{quiet}(\gamma)$ ，设 γ 没有失败的线程，并设 γ 的每个组件在其提供上都是完全的（定义 69）。则支撑集就是 Active 线程的集合：

$$A = \{n : \theta_n = \text{Active}(-, -)\} \quad (64)$$

证明. 把右端记作 A' 。由于没有线程失败，定义 49 的静默只留下 $\text{Inactive}(\perp)$ 与 Active 两种状态，并读作

$$n \in A' \iff \text{target}_n(\gamma) \neq \perp$$

由定义 46，右端恰在 $\neg\tau_n$ 且 d_n 的每个键 k 都位于 $\text{dom}(\sigma_\gamma)$ 中时成立；而由定义 69， $\text{dom}(\sigma_\gamma) = \bigcup_{m \in A'} p_m$ 。中间那条条款正是 target 不再携带的，而由注册来提供它： $\pi_n \neq \text{root}$ 的线程只由 π_n 的一次激活注册；若 $\pi_n \notin A'$ ，则 π_n 不处于 Active，所以它的累加器已运行并按定义 47 退役了 n ，给出 τ_n 。因此 A' 满足定义 67 的条款，而引理 68 给出它们的唯一解，所以 $A = A'$ 。□

引理 71.（对换）设步骤两两独立且 F^t 良构，并设步骤 t 与 $t+1$ 作用于不同的线程 m 和 n 。

1. 若两者都应用激活规则（即 L-Begin、L-Iter 或 L-Finish），且步骤 $t+1$ 在 γ_t 处可应用，则步骤 t 在步骤 $t+1$ 从 γ_t 产生的状态处可应用，且两种顺序到达同一个 γ_{t+2} 。
2. 若步骤 t 在 m 处应用激活规则、步骤 $t+1$ 在 n 处应用编排规则，且步骤 t 不注册 n ，则对两者同样成立。

证明. 对于 (1)：由表 1， m 的步骤写入 θ_m ，并在 $\Psi_t \in \mathfrak{M}(e_m)$ 内部写入表 σ_m 和效应部分。因此它不碰 θ_n 与 i_n ；而由定义 60 的第二个条件，它也不碰 i_n 所产出的逆与续延，所以只需检查步骤 $t+1$ 提及 target_n 的那些前提。其退役半部不可能失效，因为没有激活规则写入 τ 。其解析半部也不可能移动：步骤 $t+1$ 在 γ_t 处可应用，把 d_n 的每个键 k 都置于 $\text{dom}(\sigma_\gamma^t)$ 中；而定义 58 的条款 (2) 使提供这样的 k 的线程成为唯一能够提供它的线程，所以 $k \notin p_m$ ， σ 的写入不会触及 d_n 的任何键。反方向的同样论证使步骤 t 可应用。最后，由定义 60 的第一个条件， $\Psi_t \in \mathfrak{M}(e_m)$ 与 $\Psi_{t+1} \in \mathfrak{M}(e_n)$ 交换；两个编辑写入的是不同线程的控制字段，所以两种顺序下的复合相同。

对于 (2)：由表 1，编排步骤有 $\Psi_{t+1} = \text{id}_\Gamma$ ，所以两个状态映射直接交换；其 edit_{t+1} 只写 n 处的 τ_n 或 $\text{dom}(F_\gamma)$ ，而激活步骤既读不到也不写入这些：后者的前提读 θ_m 、 i_m 、 τ_m 与 target_m ；对新鲜 n 的 O-Insert 不移动任何 target （新线程不提供任何东西）；而 n 的 O-Retire 或 O-Remove 使 σ_γ 保持原样——前一种情形中 n 处于 Inactive，后一种情形中其表不受影响。所以步骤 t 保持可应用。反之，编排步骤的每条前提要么是在 n 处读取的（步骤 t 不写这些），要么是 O-Insert 的两条前提之一——更小的注册表只会放宽它们——因此

它在 γ_{t+1} 处的可应用性给出它在 γ_t 处的可应用性；这里正是步骤 t 不注册 n 这一点使 n 在 γ_t 处保持在场，而 O-Retire 和 O-Remove 在需要它如此之处要求它如此。□

引理 72. (删除) 设步骤序列两两独立，设每个组件在其提供上都是完全的 (定义 69)，设它达到一个没有失败线程的静默态 γ_T ，设 $[b, u]$ 是 n 的一个闭合的时段，设序列中没有 $n \prec m$ 的 m 的时段闭合，并设 n 在 $[b, u]$ 期间注册的线程都没有时段。把那些注册所抽取的名字记作 R 。则删除 $[b, u]$ 中作用于 n 的步骤，连同每一个作用于 R 中名字的步骤，留下一个到达与 $\gamma_T \approx$ -相等、且在 R 之外与它 $\approx$ -相等的状态的步骤序列。

证明. 被删除的步骤把状态留在它们发现它的地方。设 $t_1 < \dots < t_l$ 是 $[b, u]$ 中作用于 n 之外线程的步骤。推论 62 读作

$$\gamma_{u+1} \approx (\Psi_{t_l} \circ \dots \circ \Psi_{t_1})(\gamma_b)$$

其右端正是 $[b, u]$ 中幸存的步骤独自产生的东西： $\gamma_{b-1} \approx \gamma_b$ ，且它们的编辑写入的是 n 之外线程的、删除不触及的控制字段。由表 1， n 的被删除步骤除了 θ_n 外不写任何字段；引理 54(4) 在 u 处把它恢复到 $\text{Inactive}(\perp)$ (没有线程失败)，而它在 γ_{b-1} 处也正是这个值。

一个不变量承载着后缀。把幸存的步骤在对应于 t 的时刻所达到的状态记作 γ'_t 。我们断言：对每个 $t > u$ ， $\gamma_t \approx \gamma'_t$ ； R 的每个名字在 γ_t 处都是残留的 (vestigial) 且不出现在 γ'_t 中；两个状态在 R 之外每个名字的每个字段上一致。在 $t = u + 1$ 处，这就是上一段连同定义 47 的内容：定义 47 使 R 的每个名字都被在 u 处运行的累加器退役，处于 $\text{Inactive}(\perp)$ 并持有空表 (按假设， R 的线程没有时段)。归纳步骤是在 R 的每个名字处依次应用引理 57(1)：作用于 R 之外的步骤在两个状态处有相同的前提，再次到达 $\approx$ -相等的状态，并保持 R 的条目残留。作用于 R 中名字的步骤属于被删除的那些；引理 57(2) 正是它必须被删除而非保留的原因——对不存在的名字做 O-Retire 或 O-Remove 没有线程可作用。再由 (1)，这样的步骤不移动 R 之外的任何字段，所以丢弃它保持不变量。因此最终状态 $\approx$ -相等，且在 R 之外相等。

没有幸存步骤失去前提。作用于 $m \notin R \cup \{n\}$ 的步骤只通过 $\text{target}_m(\gamma)$ 或 $\text{relied}_m(\gamma)$ 读取 n 。前者在 m 声明一个 n 提供的键时依赖于 n (从而 $n \prec m$)，也在 n 注册了 m 时依赖于 n (这把 m 放入 R)。在第一种情形中，按假设 m 的时段不闭合，所以它在 γ_T 处是开启的；在那里静默给出 $\omega_m = \text{target}_m^T$ ，引理 70 把它的值置于 Active 线程之中，而 n 不是 Active；由于一个键至多有一个可能的提供者， n 在 m 的 L-Begin 处也没有提供 d_m 的任何键。后者只通过 ω_n 的值读取 n ，而删除该时段只会使 relied 变为假，这放宽 L-Unload 的护栏而不是阻塞它。这样的步骤对 R 中名字的读取由不变量涵盖。两两独立性是效应函数的性质，所以删除步骤保持它。□

定理 73. (合流性) 设一个步骤序列达到没有失败线程的静默态 γ_T ，设步骤两两独立、每个组件在其提供上都是完全的 (定义 69)，并设 A 如定义 67 所述。则

1. (规范形式) γ_T 从 γ_0 出发、不计归约所撤销条目的那些名字，由如下序列达到：该序列按原顺序采取相同的编排步骤——在编排器插入的线程处的那些先于每个生命周期步骤，其余每个都跟在注册它所作用线程的步骤之后——并且对 A 的一个线性化 $\prec$ 的枚举 $n_1, \dots, n_k$ ，按该顺序各采取 n_i 的一个时段。

2. (合流性) 从 γ_0 出发、采取相同编排步骤的任意两条这样的序列，在按引理 56 那样重命名之后，到达由 $\simeq$ 和 $\approx$ 关联的状态。

证明. 对于 (1): 序列的时段有两种: 闭合的, 以及在 γ_T 处仍然开启的; 后者由 quiet_T 与引理 70 可知恰是 A 中每个线程各一个时段。

闭合时段先行, 对其个数归纳。在每个阶段, 挑选一个线程 n 的闭合时段, 其中 n 在时段仍然闭合的线程中是 $\triangleleft$ -极大的; 由引理 68 与 N 的有限性, 这样的 n 存在。于是引理 72 的三条假设都得到满足。由极大性, 没有满足 $n \prec m$ 的 m 有闭合时段。而 n 在 $[b, u]$ 期间注册的线程都没有时段: 这样的线程被在 u 处运行的累加器退役 (定义 47), 并因引理 54(5) 而保持退役, 所以它的目标视图是 $\perp$, 引理 70 把它置于 A 之外, 因此它在 γ_T 处没有开启的时段; 又 $\triangleleft$ 通过它的父指针把它关联到 n , 所以由极大性它也没有闭合的时段。引理移除了该时段以及它所注册名字的步骤, 使 γ_T 在除去那些名字的意义下保持原样。度量减一, 所以不再有闭合时段。

A 之外的线程不采取任何生命周期步骤。由引理 70 与 quiet_T , 它在 γ_T 处没有开启的时段; 现在也不再有闭合的时段, 所以它根本没有时段, 并全程处于 $\text{Inactive}(\perp)$; L-Begin 是在那里唯一可应用的规则, 而应用它会开启一个时段。

接下来是编排步骤。由引理 71(2), 在编排器插入的线程处的编排步骤可以越过不同线程的生命周期步骤提前一个位置; 引理 71(2) 适用, 是因为 A 中线程的步骤不会注册这样的名字: 注册抽取的是新名字, 而这里的名字是原序列的某次 O-Insert 引入的。与同一线程的生命周期步骤之间没有可交换的: n 的 O-Insert 已经先于 n 的每个步骤, 而 n 的 O-Retire 或 O-Remove 只在 A 之外应用, 而 A 之外不采取生命周期步骤。依次把每个移到前面保持它们的相对顺序。在某个激活所注册的线程处的编排步骤不能移到前面——其前提要求那个线程在场——所以它停留在注册把它放到的地方; 由上一段, 它作用于 A 之外, 因此由引理 71 的同一条款, 它与它和注册之间的所有东西交换。

对 $|A|$ 归纳, 把时段排序并使之连续。设 n_1 是 A 中 $\triangleleft$ -极小的。则 $d_{n_1} = \emptyset$ 且 $\pi_{n_1} = \text{root}$, 因为定义 67 把 d_{n_1} 某键的提供者和注册 n_1 的线程都放入 A , 而 $\triangleleft$ 把两者都置于 n_1 之下。所以 target_{n_1} 不读其他线程的任何字段; 也不再编排步骤去写 τ_{n_1} , 也不再有 n_1 之下的线程去退役它, 因此它是恒定的。作用于 n_1 的每个步骤都是激活步骤 (没有时段闭合), 其剩余前提读 θ_{n_1} 与 i_{n_1} , 而由表 1, 只有 n_1 写这两者; 因此每一步在每一个更早的状态处都可应用, 引理 71 在不移动端点的情况下把它提前一个位置。每次应用都使先于 n_1 某一步骤的其他线程的步骤数减一, 所以 n_1 的时段变成一个初始的连续块。该论证在跟随该块的后缀上对 $A \setminus \{n_1\}$ 重复; 在那里 n_1 全程 Active 且不再采取步骤, 所以它也贡献一个恒定的 target。这样产生的枚举按构造线性化 $\triangleleft$ 。

对于 (2): 由 (1), 两条序列都归约到一条规范序列, 而两次归约在不计重命名的意义下遍历同一个 A 。定义 67 读 τ 、 π 、 d 和 p , 其中后三个随线程的条目一次性写入 (引理 54(5)), 所以要看清的是: 同样的名字携带着同样的 d 、 p 、 π 出现, 且同样的名字被退役。插入是两条序列按假设共享的。注册它们也共享: A 中线程的一次激活在其每个迭代处注册迭代器在那里所指名的组件, 而定义 60 的第二个条件使该组件在交错中保持不变, 所以 A -线程之下的注册树是该线程组件的函数; 那些注册抽取的名字不共享——正是在这里应

用引理 56，用双射匹配两棵树。而一次退役要么是编排步骤（共享），要么是累加器执行的 O-Retire——它退役的恰是同一次激活所注册的名字。线性化 $\triangleleft$ 的两个枚举相差不可比较时段的若干对换；引理 71 再次使端点在这些对换下不变，所以两条规范序列一致。结合定理 66 的终止性，生命周期关系因此具有唯一范式。□

失败被排除在结论之外，因为它是分歧的一个真实来源，演算也不应被解读为否认它：一个步骤是否 raise 取决于它所运行于其中的状态，所以一个调度可能使某个纤程失败，而另一个调度却完成它；于是两个静默态在那个纤程的生命周期状态上不同。由推论 62，它们在别的任何方面都不同——推论 62 把失败纤程对状态的贡献置于无。

在第 4.2 节的基础演算中同样的定理成立，而证明除了删去一条条款外不需要任何替换。在那里 L-Unload 不携带护栏，所以引理 72 的最后一段是空洞的；该引理的其余部分只诉诸 quiet_T ，而基础演算原样提供它。

该定理正是允许人们把 Cordis 应用当作静态装配来推理的东西。一个添加组件、移除组件、替换提供者、再撤销替换的编排器，保证会到达它在一开始就把最终组合写下来所能得到的那个状态；而一个推理哪些余效应在作用域内的组件作者，可以只推理静默态。它也划定了保证的边界：它谈论的是状态，而不是系统沿途产生的发射（emission）——这正是第 6.1 节在被追踪于边界之内的获取（acquisition）与跨越边界的发射之间所作的区分。

5 实现与案例研究

本章介绍 Cordis——它将第 3 节的诸形式模型实现为一种实用的编程抽象。Cordis 是时空可组合性的元框架：与针对特定领域（如 Web 路由、ORM、UI 渲染）的应用框架不同，它不规定任何具体场景；其唯一职责是提供通用的动态组合语义。该实现分为三层：（1）核心库（5.1 节）直接实现效应与余效应系统；（2）组件加载器（5.2 节）在核心之上扩展出配置调和与热模块替换；（3）诸如 Koishi（5.3 节）之类的应用框架在前两层之上构建领域特定功能。

5.1 核心库

表 2 概括了理论构造与其运行时对应物之间的对应关系。特别地，本节通篇使用下文引入的运行时名称，而把理论符号保留给形式对应关系。我们还用 @@name 表示框架内部的符号键，因此 $\text{ctx}[\text{@@store}]$ 中的方括号表示对上下文上不透明槽位的符号键访问，而非对字符串键映射的下标访问。

理论 (第 3、4 节)	实现
Γ_∞	<code>ctx</code> , 一等公民上下文
$\gamma \in \Gamma$	上下文树连同运行系统所触及的一切
$\mathfrak{C}_\Gamma$ 、 $\mathfrak{C}_\Gamma^{\text{iter}}$	返回/产出逆的效应回调
$\text{effect}_\Gamma(e)$	<code>ctx.effect(callback)</code>
Σ 、 Σ_{iso} 、 Σ_{inter}	<code>ctx[@@store]</code> 、 <code>ctx[@@isolate]</code> 、 <code>ctx[@@intercept]</code>
$\text{get}(k)$ 、 $\text{set}(k, v)$	<code>ctx.get(key)</code> 、 <code>ctx.set(key, value)</code>
$\text{isolate}(k, r)$	<code>ctx.isolate(key, realm)</code>
$\text{intercept}(k, \nu)$	<code>ctx.intercept(key, metadata)</code>
$\langle d, p, e, \pi, \sigma, \tau, \theta \rangle_\Gamma$	纤程 <code>fiber</code> , 组件在 $\mathfrak{C}$ 中的实例化
$\text{dom}(F_\gamma^\Gamma)$	通过 <code>ctx.registry</code> 枚举
$n : \mathfrak{N}$	<code>fiber.uid</code>
$d : \mathfrak{D}_\Gamma$	<code>fiber.inject</code>
$p : \mathfrak{P}_\Gamma$	组件的提供
$e : \mathfrak{C}_\Gamma^*$	<code>fiber.apply</code>
$\pi : \mathfrak{N}$	<code>fiber.parent.fiber.uid</code> , 拥有其被实例化所在上下文的纤程
派生实现 (定义 27)	<code>fiber.ctx</code> , 纤程运行于其中的子上下文
θ (定义 44)	<code>fiber.state</code> , 生命周期状态, 其 <code>LOADING</code> 对应 <code>Reloading</code> , 其 <code>FAILED</code> 对应 <code>Inactive(\xi)</code>
<code>recover</code> 、累加器 g	<code>fiber.dispose</code> , 累加器
ω (定义 44)	<code>fiber.committed</code> , 已提交视图
$\text{provider}_k(\gamma)$	提供者纤程处于 <code>ACTIVE</code> 的 <code>Impl</code>
$\text{target}(\gamma, n)$	<code>fiber.target</code> , 由 <code>refresh</code> 重新计算 (算法 5), 其中 $\perp$ 即 <code>INACTIVE</code>
<code>Future</code> 、惯性 (4.3.3 节)	<code>fiber.inertia</code> , 进行中迁移的句柄
<code>O-Insert</code> 、 <code>O-Retire</code> (定义 47)	<code>ctx.use</code> 及其回调的逆 (算法 4)
<code>O-Remove</code>	从运行时中移除的纤程, 其 <code>uid</code> 被清除
<code>L-Begin</code> 、 <code>L-Iter</code> 、 <code>L-Finish</code>	<code>execute</code> 的迭代循环 (算法 1)
<code>L-Divert</code>	守卫在迭代边界失败 (算法 1), 或 <code>reload</code> 链入 <code>unload</code>
<code>L-Leave</code>	<code>refresh</code> 将纤程标记为 <code>UNLOADING</code> (第 10 行)
<code>L-Unload</code>	<code>unload</code> 及其惯性链 (算法 5)
<code>L-Unload</code> 上的守卫	<code>unload</code> 等待已通知的依赖方 (第 25 行)
<code>L-Raise</code>	记录在纤程上的错误, 其 <code>target</code> 置为 $\perp$

表 2. 理论—实现对应关系

本节其余部分自底向上构建核心库。5.1.1 节实现可逆效应——修改上下文的唯一原语；5.1.2 节在其之上实现反应式余效应；5.1.3 节把二者组合为组件生命周期；5.1.4 节给出建立在它们之上的上下文级操作。

5.1.1 效应追踪

本节实现可逆效应 (3.1 节)。Cordis 中每一次上下文修改都流经单一原语 `ctx.effect`：余效应提供、组件实例化以及所有其他修改上下文的操作都归结为一次 `ctx.effect` 调用，

因此任何通过上下文执行的操作都会被自动追踪，并在组件卸载时被恢复。在操作层面，`ctx.effect` 是 $\text{effect}^{\text{iter}}$ （定义 52）的实现：它接受一个类型为 $\mathfrak{C}_T^{\text{iter}}$ 的回调，并将其提升为 $\mathfrak{C}_{\partial T}^{\text{iter}}$ ，产出一个 `dispose` 闭包，调用该闭包即恢复效应。Cordis 通过这一操作同时接受 $\mathfrak{C}_T$ 与 $\mathfrak{C}_T^{\text{iter}}$ （特设多态）；我们以迭代器形式为代表，因为普通效应函数正是产出单一逆的退化迭代器。该操作不检查的是 $\mathfrak{C}_T^*$ 所携带的证据：回调提供逆，而该逆确实恢复其所伴随的效应，这是组件作者的责任，而非运行时验证的性质。定理 61 是演算诉诸该性质之处，6.1 节则界定这一责任。

算法 1 展示了 `ctx.effect` 的构造。我们用 $f \circ g$ 表示先运行 g 再运行 f 的释放器，用 `id` 表示空操作；因此每前置一个新的逆即得到后进先出（LIFO）的恢复顺序。

算法 1（效应追踪）

```

1 async function execute(callback, guard)
2   iter ← callback()
3   inverse ← id
4   while guard()
5     (value, done) ← await iter.next()
6     if value then inverse ← value o inverse
7     if done then break
8   return inverse
9 function effect(ctx, callback)
10  armed ← true
11  task ← execute(callback, () -> armed)
12  async function dispose()
13    if not armed then return
14    armed ← false
15    recover ← await task
16    recover()
17  ctx.dispose ← dispose o ctx.dispose
18  return dispose

```

引擎 `execute` 将回调作为效应迭代器（ $\mathfrak{C}_T^{\text{iter}}$ ，定义 51）驱动，并把每一步产出的逆折叠为一个复合逆。在每一步之前，它查询调用方提供的守卫；一旦守卫触发，迭代停止，只保留迄今累积的逆。这正是 4.3.2 节的步边界中断： $\text{Maybe}(\mathfrak{C}_T^{\text{iter}})$ 续延由迭代器的 `done` 标志与 `guard` 共同实现。

`ctx.effect` 是 `execute` 之上的一个薄封装，额外增加两点。其一，自释放：守卫报告 `armed` 标志，返回的 `dispose` 将 `armed` 翻转为 `false`，这同时终止任何进行中的迭代，并使得恢复至多触发一次。触发两次会在一个并非该效应之应用所产生的状态上应用逆，此时没有任何东西保证它会撤销什么。其二，父级组合：`dispose` 被前置到外围上下文的累积逆 `ctx.dispose` 之前，因此子效应的逆本身就是作用于父级的一个效应，这正是 $\partial^2 T$ 的递归结构。组件层（5.1.3 节）复用同一个 `execute`，但守卫测试的是 `fiber.target` 的稳定性而

非 armed。

5.1.2 余效应操作

本节实现反应式余效应 (3.2节)。所有余效应操作都作用于每个上下文携带的三个符号键槽位：

- `@@store`: 值存储 $\sigma : (r : R) \rightarrow \mathcal{V}_r$, 从领域符号到类型化值;
- `@@isolate`: 领域表 $\rho : \text{Map}(K, R)$, 从余效应键到领域符号;
- `@@intercept`: 拦截表 $\iota : (k : K) \rightarrow \mathcal{M}_k$, 为每个键指定其元数据。

前两者组合为两层解析 $k \rightarrow \rho(k) \rightarrow \sigma(\rho(k))$: `ctx.get(key)` (算法 2) 先从 `@@isolate` 读取领域符号 $\rho(k)$, 再从 `@@store` 读取所绑定的值 $\sigma(\rho(k))$ 。 ρ 这一间接层使得隔离能把一个键重定向到独立的绑定; 而 `@@intercept` 只在访问绑定时被查询, 调整的是绑定的使用方式而非其解析结果。我们将这些操作分两部分实现: (1) 提供与通知, 安装或撤回绑定并把变化传播给依赖方; (2) 隔离与拦截, 重塑键的解析方式。

提供与通知。 由于 $\text{set}(k, v)$ 的类型为 $\mathfrak{E}_\Sigma$ (3.1 节), 余效应提供是一次 `ctx.effect` 调用, 并继承其自动追踪与恢复。算法 2 实现 `ctx.set(key, value)`, 即具体的 $\text{set}(k, v)$: 回调把值绑定到存储中领域符号 $\rho(k)$ 之下, 返回的 `dispose` 函数则将其移除。安装与移除都会调用 `notify`, 把变化传播给依赖组件。

算法 2 (余效应操作)

```

1 function get(ctx, key)
2   realm ← ctx[@@isolate][key] # rho(k)
3   return ctx[@@store][realm] # sigma(rho(k))
4 function set(ctx, key, value)
5   function callback()
6     realm ← ctx[@@isolate][key] # rho(k)
7     ctx[@@store][realm] ← value # sigma[rho(k)→v]
8     notify(ctx, [key])
9   return function()
10    delete ctx[@@store][realm] # sigma-rho(k)
11    notify(ctx, [key])
12  return ctx.effect(callback)

```

算法 3 将每次绑定变化传播给依赖方: 对每个活跃纤程, 测试某个变化的键是否出现在其 `fiber.inject` 中并解析到同一领域; 若是, 则调用 `refresh` (5.1.3 节) 以针对新状态重新求值该纤程, 并返回它所重新求值的纤程, 以便调用方可以等待它们。这正是定义 26 的反应式分类: 翻转满足性的变化会激活或停用纤程, 而 `refresh` 的幂等性使中性变化无害。这种重新求值与各种控制流的相互作用在 5.1.3 节展开。

算法 3 (反应式通知)

```

1 function notify(ctx, keys)
2   affected ← {}
3   for fiber in all_fibers do
4     for key in keys do
5       if key in fiber.inject and fiber.ctx[@@isolate][key] = ctx[@@isolate][key] then
6         refresh(fiber)
7       affected ← affected | {fiber}
8     break
9   return affected

```

只有当安装绑定的纤程处于 ACTIVE 时，绑定才算对依赖方可用，因此 `refresh` 针对活跃提供者而非仅针对存储来解析每个声明的键。这正是定义 46 的 `provided-by` 关系，也正是它使撤销在发生之前一步就对依赖方可见：已进入 UNLOADING 的提供者已停止提供，因此其依赖方会重新计算出未满足的目标视图，并在其绑定仍全部就位时开始自身的拆除。

隔离与拦截。这两个操作在结构上做同样的事：各自派生一个子上下文，为 `key` 调整一张继承来的表，而父上下文保持不变，因此恢复是隐式的：丢弃子上下文即可，无需运行任何显式逆。`ctx.isolate(key, realm)` 用 `realm` 覆盖领域映射 ρ （默认使用新生成的符号）（实现 `isolate`，定义 29），因此两个对同一键赋以不同符号的上下文会解析到独立的绑定。`ctx.intercept(key, metadata)` 把 `metadata` 合并进拦截表 ι （实现 `intercept`，定义 31）：按照该定义，新元数据与上下文为 `key` 已携带的任何元数据组合，并优先于后者。

5.1.3 组件生命周期

组件由 `ctx.use` 实例化为一个纤程。本节赋予纤程（5.1 节引入）以 4.3.3 节惯性状态机的操作含义。下面两个字段驱动下述算法：`fiber.parent`，即 `fiber.ctx` 的父上下文，构成组件层次（ Γ_∞ 的递归结构，3.3.1 节）；以及 `fiber.inertia`，进行中异步迁移的句柄（空闲时为 `null`）。

算法 4 展示组件实例化。组件把余效应规范 `component.inject(d)` 与效应函数 `component.apply` 配对；实例化把组件的配置绑定进 `fiber.apply`（第 9 行），即配置应用后的效应函数（ e ），生命周期随后运行它。回调函数（第 2 行）是父纤程中追踪到的效应：执行它时，通过调用 `refresh`（算法 5）启动子组件的生命周期；恢复它时，它把子组件的 `target` 强制置为 $\perp$ 并触发 `unload`。这正是定义 47 的注册原语，其中 `callback` 即其 `O-Insert`，`callback` 返回的闭包即其 `O-Retire`：一次实例化是父级的一个普通追踪效应，因此卸载父组件会级联到其子组件。

算法 4（组件实例化）

```

1 function use(ctx, component, config)
2   function callback()
3     refresh(fiber)
4   return function()

```

```

5 fiber.target ← None
6 unload(fiber)
7 fiber ← Fiber(parent: ctx, inject: component.inject)
8 fiber.ctx ← ctx[fiber -> fiber]
9 fiber.apply ← () -> component.apply(fiber.ctx, config)
10 ctx.effect(callback)
11 return fiber

```

算法 5 实现 4.3.3 节的惯性状态机, 其中 reload 与 unload 是惯性的: 一旦进入, 迁移会运行完毕, 系统才响应目标状态的变化。它使用对余效应存储的两次辅助查询: `resolve(inject)` 返回声明的键当前解析到的绑定, `provided(fiber)` 返回该纤维所安装绑定的键。`refresh` 函数从余效应存储重新计算 `fiber.target`, 并且若纤维尚未处于迁移中, 则启动 reload 或 unload 任务²。`reload` 函数记录当前目标并执行组件的效应函数 `apply`。完成后, 它检查目标是否仍然匹配: 若匹配, 纤维进入 ACTIVE; 若不匹配 (无论新目标是 $\perp$ 还是不同的提供者集合), 则链入 unload。对称地, unload 以 LIFO 顺序恢复所有追踪到的效应, 然后进入 INACTIVE 或链入 reload。这种相互递归实现了惯性性质: 一旦迁移开始, 它会在任何新迁移能够开始之前完成。

算法 5 (组件生命周期)

```

1 function refresh(fiber)
2   target ← target(gamma,n)
3   if target = fiber.target then return
4   fiber.target ← target
5   if fiber.inertia then return
6   if target != None then
7     fiber.state ← LOADING
8     fiber.inertia ← create_task(reload(fiber))
9   else
10    fiber.state ← UNLOADING # out of service before any inverse is scheduled
11    fiber.inertia ← create_task(unload(fiber))
12    async function reload(fiber)
13      target0 ← fiber.target
14      fiber.committed ← resolve(fiber.inject) # commit the view
15      recover ← await execute(fiber.apply, () -> fiber.target = target0)
16      fiber.dispose ← recover o fiber.dispose
17      if fiber.target = target0 then
18        fiber.state ← ACTIVE

```

²`create_task` 调度一个异步函数并发运行, 并返回其句柄 (存入 `fiber.inertia`)。我们显式写出它以保持语言无关性: 在急切调度 (如 TypeScript 的 Promise) 下, 该调用是隐式的, 返回的 promise 即句柄; 而在惰性调度 (如 Python 协程、Rust 的 future) 下, 宿主必须派生子任务它才会推进。

```

19 notify(fiber.ctx, provided(fiber))
20 fiber.inertia ← null
21 else
22 fiber.state ← UNLOADING
23 fiber.inertia ← create_task(unload(fiber))
24 async function unload(fiber)
25   await all(notify(fiber.ctx, provided(fiber)).map(f -> f.await())) # drain dependents
26   await fiber.dispose()
27   fiber.dispose ← id
28   fiber.committed ← None
29   if fiber.target = None then
30     fiber.state ← INACTIVE
31     fiber.inertia ← null
32   else
33     fiber.state ← LOADING
34     fiber.inertia ← create_task(reload(fiber))

```

`fiber.target` 通过把每个声明的键针对当前余效应存储解析、并元组化提供该键的纤维的 `uid` 来计算，因此它是 $\text{target}(\gamma, n)$ (定义 46) 的一个摘要。按提供者而非按值来识别绑定，正是使一次对已记录目标的比较就足够的原因：`uid` 每次新取且从不复用，因此被替换的提供者不会被误认为替换它的那一个，即使两者提供相等的值。由于 `notify` (5.1.2 节) 在每次余效应变化时都重新计算 `target`，纤维恰在其某个声明键改由不同纤维提供时重新加载。因此，原地覆盖自己绑定的提供者不会被观察到；想让替换传播出去的组件会撤回该绑定并重新安装它。

该算法在两个互补的层面运作。在迁移层面，`reload` 与 `unload` 在完成时检查 `target`，从而实现跨迁移的惯性链。在每个迁移内部的迭代层面，效应执行 (算法 1) 在每个迭代边界检查 `target`，从而实现在单个迁移内的部分回滚。这两种机制分别对应 4.3.3 节的迁移间链与定理 64 所依赖的迁移内过期检查。

三行代码承载定理 63 的余效应排序，而每一行所处的位置正是该排序成立的原因。`reload` 在第 14 行提交解析得到的视图，`unload` 只在每个逆都运行完毕后才丢弃它，因此只要纤维处于加载状态 (其自身的拆除也包含在内)，它就读到相同的绑定。`refresh` 在迁移任务创建之前于第 10 行把纤维标记为 `UNLOADING`，这正是 `L-Leave` 步：纤维停止提供，依赖方在它的任何逆被调度之前就据此重新计算。随后 `unload` 在第 25 行等待每个被通知的依赖方到达 `INACTIVE`，这正是 `L-Unload` 上的守卫；`notify` 只有在依赖方声明的键解析到与提供者相同的领域符号时才接纳该依赖方，这是该守卫之要求的运行时形态——依赖方必须从该纤维看到这个键，而非仅仅声明它。等待位于整个恢复过程之前，而不是位于某个被等待的逆内部，因为 `fiber.dispose` 并发地启动一个纤维的诸效应，若把等待放在其中一个内部，其余效应就会失去顺序。终止性由定理 66 保证：纤维只等待那些已经不再可满足的依赖方，而本身也是提供者的依赖方以同样的方式等待它自己的依赖方，因此提供者图是按需遍历的，而非预先分析。

5.1.4 上下文访问

5.1.2 节的余效应操作构成一个反射式 API：余效应用 `ctx.set(key, value)` 写入、用 `ctx.get(key)` 读取，二者都以名称作为键。Cordis 在此反射式 API 之上又叠加了第二种更原生的扩展与消费上下文的方式：属性访问。组件可以把余效应作为属性 `ctx[key]` 来访问，仿佛它是上下文自身的原生结构，而无需通过方法调用。在 TypeScript 中，Cordis 用一个 Proxy 实现这一点，其 get 陷阱 (trap) 中介每一次属性访问。算法 6 展示上下文如何在 5.1.2 节的原语 `get` 之上把这样一次访问解析为一个余效应。

算法 6 (代理中介的上下文访问)

```

1 function resolve(ctx, key)
2   fiber ← ctx.fiber
3   repeat
4     if key in fiber.committed then return fiber.committed[key]
5     if key in fiber.inject then throw INACTIVE_ACCESS
6     if fiber = root then throw UNDECLARED_ACCESS
7     fiber ← fiber.parent.fiber

```

算法 6 从访问上下文出发沿纤程链向上遍历：在第一个已提交视图绑定 `key` 的纤程处，访问被授权并返回该绑定；若遍历到达一个声明了 `key` 却尚未提交它的纤程，则该纤程未加载，访问失败；若到达根而没有任何声明，则访问被以未声明为由拒绝。这正是代理与裸 `ctx.get` 的区别所在：`ctx.get(key)` 是对存储的一次查找，返回绑定的值或空，从不失败；而代理针对访问纤程自身的视图解析，并在使用点强制余效应规范 d 。读取视图而非存储也是定理 63 所依赖的，因为正是这一点使得依赖对该组件保持可读——即使该组件的拆除正是由该依赖的离去所触发的。

这种拒绝是在访问点执行的运行时检查。由于组件的余效应规范 d 是静态声明的，同样的违规原则上可在编译期检测到——在运行前针对声明的 d 解析每个 `ctx[key]` 即可；6.4 节讨论宿主语言的类型级依赖声明与编译期元编程如何恰好完成这种中介。

5.2 组件加载器

核心库为组件开发者提供动态组合的命令式原语，如 `ctx.effect`、`ctx.use` 和 `ctx.set`。应用编排者面对的是一个不同的问题：他们把现成组件装配成运行中的系统，并在系统存续期内调整组合。组件加载器通过引入声明式配置层来解决这一问题：编排者把期望的组合指定为一个持久数据结构，加载器则把对该规范的变更翻译为相应的命令式纤程操作。

5.2.1 声明式配置

第 4 节把运行中的系统分解为纤程，每个纤程是一个组件的实例化。实例化所需的一切都可以声明，因此编排者可以把整个系统描述为一份声明式配置：一个持久记录，加载器把它实现为纤程并与之保持同步。

条目。配置由条目组成。每个条目指定并管理一个线程，且这种绑定是双向的：加载器通过调整线程来响应条目字段的变化，而修订自身配置或禁用自身的组件，其变化会被写回它的条目。

定义 74. 条目声明单个线程，记录：

- `id`——稳定标识符，当其所在组的子列表变化时用作调和键；
- `url`——要实例化的组件模块的 URL；
- `isolate`——应用于条目上下文的隔离注解；
- `intercept`——应用于条目上下文的拦截注解；
- `config`——绑定进组件以构成其效应函数 `apply` 的配置；
- `disabled`——条目是否被管理上关闭。

条目能够充当忠实的规范，是因为支撑一个线程的正是条目记录的内容。定义 67 的支撑集只读取 τ 、 π 、 d 和 p ，别无其他；而条目恰好给出全部四个：`disabled` 给出 τ ，条目在树中的父级给出 π ，`url` 选出声明 d 与 p 的组件。支撑集不读取的字段是线程的运行时状态——实例化同样不需要它们；引理 70 把支撑集等同于静默态（定义 49）中的 **Active** 线程，前提是每个组件都安装它声明的每个键（定义 69）。

这些条目构成配置树，它是系统加载内容的权威记录。条目可以是映射到单个线程的叶子，其组件也可以转而加载更多组件，使条目成为分支节点。`Cordis` 为此类分组与嵌套加载提供组件：`@cordisjs/group` 把子条目列表作为其配置并作为一个子组加载它们；`@cordisjs/include` 加载外部配置文件（YAML 或 JSON）并把它包含的条目作为嵌套子树嫁接进来。两者都是建立在定义 47（算法 4）注册原语之上的普通组件，因此嵌套树仍处于演算之内，下述结果对它同样成立。

调和。当条目的记录变化时，加载器增量调和，而不是拆除线程并整体重建。这样调和是健全的，其原因由元理论给出。

- 定理 73 使静默态成为最终配置本身的函数：无论加载器在途中执行多少次实例化与退役、以何种顺序执行，系统最终静默于从零加载最终配置所会到达的状态。最终加载了哪些组件，只能依据声明读出——前提是每个组件都安装它声明的每个键（定义 69）；一个声明了某键却只在某些配置下安装它的组件，加载器仍可调和，但此时已加载组件的集合也随这些配置而定。
- 定理 66 证明系统确实会静默，因此一旦其所需的实例化与退役均已发出，一次调和即告完成。
- 推论 62 表明离去的线程对状态的贡献为零，因此重建一个条目会撤回其线程所安装的内容，并使其周围的线程保持原样。

- 定理 63 允许各条目一起实例化，无需编排者安排加载顺序：声明键尚未得到提供的线程在它的 L-Begin 处等待，而提供者离去的线程会在它之前被停用。因此依赖约束的是线程何时激活，而非其模块何时被获取和求值，所以加载器并发地加载模块——这正是拉起大型配置所耗费时间的地方。

在条目声明的线程之上，加载器按条目的哪个字段发生变化进行分派，并对每个字段应用扰动最小的操作。

- `id`、`url`——重建条目，因为其身份或其组件已变；
- `isolate`——重新指派条目的领域（算法 7）；
- `intercept`——原地更新，因为拦截元数据在读取时被查询，无需重新加载；
- `config`——交给组件，由组件决定如何应用新载荷，典型地是与前一个载荷做差异比较，仅在实质性变化时重新加载。特别地，`@cordisjs/group` 条目的 `config` 是其子条目列表，因此它以子 `id` 上的键控差异（keyed diff）应用更新，创建、移除或更新每个子条目；由于更新幸存的子条目会重新进入同样的按字段分派，组调和与条目更新会一起沿树向下递归；
- `disabled`——置位时卸载线程，清除时重新加载。

托管领域。核心库中的隔离派生一个子上下文，在某个键上覆盖领域表 ρ (5.1.2 节)；当上下文树静止不动时这已经足够。条目可能在运行时被移动到其他组，因此加载器自行管理领域，`isolate` 字段在每个键上二选一地选择两种作用域规则。值为 `true` 表示需要一个局部领域，它对条目私有并以该条目的 `id` 作标记，条目无论移到哪里都随身携带它；值为字符串表示需要一个全局领域，由每个以该字符串命名的条目共享，因此移动这样的条目改变的是它与哪些条目共享绑定，而非它属于哪个领域。一旦没有条目以某个领域命名，该领域即被丢弃。

重新指派条目的领域取决于：哪些键改变了领域、条目自身是否是某个变化键处的提供者、以及要通知哪些依赖方。中间那个问题最难，因为一个领域符号可能被多个线程共享，而其中只有一个是提供者。加载器用定界符（delimiter）回答它：每个键一个符号 δ_k ，每个上下文在其下存储自己的标记。定界符写在上下文上并由其后代继承，因此条目的标记与提供者的标记恰好一致，当且仅当二者是在针对 k 的同一个 `isolate` 作用域内派生出来的——这正是 k 处的绑定属于条目自身、必须随其移动的情形。

算法 7（隔离领域重新指派）

```

1 function patch_isolation(entry, rho')
2   rho ← entry.ctx[@@isolate]
3   store ← entry.ctx[@@store]
4   Δ ← {k | rho(k) ≠ rho'(k)} # keys whose realm changes
5   for k in Δ do

```

```

6 entry.ctx[deltak] ← fresh tag
7 diff[k] ← (rho(k), rho'(k), entry.ctx[deltak], store[rho(k)].fiber.ctx[deltak])
8 entry.ctx[@@isolate] ← rho'
9 reload(entry.fiber)
10 for k in Δ do
11 (s1, s2, d1, d2) ← diff[k]
12 if d1=d2 and store[s1] and not store[s2] then # the binding is the entry's own
13 store[s2] ← store[s1]
14 delete store[s1]
15 function affected(fiber, k)
16 (s1, s2, d1, d2) ← diff[k]
17 return fiber.ctx[@@isolate][k] in {s1, s2} and (fiber.ctx[deltak] = d1) != (d2=d1)
18 notify(entry.ctx, Δ, affected) # in place of the realm test of Algorithm 3

```

该测试依赖于定界符的一个性质。 δ_k 之下的标记写在条目的上下文上，并由从它派生的每个上下文继承，且在每次重新指派时新取，因此对上下文 γ' ：

$$\gamma'[\delta_k] = d_1 \iff \gamma' \text{ is derived from the entry's context} \quad (65)$$

用 $\text{own}(\gamma')$ 表示该条件，而 $d_2 = d_1$ 正是它在提供者处的实例。重新指派把满足 own 的上下文从 s_1 移到 s_2 ，其余保持不变；由上面的循环，它恰好当提供者满足 own 时把绑定移到 s_2 。依赖方在 k 处自身的领域正是绑定所在领域时看到该绑定。在 own 对依赖方与提供者一致的地方，二者要么都移动要么都不移动，因此依赖方之后看到该绑定，当且仅当它之前看到它。在 own 将二者分开的地方，一侧移动而另一侧停留，因此依赖方获得或失去该绑定。该不等式所表达的正是这种分离，而成员测试会丢弃那些在任一领域中都不解析 k 的依赖方——移动的任何部分都触及不到它们。

5.2.2 热模块替换

热模块替换（HMR）在模块层面应用可逆效应模式：当源文件变化时（典型地发生在开发期间），系统在不重启进程的情况下原地替换受影响的模块。由于线程已经界定了其组件所有的效应与余效应，本身是组件的模块仅通过线程操作即可替换：释放旧线程会恢复组件安装的一切，而从重新加载的模块实例化的新线程会重新安装它。因此 HMR 不需要开发者标注的接受边界，这与 Webpack [46] 或 Vite [47] 的 HMR 不同。

@cordisjs/hmr 组件提供 HMR 引擎，它分三个阶段运作。

阶段 1：模块分类。引擎接受两个输入：暂存集（自上次重新加载以来内容发生变化的文件 URL）与外部集（不能被热替换、转而触发完全重启的模块）。用 `get_imports(url)` 表示 url 直接导入的模块，它对变更的依赖子图进行分类，把每个模块标记为 `accepted` 或 `declined`：

算法 8（模块分类）

```
1 function classify(stashed, externals)
2   accepted ← stashed
3   declined ← externals
4   pending ← {}
5   for url in stashed do
6     pending ← pending | (get_imports(url) - (accepted | declined))
7   repeat
8     progress ← false
9     for url in pending do
10      if get_imports(url) & accepted != {} then
11        accepted ← accepted | {url}
12        pending ← pending - {url}
13        progress ← true
14      else if get_imports(url) <= declined then
15        declined ← declined | {url}
16        pending ← pending - {url}
17        progress ← true
18      else
19        pending ← pending | (get_imports(url) - (accepted | declined))
20    until not progress
21    declined ← declined | pending
22  return (accepted, declined)
```

以暂存文件的导入为种子，这一不动点在一个模块的某个导入被接受时接受它，在它的所有导入都被拒绝时拒绝它；任何仍未决的模块——例如陷入导入环的模块——默认归入 declined。

阶段 2：过期条目检测。利用 accepted 与 declined，引擎随后把组件条目过滤到过期的那些——其依赖树触及某个被更改模块的条目。它用 `get_dependencies` 遍历每个条目的树，该函数收集一个模块的传递导入，同时把 declined 视为边界：

算法 9（过期条目检测）

```
1 function get_dependencies(root, declined)
2   deps ← {}
3   function traverse(url)
4     if url in deps or url in declined then return
5     deps ← deps | {url}
6     for child in get_imports(url) do traverse(child)
7   traverse(root)
8   return deps
9 function detect(entries, accepted, declined)
```

```

10 stale_entries ← {}
11 for entry in entries do
12   tree ← get_dependencies(entry.url, declined)
13   if tree & accepted != {} then
14     accepted ← accepted | tree
15   stale_entries ← stale_entries | {entry}
16 return stale_entries

```

条目恰好在其树与 `accepted` 相交时是过期的；随后该树被并入 `accepted`，因此沿树的所有过期模块在下一阶段被作废。

阶段 3：事务性重新加载。最后，引擎重新加载过期的条目。它使 `accepted` 模块的缓存失效³，备份每个被移除的模块以支持回滚，然后按 `url` 重新导入每个过期条目的组件模块并换入一个新的线程：

算法 10（事务性模块重新加载）

```

1 function reload(ctx, accepted, stale_entries)
2   backup ← invalidate_caches(accepted)
3   try
4     for entry in stale_entries do
5       entry.fiber.dispose()
6       entry.fiber ← ctx.use(import(entry.url), entry.config)
7     catch error
8       restore_caches(backup)
9     for entry in stale_entries do
10      entry.fiber.dispose()
11      entry.fiber ← ctx.use(backup[entry.url], entry.config)
12    throw error

```

事务性保证确保系统绝不进入半重新加载状态：若任何模块导入失败（例如由于语法错误），则恢复缓存，每个过期条目都从 `backup[entry.url]`——即缓存刚被恢复的前一个组件——重建，从而撤销已经完成的换入。

5.3 案例研究：Koishi

Koishi 是构建于 Cordis 之上的开源聊天机器人应用框架⁴。经过四年的开发，它积累了 4000 多个社区贡献的插件⁵，从即时通讯（IM）适配器、数据库驱动到管理控制台与终端用

³在 Node.js 上，这意味着清空 ES 模块与 CommonJS 模块两个系统的缓存，因为通过 ES 加载器导入的模块可能同时出现在两者中。

⁴Koishi 目前使用 Cordis v3。本文介绍的是 Cordis v4，它细化了效应与余效应语义并重新设计了加载器；两版共享相同的核心组合模型。

⁵Koishi 用 `plugin`（插件）一词指代本文形式化为 `component`（组件）的概念。

户功能，不一而足。它的规模与多样性使其成为对 Cordis 动态组合能力在生产环境中的代表性验证。

元框架的表达力与普适性。Koishi 作为服务端机器人运行，其每个功能都实现为基于 5.1 节上下文原语的插件；Koishi 本身只贡献聊天机器人领域的词汇。同样的模型在完全不同的运行时中重现：Koishi 的 Web 控制台是第二个独立的 Cordis 应用，其插件组合的是浏览器及其用户界面的原语，而非服务端的原语。上述迥异的场景确立了第 3 节模型的两个性质。(1) 它富有表达力：其原语足以承载一个完整的生产系统，宿主框架只提供领域词汇。(2) 它具有普适性：它固定了效应与余效应的组合方式，而把它们含义留给各个应用，因此既不预设特定领域，也不预设特定运行时。

无认知开销的时间可组合性。1.2.1 节综述的插件系统无法在不重启扩展的情况下卸载单个扩展的效应；宿主 Koishi 例行执行这一操作：编排器 (orchestrator) 从控制台停用某个插件，其效应被就地撤销；在开发期间，HMR 引擎会在保存时重新应用编辑过的插件，同时保持系统中其他位置的缓存状态与活跃连接不变。Cordis 使这种移除对插件作者而言不仅可行，而且毫不费力。由于通过上下文执行的效应会被追踪，且其逆元会被自动复合（第 3.1 节），即使是缺乏经验的作者，也能在不编写卸载路径的情况下，为插件的上下文中介效应 (context-mediated effects) 获得有序的清理事。这实现了关注局部性 (locality of concern)——第 1.2.1 节所指出缺失的正是这种局部性：本应依赖每位作者勤勉才能保证的正确性，改由该抽象一次性承担。

跨开放生态系统的空间可组合性。与第 1.2.1 节中插件间依赖基本缺失的插件系统相比，Koishi 的生态系统呈现出真实的依赖拓扑：IM 适配器提供对各消息平台的访问，数据库驱动提供持久化存储，功能插件将这些声明为余效应并加以访问。在运行时重新配置提供者（例如切换存储后端或重连适配器）只会重新激活那些已解析依赖发生变化的依赖者（第 3.2 节）；依赖不可用的插件会保持非激活状态，直到该依赖出现，而不会报错。该案例研究证实的是：这种组合在独立编写的代码之间依然成立——插件与其依赖通常由不同作者编写，他们除了连接彼此的余效应之外无需协调任何其他事项，因此反应式余效应能在由独立贡献者构成的开放生态系统中保持组装体的一致性。

有效性威胁 (threats to validity)。此处的证据取自单一宿主语言中的单一生态系统，因此无法将范式的优点与其 TypeScript 实现或 Koishi 特定领域的优点区分开来；而且它是观察性的，而非与替代架构进行的受控对比。因此，该案例研究所确立的是一种存在性与采纳性 (existence-and-adoption) 结果，而非定量结果；以基线为对照测量该抽象的开销及其对开发者生产力的影响，仍是未来的工作。

6 讨论

前面各节给出的形式模型与实现引入了一种面向动态组合的编程范式。本节考察该范式如何推广到更广泛的工程关注点，并讨论其中的设计张力与开放问题。

6.1 系统边界

第 3.1 节中的每个效应都带有逆元，而该逆元究竟意味着什么，由系统边界 (system boundary) 决定。边界把系统所运行于其中的环境划分为两部分。(1) 当系统能够独占性地修改某位置 (location) 并在该修改发生之前恢复其状态时，该位置位于边界之内，因此对它的操作会在 Γ 中被追踪，日后可以恢复。(2) 当上述两种能力有任一不具备时，该位置位于边界之外，因此对它的操作表现为 id_Γ ，既不被追踪也不被恢复。本节阐述该边界的性质及其对恢复的推论。

由余效应产生的边界。余效应通过具体化 (reify) 外部位置来移动边界：它把对该位置的每次访问都限制在它所提供的一组操作内，并为其中每个操作都能提供逆元，于是原本表现为 id 的操作开始被记录在 Γ 中并得以恢复。因此， Γ 边界是按位置而非按介质划定的，因为上述两种能力都是位置的性质，而具体化改变的是位置的访问方式，同时保持其介质不变。例如，当只有系统写入某内存区域时，该区域位于边界之内；当其他进程也写入它时，则位于边界之外。文件亦然：当只有系统能够访问它时（例如私有路径下的临时文件），它位于边界之内；当它是其他程序读写路径时，则位于边界之外。移动边界本身是一种权衡：一边是环境是否为某个位置提供可逆语义，另一边是为每次访问提供这些语义所付出的代价。我们将在第 6.7 节讨论这一权衡所暗示的协同设计。

获取与发射 (acquisition and emission)。触及边界之外的操作通常分两个阶段进行。(1) 在获取 (acquisition) 阶段，操作获得访问权并在边界内安装一条记录：`open` 安装一个由 `close` 移除的描述符，`malloc` 保留一个由 `free` 释放的内存块，`fork` 启动一个由 `kill` 终止的子进程。该记录本身是具体化该位置的余效应的一部分，例如余效应所维护映射中的一项，而安装这一条目是一个可逆效应。该记录同时是数据得以离开的通道。(2) 在发射 (emission) 阶段，操作通过该通道推送数据，如同 `write` 交给文件的字节或 `send` 放到线路上的数据报；这一推送表现为 id_Γ ，把数据留在其他各方可以读写之处。因此，两个阶段落在边界的两侧：获取停留在边界之内，而发射则跨越到边界之外。

扣留与补偿 (withholding and compensation)。尽管如此仍必须从发射中恢复的系统有两种可用途径。其一是扣留 (withhold) 发射，直到产生该发射的状态确定能够持久化——这就是回滚恢复 (rollback-recovery) 中的输出提交 (output commit) 问题 [48]。其二是补偿 (compensation) [49]：一种把状态恢复到应用所提供的某个等价关系的动作，该等价关系比定义 33 的 $\simeq$ 更粗，例如删除一个已创建的文件或退还一笔已收取的费用。这类动作与逆元一样按后进先出 (LIFO) 顺序复合，因此第 3.1 节的复合可以直接迁移到它们身上。但元理论不能：定义 60 的可交换性是基于 $\simeq$ 证明的，必须针对更粗的等价关系重新建立。

6.2 服务多路复用

OSGi [50] 等动态组件平台围绕服务组织组合：服务是提供者在某接口下发布、消费者绑定的一类功能单元。Cordis 的余效应模型呼应了这一概念：服务对应于键背后的接口。提供服务的组件是它的提供者，注入服务的组件是它的消费者。单个服务可以由多个提供者实现，这种多重性可以通过两种形式实现。(1) 独占绑定 (exclusive binding)：多个实现共享一个接口，但同一时刻至多绑定一个；由编排器选择绑定哪个实现，而切换需要在卸载一个

提供者的同时加载另一个，会瞬时扰动每个消费者的依赖。(2) 服务代理 (service broker)：一个充当该接口入口点的中心服务，由后端的提供者和消费者双方注入，从而多个提供者共存，代理在它们之间分发每个请求。与独占绑定相比，代理吸收了这种扰动：更新后端提供者时代理保持原位，因此消费者看不到自身依赖的任何变化，也不会触发重载。

服务代理支撑三种能力：负载均衡、滚动更新与跨进程调用。

负载均衡 (load balancing)。当多个提供者共存时，代理按照可配置的策略（例如轮询、最少负载、按延迟加权）或消费者指定的显式目标，在它们之间分发请求。由于提供者是普通组件，可以通过添加或移除来扩大或缩小容量；每个提供者都通过一个可逆效应向代理注册，因此卸载提供者会撤销注册，并自动将其从代理的路由集合中移除。

滚动更新 (rolling updates)。在运行时升级服务实现可以归结为一次受控的提供者迁移 (provider transition) [51, 52]。为执行迁移，新提供者作为额外的线程被加载并向代理注册；一旦它变为 ACTIVE 状态，流量便从旧提供者逐渐转移到新提供者（例如通过调整选择权重），旧提供者在不再承载进行中的请求后被卸载。这种提供者迁移把传统上属于基础设施层面的操作（例如容器编排、蓝绿部署）转变为应用层面的组合模式。

跨进程调用 (cross-process invocation)。服务代理也可以跨越进程边界使用 [53]。每个进程承载自己的 Cordis 上下文及本地提供者；一个协调组件将它们连接起来，把每个进程视为远程提供者。跨进程的服务访问由保持接口不变的 RPC 机制进行中介，从而使这种分布对消费者透明。一个需要注意的地方是：跨进程调用会产生延迟，并且可能在途中失败，因此同步暴露它会阻塞调用者。因此，打算跨进程暴露的接口必须按照异步契约来设计。

6.3 访问控制与沙箱化

对于由独立组件组装而成的应用，保障其安全需要两种互补机制：(1) 限制组件可以访问的依赖；(2) 将不可信代码与宿主环境进行沙箱化隔离。Cordis 通过依赖声明与拦截来支持第一种；第二种则需要外部的沙箱。

基于能力的访问控制 (capability-based access control)。依赖访问机制（第 5.1.4 节）已经构成了一种针对代理中介 (proxy-mediated) 属性的访问控制：组件只能访问它已声明的依赖；未声明的访问会引发错误。这在结构上类似于基于能力的安全模型 [54–56]：权威 (authority) 通过持有引用而非环境权威 (ambient authority) 来授予。inject 声明充当能力请求，上下文代理充当能力中介者。由于这些请求是静态声明的，组件所需的全部代理中介能力在它运行之前就已知晓，这让编排器能够在加载时审查并批准它们，而不是在访问发生时才发现。

这种中介通过拦截机制推广为细粒度策略。访问控制元数据可以由上下文携带或由组件声明（定义 30），提供者在依赖被调用时查阅它以决定请求是否被允许。例如，文件系统依赖可以携带声明组件可读写哪些路径的元数据，提供者对每次调用按元数据进行检查。由于这种拦截存在于上下文上而非任何一方的代码中，编排器可以调整它以约束任何组件对某依赖的访问而无需修改提供者，例如向社区组件授予只读数据库访问权，而核心组件保留完整访问权。此外，由于拦截只影响依赖的调用方式，而不影响其是否被满足，它可以在运行时被安装、重新配置或移除，而不会触发任何重载或扰动依赖图。

对不可信组件的沙箱化 (sandboxing untrusted components)。当组件的代码不可信时，语言层面的访问控制是不够的，因为能够访问宿主运行时的恶意组件可以直接触及底层对象，使这类检查形同虚设。沙箱化需要一个语言层面手段无法触及的执行边界，例如软件故障隔离 [57]、独立语言运行时、沙箱化进程或虚拟化容器 [58]。无论采用何种机制，不可信组件都在自己的沙箱化上下文中运行，并通过一座桥 (bridge) 访问宿主提供的依赖，这是对第 6.2 节跨进程调用的推广：同样的透明性论证使这种桥接访问与本地注入无法区分。在宿主一侧，桥是一个普通线程，其能力可以被上述访问控制所衰减 (attenuate)。

6.4 语言无关性与选择

尽管 Cordis 是用 TypeScript 实现的，上下文范式与语言无关：时空可组合性仅由它的两个可组合性维度定义，因此可以在任何沿这两个维度满足特定要求的语言中实现。下面我们逐一分析这两个维度上的要求。

时间可组合性。时间可组合性最基本的要求是闭包：可逆效应把动作与逆元配对，而该逆元必须连同它所恢复的状态一起作为值被捕获，以便在拆除 (teardown) 时重放。除此之外，组件的代码以及加载它所产生的副作用必须能够在运行时引入和收回 (retract)。

语言如何满足这第二个要求取决于其执行模型。在托管运行时中，这表现为程序化模块注册表 (module registry)：已加载的模块可以在不再被引用时从注册表中被驱逐并被垃圾回收；例如，Node.js 就暴露了这样的注册表。⁶原生代码不暴露模块注册表，因此引入和收回表现为显式的动态链接与解链（例如 Unix 上的 `dlopen/dlclose`、Windows 上的 `LoadLibrary/FreeLibrary`）[59]，即把目标代码加载进运行中的进程，之后再将其分离。WebAssembly 依据其嵌入环境 (embedder) 选择其中一条路径：在托管嵌入环境下（例如 JavaScript 宿主），模块实例由宿主的收集器回收；在原生嵌入环境下（例如 Wasmtime），模块实例在宿主丢弃它时被释放。在这些机制之上，可逆效应模型把加载视为对上下文的一次效应，其逆元撤销模块引入的符号、类型或处理程序的注册。

空间可组合性。空间可组合性要求具备组件声明其依赖、运行时提供并注入这些依赖的机制。这归结为依赖注入 (dependency injection, DI) 问题 [38]，它在两个随语言而异的层面上显现：依赖如何被类型化，以及依赖访问如何被中介。

在类型层面，语言应为开发者提供表达良类型依赖访问的方式。消费者通过从上下文读取键来获得余效应，因此上下文类型（第 3.2.1 节）必须记录每个键的余效应。类型类 (typeclass, Haskell) [60] 和 trait (Rust) [61] 通过让提供者从自身模块以 `instance` 或 `impl` [62] 扩展上下文类型来实现这一点。TypeScript 的模块扩充 (module augmentation) [63] 同样让提供者模块把声明合并进上下文类型。

在运行时层面，依赖访问必须被动态中介：键背后的余效应可能随着提供者的加载与卸载而改变，并且可能在不同上下文中解析出不同的结果。因此，语言需要一种透明地介入 (interpose) 访问的方式，保持消费者代码不变，例如通过 JavaScript 的 Proxy 对象 [64] 或 Python 的描述符协议 (descriptor protocol, `__get__`) [65]。在缺乏这种原语的情况下，运

⁶CommonJS 通过 `require.cache` 暴露模块缓存；ES 模块没有公开的驱逐 API，不过模块仍可通过引擎内部接口来管理。

行时反射 [66, 67] 可以动态中介访问，但代价是类型安全与开发者体验。

跨越这两个层面，元编程设施可以同时提供类型化与中介。注解 (annotation) [68] 和装饰器 (decorator) 把元数据附加到声明上，处理器将其展开为中介访问的访问器；编译期元编程（例如 Rust 过程宏、Scala 宏 [69]、Zig 的 `comptime`）为每个依赖生成一个带类型的声明以及这样的访问器，从而免去通用拦截原语。

6.5 相互依赖与组件粒度

在反应式余效应模型中，依赖环只会让涉及的组件永久保持非激活：给定两个组件 A 和 B ，如果 A 需要 B 提供的键而 B 需要 A 提供的键，那么两者的满足谓词都不可能变为真。与并发系统中的死锁不同——死锁依赖于调度，必须在发生时才能检测到——这一情形仅凭依赖声明就可预测，因此运行时可以在组件加载时报告它。

在实践中，大多数表面上的相互依赖都可以分解为消除环的更细粒度组件。考虑两个组件：服务器（提供网络接口）和访问控制器（实施授权策略）。这两个组件是双向交互的：访问控制器中介到达服务器的请求，而服务器暴露一个用于修改访问控制策略的端点。单体设计会让每个组件都依赖另一个。然而，这两个交互方向在逻辑上是相互独立的关注点。将它们分解得到四个组件：`server-core`（服务器核心）、`access-control-core`（访问控制核心）、`request-mediation`（请求中介，依赖两个核心以对入站请求施加访问控制）和 `policy-management`（策略管理，依赖两个核心以通过服务器暴露策略修改）。通过这种方式，环被消除，因为两个核心互不依赖；只有集成组件同时依赖两者。

这种分解原则上总是可行的，因为每个双向交互都可以分解为独立的单向绑定，但它会增加组件数量：在一般情况下，给定 n 个相互交互的组件，集成组件的数量可能随 n 二次增长，因为每对交互组件在每个交互方向上可能都需要一个独立的组件。这不会影响正确性或运行时性能（组件是轻量的），而且更细的粒度可能是有益的：用户获得只加载他们所需的具体集成绑定的能力，从而有效提高系统的可组合性。然而，它可能影响开发者体验：更多组件需要更多配置、更多命名，以及理解依赖图时更多的认知开销。

缓解这种粒度成本是一个工程问题而非理论问题。实用策略包括包捆绑 (package bundling, 即将相关的细粒度组件归入一个可安装单元)、基于约定的接线 (convention-based wiring, 即自动连接名称或类型匹配某种模式的组件) 以及脚手架工具 (scaffold tooling, 即从声明式规范生成样板集成组件)。这些策略在保持无环模型的形式保证的同时，把编写负担降低到接近单体情形。

6.6 依赖类型化与版本管理

在形式模型中，依赖链接纯粹由键的同一性建立：提供键 k 的组件满足任何在其依赖集合中声明 k 的组件。类型族 $\mathcal{V}_k$ 保证单个编译单元内的类型层面一致性，但当组件被独立开发和构建时——这在组件生态系统中是常见情形——这种保证就会失效。这一失效导致两个不同的问题。

接口漂移 (interface drift)。提供者可能在版本之间修改与 k 关联的接口（添加字段、

改变方法签名、变更行为契约)，而针对较早接口编译的消费者继续声明同一个键 k 。依赖在余效应层面得到满足 ($k \in \text{dom}(\sigma)$)，但运行时值不再符合消费者的预期，导致类型错误、方法未找到 (method-not-found) 失败或静默的行为分叉 [70]。

键冲突 (key collision)。两个独立开发的提供者可能使用同一个键名 k 来表示完全无关的接口。由于仅凭键的同一性就建立链接，期望某个提供者接口的消费者会在不做任何兼容性检查的情况下接受另一个提供者的值。与接口漂移不同——在那里提供者和消费者至少共享共同的血统 (lineage)——键冲突在预期类型与实际类型之间没有任何关系，使由此产生的失败难以预测、难以诊断。

这两个问题指向同一个缺口：余效应模型只提供名义链接（按键名），而不提供版本化或结构化链接（按接口兼容性）[71]。下面我们讨论填补这一缺口的三种方法，从与基础设施耦合最紧到与语言无关程度最高。

键命名空间化 (key namespacing)。把键空间从 K 扩展到 $K \times P$ ，其中 P 标识定义接口的包，可以从构造上消除键冲突：具有相同局部名称的独立开发接口占据不同的键。这是最直接的解决方案，但耦合也最紧：它把包命名空间嵌入形式模型本身，使系统在键同一性上依赖于外部的包注册表。

对等依赖 (peer dependency)。更轻的耦合是通过宿主语言的包管理器声明版本约束 [72]。这也是 Cordis 目前采用的方法。组件依赖在语义上是对等依赖：组件不在内部捆绑其依赖，而是期望运行时上下文提供它们。支持对等依赖的包管理器（例如 npm）可以强制版本兼容性：如果提供某个键的包版本落在消费者声明的对等范围之外，不兼容会在安装时被发现，而不是表现为运行时失败。然而，这种方法有两个局限：(1) 它依赖提供者忠实遵循语义化版本 (semantic versioning)，而这是一种无法强制执行的约定；(2) 包管理器通常把每个依赖解析为单一版本，这阻止了在同一应用中加载来自同一包多个版本的组件。

结构化兼容性 (structural compatibility)。一种完全与语言无关的方法是用兼容性谓词替换成员检查 $k \in \text{dom}(\sigma)$ ，该谓词验证提供者的实际接口在结构上包含 (subsume) 消费者的预期。这类似于结构化子类型 [73]：如果提供的接口是所需接口的子类型，提供者就满足消费者。挑战在于以语言无关的方式定义这个谓词：结构化兼容性对记录类型（宽度子类型，width subtyping）是直截了当的，但对行为契约（例如前置/后置条件 [74]、效应规范 [22]）变得复杂，而一旦参数化多态引入有界量化 [75]，它又变得不可判定。

这三种方法分别针对问题的不同方面。设计一个统一依赖模型，把这些方法结合起来，同时保持余效应模型的动态组合保证，仍然是一个开放问题。

6.7 与语言和操作系统的协同设计

第 6.4 节确定了宿主语言必须为上下文范式提供的最低要求。本节讨论相反的问题：与该范式协同设计的语言或操作系统，能在这些最低要求之外提供什么。

与语言的协同设计 (co-design with languages)。围绕上下文范式设计的语言可以在两个方面优于库实现：它赋予上下文的语义，以及它赋予效应和余效应的原语。

这样的语言可以在保持第 3.3 节上下文语义的同时，让上下文重新变为隐式的。命令式语言本来就针对隐式上下文执行每条语句，而那个单一的上下文既不追踪效应，也不解析余

效应。上下文范式则区分多个上下文：操作要么修改它所运行于其中的上下文，要么从中派生出另一个上下文（定义 27）。就地（in-place）实现修改环境上下文（ambient context），正如命令式语言所做的那样；派生（derived）实现则引入一个单独的上下文，语言必须为它提供某种构造。让上下文隐式化带来人机工效和安全两方面的好处。（1）在库实现中，每个涉及效应或余效应的函数都把上下文作为普通参数或接收者（receiver），如第 5.1 节所示；当语言隐式提供上下文时，函数不再需要携带它。（2）每个上下文都携带自己的生命周期状态与已提交视图（第 4.1 节）。库实现把上下文当作普通变量传递，因此组件可能通过闭包或全局变量错误地触及另一组件的上下文。它在那里安装的效应会泄漏出自身生命周期，它在那里读取的余效应会逸出其依赖规范。让上下文隐式化可以同时堵住这两个漏洞。

这样的语言还可以让效应和余效应为编译器所知。（1）对于效应，效应迭代器（定义 51）在每一步都分配一个闭包，以保存逆元及其所恢复的状态。有了执行效应的语法，编译器可以为整个迭代生成单个状态机，并把那些逆元保存在其帧（frame）中。（2）对于余效应，余效应规范可以被纳入类型系统，带来两个好处。首先，依赖环在编译期被报告，而不是留给运行时（第 6.5 节）。其次，依赖可以按类型的结构而非仅按键的同一性进行比较，正如行类型（row types）所做的那样 [28]，这是对第 6.6 节结构化兼容性的类型层面支持。

与操作系统的协同设计（co-design with operating systems）。第 1.2.3 节观察到一个动态组合的粗粒度替代品：操作系统以进程为粒度提供时间可组合性，其上的容器编排器以服务为粒度提供空间可组合性。与该范式协同设计的操作系统将支持细粒度组合：让组件声明的余效应规范构成它所能触及的全部，并把操作系统自身的资源作为余效应提供。

这样的操作系统可以提供第 6.3 节推迟给语言外部机制的沙箱。它通过把组件限定于它所声明的依赖来实现这一点：在组件加载时提供这些依赖，并使其内部无法触及任何其他内容，正如 WebAssembly 模块在实例化时从嵌入环境接收其导入 [76]。它（操作系统）也可以把第 3.2.3 节的余效应隔离与拦截作为自身的能力提供：为每个组件以不同的方式绑定键，并对其所提供的访问进行中介。这样的操作系统还可以把自身的资源作为余效应来提供。位于边界之外的资源会被做成可逆的：运行时记录每次获取及其对应的发起组件（第 6.1 节），而每个运行时也各自保留一份自己的记录。把资源作为余效应来提供的操作系统只需保留一份记录，因为它是分发资源的一方，可以把资源归属到提出请求的组件。内存与文件描述符是最直接的候选，而且为了恢复而对它们进行追踪的工作已经在内核接口层面实现 [77, 78]。此外，操作系统还可以把第 6.1 节只能扣留或补偿的某些操作变为可逆的。以事务方式对持久化存储执行写入的系统可以把写入回滚 [79]，而构建在写时复制（copy-on-write）或不可变存储之上的系统则通过移动指针即可回到较早的状态 [80, 81]。

7 相关工作

动态组合与多个已确立的研究领域存在交集。我们综述其中最具相关性的工作脉络，并逐一说明我们的贡献与它们的区别。

7.1 效应与余效应系统

第 2 节把效应与余效应作为支撑我们工作的理论支柱进行了回顾。我们先介绍如今在工业实践中常见的单子效应系统，然后综述三条与 Cordis 相关的、对效应与余效应加以扩展的研究脉络：把代数效应重塑为能力、赋予效应可逆语义、以及在统一的分级规范之下把效应与余效应统一起来。

7.1.1 单子效应系统 (Monadic effect systems)

有一类库把效应编码进现有通用语言的类型系统中，将它们表示为由运行时执行的一元值。Scala 中的 ZIO [82] 把计算建模为 $ZIO[R, E, A]$ ，TypeScript 中的 Effect-TS [83] 则建模为 $Effect<A, E, R>$ ——一个其参数分别描述计算结果、类型化错误以及上下文必须提供的服务的泛型类型；fp-ts 库 [84] 通过基于 Reader 的单子变换器编码了同样的错误通道与需求通道。有两个特征把这类系统与 Cordis 区分开来。其一，追踪是以一元嵌入为代价换来的：程序只有写在效应类型内部才能获得追踪，而 Cordis 则把效应追踪作为普通宿主代码之上的一层覆盖。其二，需求通过解释来满足，即由一个已安装的服务提供其操作；当该服务被撤销时，其操作所产生的结果仍然留在原处；而 Cordis 则把每个效应与一个逆配对，并在提供者来来去去的过程中重新解析需求（第 3.1 节、第 3.2 节）。

7.1.2 作为能力的代数效应 (Algebraic effects as capabilities)

代数效应（第 2.1 节）使效应操作对类型系统可见。与我们的工作最接近的扩展是 Brachthäuser 等人提出的 Effekt 语言，它把效应类型重新解释为能力 [85, 86]：效应类型表达的是计算从其上下文需要什么，而非它可能产生什么副作用。与我们的视角一样，这一视角把上下文视为能力的中介者。Cordis 与 Effekt 的区别体现在两个方面。(1) 在目的上，代数效应使效应可见是为了支持模块化解释，让同一操作拥有多种处理器语义；而 Cordis 使效应可见是为了支持追踪与逆转，把每个上下文变换与一个逆配对。(2) 在设定上，Effekt 在类型层面静态地规范效应，默认采用基于作用域的推理——能力是二等的，被限制在其词法作用域内，并通过装箱恢复一等用法，即在类型中追踪被捕获的能力来解除这一限制；而 Cordis 则在运行时规范效应，以在组件移除时实现完全的资源恢复为目标；第 6.7 节讨论了在这种意义上把上下文做成二等的语言能够提供什么。

7.1.3 可逆效应语义 (Reversible effect semantics)

另一条平行的工作脉络赋予效应可逆语义而非解释性语义。Heunen 等人 [87] 把 Hughes 的箭头改造成 dagger 箭头与逆箭头，从而在可逆设定下对副作用建模，刻画了序列化与可变存储等其操作允许逆的效应。这是与我们的可逆效应最接近的形式化工作：两者都把每个效应与撤销它的手段配对，而不是通过处理器来满足它。两者的区别在于可逆性寄居于何处，以及它们对可逆性要求多少。Heunen 等人工作在指称的、范畴论的设定中，其中可逆性是全局性质，由构造保证——因为每个计算都是可逆的，且逆是双侧的，可从范畴结构中恢复出来。Cordis 在运行时追踪逆，且对逆的要求更低：不要求整个计算可逆，只要求每个

原子效应承认一个单侧逆——该逆由调用者在应用点提供而非推导而来，任何复合效应的逆都可由复合运算从这些单侧逆得出（第 3.1 节）。

7.1.4 作为效应与余效应统一体的分级类型 (Graded types as unified effects and co-effects)

Orchard 等人 [88] 提出把分级模态类型作为统摄效应推理（经由分级单子）与余效应推理（经由分级余单子）的总括概念，并在 Granule 语言中实现，表明单一类型系统可以同时追踪计算做了什么以及计算需要什么；更近的工作把余效应扩展到命令式、类 Java 的语言 [89, 90] 以及 call-by-push-value [91]。所有这些工作都在类型层面进行：效应与余效应是编译期检查的静态注解，其作用域在词法上固定。我们的贡献与这一分析路线正交：我们把这两个概念提升为运行时机制，从而使 Cordis 能够处理动态组合。时间回缩与空间依赖会随着已加载组件集合的演化而被重新解析，而不是在固定的程序文本上一次性地确定下来。

7.2 编程范式

第 3.3.3 节确立了上下文范式——一种通过显式上下文来调节效应与余效应的规范。有两个已确立的范式值得明确比较：一个与我们的术语相同，另一个则与我们处理横切关注点的方式相同。

7.2.1 面向上下文的编程 (Context-oriented programming)

COP [92, 93] 为语言配备层 (layers) ——根据执行上下文在运行时被激活与停用的部分方法定义和类定义，从而使行为无需基础代码指名其上下文依赖即可自适应 [94]。COP 与 Cordis 在把上下文视为一等公民、运行时可变实体以及动态激活与停用行为这两点上是一致的，但这种相似只是名义上的。在 COP 中，“上下文”指环绕的执行情境（例如位置、用户、模式），激活在动态作用域的范围内改变方法分派；层既不追踪它引发的副作用，也不逆转它们，且激活不受依赖满足情况的支配。在 Cordis 中，上下文是调节效应与余效应的 Γ 实体：激活运行组件的可逆效应，并由反应式余效应的满足来驱动（第 3.2 节），停用则完整地逆转它们。COP 改变的是运行什么行为；Cordis 组合并逆转的是组件安装了什么效应与依赖。它们的区别在于权衡取舍。COP 把激活折叠进宿主语言的方法分派中，以语言特异性为代价获得了动态作用域的范围；而 Cordis 作为与语言无关的覆盖层，在共享上下文上以反应式的方式解析激活。因此，Cordis 只能把 COP 中全局的、由值驱动的部分表达为余效应：即实现之间的上下文相关选择，而非动态作用域的激活。

7.2.2 面向切面的编程 (Aspect-oriented programming)

AOP [95, 96] 把横切关注点模块化为一个切面 (aspect)：一个在基础程序中所选连接点上进行量化的切入点 (pointcut)，以及在每个连接点织入的通知 (advice)。Cordis 处理的是同样的、否则会散落在各组件中的上下文行为问题，但它与切面相对应的是余效应：一

个许多组件声明依赖的共享中介点，从而可以在该处重塑横切行为而无需编辑任何组件。两种范式随后在两个轴向上有所区别。(1) 声明式与无感知之别：AOP 的切入点是无感知的、量化的，匹配任意连接点，而这些连接点的代码并不知道自己被织入了通知；Cordis 则把横切行为限定在各组件声明的余效应之内，因此其作用范围恰好就是所声明的表面。这带来了确定性与可追溯性：应用编排器可以在配置层检查并管控组件被横切的内容，而无需阅读或分析其源码；而 AOP 的关注点只有通过在其上量化的切面才能被解读。(2) 生命周期集成：Cordis 中的横切变更由组件的效应承载，在组件卸载时被逆转，并以反应式方式传播给其依赖者，因此它是动态组合模型内部的一步操作；动态 AOP 系统 [97, 98] 也可以在运行时织入与解织 (unweave)，但那是独立操作，既不绑定组件的生命周期，也不在被织入的代码之间触发重新解析。

7.3 时间可组合性

时间可组合性关注的是在运行中的程序里替换或移除组件，同时恢复它所安装的效应。已有方法按它们如何处理离场组件的状态与效应来划分：把状态向前携带到后继版本、通过开发者编写的清理逻辑恢复效应、在预先固定的作用域内自动逆转效应、或者依据运行时在某个接口上拦截而累积的记录回收资源。

7.3.1 有状态的前向迁移 (Stateful forward migration)

一大类系统通过在版本间向前携带状态的方式，在不停机的情况下替换运行中程序里的组件。它们都遵循同样的时机纪律：组件只有到达一个安全的、无交互的点之后才能被换出。Kramer 与 Magee 把这一准则确立为静默态 [51]，Vandewoude 等人后来将其放宽为破坏性更小的宁静态 (tranquility) [52]；我们的滚动更新模式 (第 6.2 节) 通过在卸载提供者之前排空进行中的请求来强制这一准则。动态软件更新 (DSU) 随后通过手写的转换函数向前迁移状态：Hicks 等人的通用 C 语言 DSU [99]、Stoye 等人通过 con-freeness 分析实现的类型安全更新点 [100]、以及 Hayden 等人的 Kitsune [101]，都把旧版本的数据映射为新版本的表示，原地继承堆对象、已打开的文件与连接，同时重新初始化一切未被迁移的部分。同样的纪律也延伸到持久状态：Overeem 等人 [102] 在保持系统可用性的同时，通过手写的升级操作转换运行中的事件存储 (event store) 在不同模式 (schema) 版本之间的数据。Erlang/OTP [15] 在进程层面采取同样的立场，通过 `code_change/3` 迁移状态，并通过重启被监管的进程而非逆转其效应来从故障中恢复；JavaScript 的热模块替换 (例如 webpack [46]、Vite [47]) 在模块层面做同样的事情，在重载时通过 `module.hot` 或 `import.meta.hot` API 把状态向前传递。与 Cordis 的模块替换 (第 5.2 节) 相比，这些方法能更优雅地迁移内存状态：Cordis 逆转旧组件已追踪的效应，并从一张白纸开始重新应用新组件的效应，因此组件自身的内存状态在重载后不会存续，除非它被放入一个寿命更长的依赖中；在可逆效应之上叠加 DSU 式的前向迁移是未来工作。尽管如此，Cordis 的方法在两个意义上更通用：它不需要 DSU 和 HMR 所需要的那类手写迁移函数，而且它支持完整卸载一个组件并回收其资源，而不只是在原地更新一个组件。

7.3.2 开发者编写的恢复 (Developer-authored recovery)

第二类方法通过开发者手写的清理或补偿逻辑来恢复组件的效应。插件生命周期约定 (例如 OSGi [50]、Eclipse 的扩展点、IntelliJ 与 VSCode) 把清理委托给开发者编写的卸载回调; 命令模式 [103] 把操作与撤销 (undo) 方法封装在一起, 用于撤销/重做栈; saga 模型 [49] 把长生命周期事务组织为一系列步骤, 每一步与一个补偿动作配对; 代数效应处理器可以在拆卸 (teardown) 时附加并运行终结器 (finalizers) [104]; 事件溯源 (event sourcing) [105] 通过追加补偿事件来回缩状态, 而不是执行逆。在所有这些方法中, 逆都是不受强制约束的责任, 与操作解耦, 因此一旦遗漏就会静默地泄漏资源 (第 1.2.1 节对此有经验性记录)。React 的 `useEffect` 钩子 [106] 最接近在结构上把效应与其逆配对的做法: 它返回一个清理函数, 运行时在每次重新执行之前以及卸载 (unmount) 时调用它。它的不足之处在于可组合性: 钩子只能在组件或另一个钩子的顶层调用, 绝不能出现在条件、循环或嵌套函数内部, 而且其效应主体既不能接受异步函数, 也不能接受迭代器。因此, 效应无法由其他效应组装, 也无法与控制流交错, 也就没有任何东西可供推导复合逆。Cordis 的效应没有这样的限制: 它们是可自由复合、可异步运行的普通操作, 只需为每个原子效应手写一个逆, 任何复合效应的逆都可由复合运算推导出来, 因此组装已有效应完全不需要编写逆。这种把每个效应与其逆进行结构性配对的做法, 使完全恢复成为系统的不变量, 而非开发者纪律的问题。

7.3.3 静态作用域的逆转 (Statically scoped reversal)

第三类方法通过构造自动逆转效应, 但把逆转限制在预先固定的作用域内。软件事务内存 [107, 108] (源自硬件事务内存 [109]) 记录读/写日志, 使一组内存操作要么提交、要么中止, 把内存回滚到事务之前的状态。可逆计算——从 Landauer 与 Bennett 的热力学分析 [110, 111] 到 Janus 等可逆语言 [112]——走得更远, 使整个计算的每一步都全局可逆。可逆进程演算把回溯构建进语义本身: RCCS [113] 为每个进程附带一块记忆, 并允许在其导向的过去因果等价时撤回一步; Phillips 与 Ulidowski [114] 在保持 CCS、ACP 与 CSP 前向操作语义的同时, 统一地推导出它们的可逆算子。它们的因果一致性准则与 Cordis 恢复所遵循的顺序是并发的对应物: 累加器以后进先出的顺序应用组件自身的逆, 而第 4.3.1 节的守卫 (guard) 把提供者的撤销推迟到其消费者全部停用之后 (定理 63)。然而, 其作用范围由语义固定, 所执行的每个动作都保持可撤销; 而 Cordis 组件为每个原子效应提供一个逆, 其累加器把上下文带回到组合开始之处。线性类型 [115]、RAII [4] 与 Rust 的所有权系统 [61] 把资源的释放绑定到词法区域。它们各自静态地固定逆转的作用域与范围; 相比之下, Cordis 不预先固定这样的作用域: 它在组件的生命周期内逆转任意的上下文操作, 并把词法资源管理视为互补的手段, 适用于单个组件内部的局部资源。

7.3.4 拦截式回收 (Interposed reclamation)

第四类方法在组件自身不提供逆的情况下回收组件所获取的资源, 做法是在运行时控制的接口处记录组件的获取行为。Nooks [77] 包装跨越 Linux 内核与其可加载扩展之间边界

的每一次调用，使扩展所触及的内核对象都经过一个对象追踪器，其记录告诉恢复管理器在扩展失败时应释放什么；影子驱动 (shadow drivers) [78] 从另一侧截取同样的调用，记录决定驱动状态的请求与配置，从而使重启后的实例可以被恢复到该状态。Akeso [116] 则改由编译器插桩 (instrumentation) 获得记录，把内核执行划分为可嵌套的恢复域，记录其状态变化与跨线程依赖，并把发生故障的请求连同依赖它的每个域一起回滚。因此，回收依据的是运行时维护的记录，而非开发者记得去写的清理代码，这使得这一类方法成为可逆效应在系统层面最接近的先例。它在词汇与作用范围两方面与 Cordis 不同。平台固定了可记录的内容——无论是按内核对象类型给出释放代码、按驱动类别给出一个影子，还是按被插桩的分配器给出一个逆——因此组件只能持有平台已经知道如何释放的资源；而 Cordis 组件则引入自己的效应，并为每个原子效应提供一个逆 (第 3.1 节)。同样地，回收被一次提交的请求或同一扩展的重启所限定；而 Cordis 在组件的整个生命周期内进行逆转，并把移除传播给其依赖者，依赖者继而释放各自的效应 (第 3.2 节)。

7.4 空间可组合性

空间可组合性关注的是组件对其他组件的依赖如何被声明与绑定。已有机制按绑定如何响应变化来划分：在初始化时一次性接线依赖、响应整个组件的可用性、或者在单个值的粒度上传播变化。

7.4.1 初始化时的依赖接线 (Initialization-time dependency wiring)

有两个已确立的机制在初始化时把组件接在一起。依赖注入框架 [38] (例如 Spring [117]、Guice、Angular、Inversify) 在初始化时把依赖注入组件，而 UI 框架的上下文 (例如 Vue.js 的 `provide/inject` 与 React 的 Context API) 则沿组件树传递它们。有些支持动态作用域 (例如 Spring 的 `prototype/request` 作用域、Angular 的分层注入器)，但两者都不会反应式地重新解析：当提供者在运行时被替换或移除时，已有的依赖者既不会被停用，也不会被重新初始化，而且它们都不提供我们的组件状态机所提供的那类生命周期管理。Cordis 的反应式余效应 (第 3.2 节) 提供了这一点：每当满足谓词发生变化时，通知机制就会触发生命周期迁移。

7.4.2 可用性反应式组件模型 (Availability-reactive component models)

与我们的反应式余效应最接近的先例响应服务的可用性。OSGi 的 Declarative Services 与 iPOJO [118, 119] 允许组件声明提供的与所需的服务，运行时在服务出现与消失时自动激活与停用它们；iPOJO 的 Gravity 项目 [119] 明确以自主的运行时适应于变化的服务可用性为目标，其 `provide/require` 模型直接预示了 Cordis 的 `ctx.provide/ctx.get` 模式。R-OSGi [53] 通过 RPC 把同样的抽象透明地扩展到分布式环境，把网络故障映射为服务撤销事件——第 6.2 节把这一模式讨论为 Cordis 模型的一种扩展。所有这些系统都通过停用回调来恢复，而这种回调在两个方面受限。其一，回调是手写的，因此资源安全依赖于开发者的纪律，遗漏的回调会静默地泄漏资源。其二，回调是同步的：如果拆卸需要与离场的依

赖进行异步交互，这些框架不提供等待它的协议，只能对可能已经过期的引用进行阻塞式等待。Cordis 的反应式余效应弥补了这两个缺口：停用逆转依赖者累积的效应，而其惯性的 Unloading 状态（第 4.3.3 节）在响应进一步的变化之前先把异步拆卸运行完毕。

7.4.3 值级反应性 (Value-level reactivity)

函数式反应式编程 (FRP) [120] 及其现代形态（例如 SolidJS 中的信号 (signals) [121, 122]、Vue 的反应式系统、Angular Signals）在值级的粒度假传播变化：当信号改变时，派生计算被同步地或在调度器 (scheduler) 之下重新求值 [123]。Cordis 的反应式余效应在组件级的粒度假起作用，增加了值级传播无法建模的异步生命周期语义。同样的粒度差异在一致性方面则方向相反：在一轮 (turn) 内、按依赖图固定的顺序传播，使 FRP 可以要求任何派生计算都不得读到更新与过期输入的混合体，即无毛刺 (glitch freedom) [124]；而 Cordis 没有与一轮对应的概念，编排动作一次到达一个，并且只保证没有任何单个迁移跨越其余效应的两次解析（定理 64）。两者是互补而非竞争的：Cordis 的余效应本身可以承载反应式值，组件只在其实际消费的部分上更新，从而把组件级反应性细化成跨越两个层级的更细粒度反应式余效应。

8 结论

我们把经典的效应与余效应概念提升为运行时机制，从而为动态组合给出了一个形式化基础。可逆效应解决局部的时间可组合性：每个上下文变换都携带一个由运行时追踪的逆，且追踪与恢复都保持组合性，因此上下文会在组件移除时被恢复。反应式余效应解决局部的空间可组合性：每当上下文变化时，组件都会依据其余效应规范收到通知，每个变化被分类为激活、停用或中性；余效应隔离改变已声明键解析为何物，余效应拦截改变绑定被使用的方式。我们把效应上下文与余效应上下文统一为单一的上下文类型，其中余效应上的观测等价效应提供了独立性，从而构成了时空可组合性的编程范式。把这些机制组合进组件的概念之中，便得到了动态组合的演算，其元理论把时空可组合性从单个组件推广到由交错组件构成的整个系统。我们以 Cordis 元框架实现了这一范式：核心库提供效应追踪与余效应解析，并提供一个带配置调和与热模块替换的声明式组件加载器。Koishi 案例研究在一个拥有 4000 多个社区插件的生产系统中验证了 Cordis 的设计。

在人工策展的插件生态系统之外，未来验证的一个有说服力的方向是自演化智能体框架（第 1.2.2 节）：AI 智能体在很少人工监督的情况下持续生成并替换自己的框架组件。在这样的环境中应用 Cordis，将验证快速组件替换下完全恢复的时间性保证，以及频繁拓扑变化下依赖协调的空间性保证。这样的验证将展示该范式作为基础的适用性，为智能体框架及其他自治系统中的可恢复、可协调且持续的自演化提供支撑。

9 参考文献

- [1] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972, doi: 10.1145/361598.361623.
- [2] D. Birsan, “On Plug-ins and Extensible Architectures,” *ACM Queue*, vol. 3, no. 2, pp. 40–46, 2005, doi: 10.1145/1053331.1053345.
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kuber- netes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.
- [4] B. Stroustrup, *The Design and Evolution of C++*. Addison-Wesley, 1994.
- [5] S. Marlow, S. Peyton Jones, A. Moran, and J. Reppy, “Asynchronous Exceptions in Haskell,” in *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*, in *PLDI '01*. New York, NY, USA: Association for Computing Machinery, 2001, pp. 274–285. doi: 10.1145/378795.378858.
- [6] L. Cardelli, “Program Fragments, Linking, and Modularization,” in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 1997)*, ACM Press, 1997, pp. 266–277. doi: 10.1145/263699.263735.
- [7] C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Addison- Wesley, 2002.
- [8] R. Lopopolo, “Harness Engineering: Leveraging Codex in an Agent-First World.” [Online]. Available: <https://openai.com/index/harness-engineering/>
- [9] Anthropic, “Harness Design for Long-Running Application Development.” [Online]. Available: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- [10] L. Wang et al., “A Survey on Large Language Model Based Autonomous Agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024, doi: 10.1007/s11704-024-40231-1.
- [11] Y. Qin et al., “Tool Learning with Foundation Models,” *ACM Computing Surveys*, 2025, doi: 10.1145/3704435.
- [12] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” *CoRR*, vol. abs/2310.08560, 2023.
- [13] T. Guo et al., “Large Language Model Based Multi-Agents: A Survey of Progress and Challenges,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, in *IJCAI 2024*. 2024, pp. 8048–8057. doi: 10.24963/ijcai.2024/890.

- [14] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large Language Models as Tool Makers,” in Proceedings of the Twelfth International Conference on Learning Representations, in ICLR 2024. 2024. [Online]. Available: <https://openreview.net/forum?id=qV83K9d5WB>
- [15] J. Armstrong, “Making Reliable Distributed Systems in the Presence of Software Errors,” Doctoral dissertation, 2003. [Online]. Available: https://erlang.org/download/armstrong_thesis_2003.pdf
- [16] E. Moggi, “Notions of computation and monads,” *Information and Computation*, vol. 93, no. 1, pp. 55–92, 1991, doi: 10.1016/0890-5401(91)90052-4. 80
- [17] G. Plotkin and J. Power, “Adequacy for Algebraic Effects,” in *Foundations of Software Science and Computation Structures*, F. Honsell and M. Miculan, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–24.
- [18] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: unified static analysis of context-dependence,” in *Proceedings of the 40th International Conference on Automata, Languages, and Programming - Volume Part II*, in ICALP’13. Riga, Latvia: Springer-Verlag, 2013, pp. 385–397. doi: 10.1007/978-3-642-39212-2_35.
- [19] M. Gaboardi, S.-ya Katsumata, D. Orchard, F. Breuvar, and T. Uustalu, “Combining effects and coeffects via grading,” in *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, in ICFP 2016. Nara, Japan: Association for Computing Machinery, 2016, pp. 476–489. doi: 10.1145/2951913.2951939.
- [20] A. Church, “A Formulation of the Simple Theory of Types,” *The Journal of Symbolic Logic*, vol. 5, no. 2, pp. 56–68, 1940, doi: 10.2307/2266170.
- [21] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
- [22] J. M. Lucassen and D. K. Gifford, “Polymorphic Effect Systems,” in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’88. San Diego, California, USA: Association for Computing Machinery, 1988, pp. 47–57. doi: 10.1145/73560.73564.
- [23] P. Wadler, “Monads for functional programming,” in *Program Design Calculi*, M. Broy, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 233–264.
- [24] G. Plotkin and J. Power, “Notions of Computation Determine Monads,” in *Foundations of Software Science and Computation Structures*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 342–356. doi: 10.1007/3-540-45931-6_24.
- [25] G. Plotkin and M. Pretnar, “Handlers of Algebraic Effects,” in *Programming Languages and Systems (ESOP)*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 80–94. doi: 10.1007/978-3-642-00590-9_7.

- [26] M. Pretnar, “An Introduction to Algebraic Effects and Handlers. Invited tutorial paper,” *Electron. Notes Theor. Comput. Sci.*, vol. 319, no. C, pp. 19–35, Dec. 2015, doi: 10.1016/j.entcs.2015.12.003.
- [27] D. Leijen, “Koka: Programming with Row Polymorphic Effect Types,” *Electronic Proceedings in Theoretical Computer Science*, vol. 153, pp. 100–126, Jun. 2014, doi: 10.4204/eptcs.153.8.
- [28] D. Leijen, “Type directed compilation of row-typed algebraic effects,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, in POPL ’17. Paris, France: Association for Computing Machinery, 2017, pp. 486–499. doi: 10.1145/3009837.3009872.
- [29] A. Bauer and M. Pretnar, “Programming with algebraic effects and handlers,” *Journal of Logical and Algebraic Methods in Programming*, vol. 84, no. 1, pp. 108–123, Jan. 2015, doi: 10.1016/j.jlamp.2014.02.001.
- [30] K. Sivaramakrishnan et al., “Retrofitting parallelism onto OCaml,” *Proc. ACM Program. Lang.*, vol. 4, no. ICFP, Aug. 2020, doi: 10.1145/3408995. 81
- [31] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: a calculus of context-dependent computation,” in *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, in ICFP ’14. Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 123–135. doi: 10.1145/2628136.2628160.
- [32] T. Uustalu and V. Vene, “Comonadic Notions of Computation,” *Electronic Notes in Theoretical Computer Science*, vol. 203, no. 5, pp. 263–284, 2008, doi: 10.1016/j.entcs.2008.05.029.
- [33] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, “A Core Quantitative Coeffect Calculus,” in *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*, Berlin, Heidelberg: Springer-Verlag, 2014, pp. 351–370. doi: 10.1007/978-3-642-54833-8_19.
- [34] J. Reed and B. C. Pierce, “Distance makes the types grow stronger: a calculus for differential privacy,” *SIGPLAN Not.*, vol. 45, no. 9, pp. 157–168, Sep. 2010, doi: 10.1145/1932681.1863568.
- [35] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’99. San Antonio, Texas, USA: Association for Computing Machinery, 1999, pp. 147–160. doi: 10.1145/292540.292555.
- [36] D. E. Denning, “A lattice model of secure information flow,” *Commun. ACM*, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.

- [37] U. Dal Lago and F. Gavazzo, “A relational theory of effects and coeffects,” *Proc. ACM Program. Lang.*, vol. 6, no. POPL, Jan. 2022, doi: 10.1145/3498692.
- [38] M. Fowler, “Inversion of Control Containers and the Dependency Injection pattern.” [Online]. Available: <https://martinfowler.com/articles/injection.html>
- [39] A. M. Pitts and I. D. B. Stark, “Observable Properties of Higher Order Functions that Dynamically Create Local Names, or What’s New?,” in *Mathematical Foundations of Computer Science 1993 (MFCS 1993)*, in *Lecture Notes in Computer Science*, vol. 711. Springer, 1993, pp. 122–141. doi: 10.1007/3-540-57182-5_8.
- [40] G. D. Plotkin, “LCF Considered as a Programming Language,” *Theoretical Computer Science*, vol. 5, no. 3, pp. 223–255, 1977, doi: 10.1016/0304-3975(77)90044-5.
- [41] D. R. Ghica, K. Muroya, and T. Waugh Ambridge, “A Robust Graph-Based Approach to Observational Equivalence,” *Logical Methods in Computer Science*, vol. 21, no. 2, p. 8:1–8:95, 2025, doi: 10.46298/LMCS-21(2:8)2025.
- [42] X. Leroy and S. Blazy, “Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations,” *Journal of Automated Reasoning*, vol. 41, no. 1, pp. 1–31, 2008, doi: 10.1007/s10817-008-9099-0.
- [43] R. P. James and A. Sabry, “Yield: Mainstream Delimited Continuations,” in *First International Workshop on the Theory and Practice of Delimited Continuations (TPDC 2011)*, 2011, pp. 20–32. [Online]. Available: <https://homes.luddy.indiana.edu/sabry/files/yield.pdf>
- [44] A. W. Mazurkiewicz, “Trace Theory,” in *Petri Nets: Central Models and Their Properties, Advances in Petri Nets 1986, Part II*, in *Lecture Notes in Computer Science*, vol. 255. Springer, 1986, pp. 279–324. doi: 10.1007/3-540-17906-2_30. 82
- [45] U. A. Acar, G. E. Blelloch, and R. Harper, “Adaptive functional programming,” *ACM Transactions on Programming Languages and Systems*, vol. 28, no. 6, pp. 990–1034, 2006, doi: 10.1145/1186632.1186634.
- [46] webpack, “Hot Module Replacement.” [Online]. Available: <https://webpack.js.org/api/hot-module-replacement/>
- [47] Vite, “HMR API.” [Online]. Available: <https://vite.dev/guide/api-hmr>
- [48] E. N. (M. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A Survey of Rollback-Recovery Protocols in Message-Passing Systems,” *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002, doi: 10.1145/568522.568525.
- [49] H. Garcia-Molina and K. Salem, “Sagas,” in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, in *SIGMOD ’87*. 1987, pp. 249–259. doi: 10.1145/387113.38742.

- [50] OSGi Alliance, OSGi Core Release 8. OSGi Alliance, 2020. [Online]. Available: <https://docs.osgi.org/specification/osgi.core/8.0.0/>
- [51] J. Kramer and J. Magee, “The Evolving Philosophers Problem: Dynamic Change Management,” *IEEE Transactions on Software Engineering*, vol. 16, no. 11, pp. 1293–1306, 1990, doi: 10.1109/32.60317.
- [52] Y. Vandewoude, P. Ebraert, Y. Berbers, and T. D’Hondt, “Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates,” *IEEE Transactions on Software Engineering*, vol. 33, no. 12, pp. 856–868, 2007, doi: 10.1109/tse.2007.70733.
- [53] J. S. Rellermeyer, G. Alonso, and T. Roscoe, “R-OSGi: Distributed Applications Through Software Modularization,” in *Proceedings of the ACM/IFIP/USENIX 8th International Middleware Conference*, in *Middleware ’07*. 2007, pp. 1–20. doi: 10.1007/978-3-540-76778-7_1.
- [54] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252.
- [55] M. S. Miller, K.-P. Yee, and J. Shapiro, “Capability Myths Demolished,” technical report SRL2003–2, 2003. [Online]. Available: <http://zesty.ca/capmyths/usenix.pdf>
- [56] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, “Capsicum: Practical Capabilities for UNIX,” in *Proceedings of the 19th USENIX Security Symposium*, 2010, pp. 29–46. [Online]. Available: https://www.usenix.org/legacy/events/sec10/tech/full_papers/Watson.pdf
- [57] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, “Efficient Software-Based Fault Isolation,” in *Proceedings of the 14th ACM Symposium on Operating Systems Principles*, in *SOSP ’93*. 1993, pp. 203–216. doi: 10.1145/168619.168635.
- [58] A. Barth, A. P. Felt, P. Saxena, and A. Boodman, “Protecting Browsers from Extension Vulnerabilities,” in *Proceedings of the 17th Annual Network and Distributed System Security Symposium*, in *NDSS ’10*. 2010. [Online]. Available: <https://www.ndss-symposium.org/ndss2010/protecting-browsers-extension-vulnerabilities/>
- [59] W. W. Ho and R. A. Olsson, “An Approach to Genuine Dynamic Linking,” *Software: Practice and Experience*, vol. 21, no. 4, pp. 375–390, 1991, doi: 10.1002/SPE.4380210404.83
- [60] P. Wadler and S. Blott, “How to Make Ad-hoc Polymorphism Less Ad Hoc,” in *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in *POPL ’89*. 1989, pp. 60–76. doi: 10.1145/75277.75283.

- [61] N. D. Matsakis and F. S. K. II, “The Rust Language and Type System,” in ACM SIGPLAN ML Family Workshop, Gothenburg, Sweden, Sep. 2014.
- [62] D. Dreyer, R. Harper, M. M. T. Chakravarty, and G. Keller, “Modular Type Classes,” in Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’07. 2007, pp. 63–70. doi: 10.1145/1190216.1190229.
- [63] Microsoft, “Declaration Merging.” [Online]. Available: <https://www.typescriptlang.org/docs/handbook/declaration-merging.html>
- [64] T. Van Cutsem and M. S. Miller, “Proxies: Design Principles for Robust Object-oriented Intercession APIs,” in Proceedings of the 6th Symposium on Dynamic Languages, in DLS ’10. 2010, pp. 59–72. doi: 10.1145/1869631.1869638.
- [65] R. Hettinger, “Descriptor HowTo Guide.” [Online]. Available: <https://docs.python.org/3/howto/descriptor.html>
- [66] P. Maes, “Concepts and Experiments in Computational Reflection,” in Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1987, pp. 147–155. doi: 10.1145/38765.38821.
- [67] G. Bracha and D. M. Ungar, “Mirrors: design principles for meta-level facilities of object-oriented programming languages,” in Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOP- SLA), 2004, pp. 331–344. doi: 10.1145/1028976.1029004.
- [68] R. Rouvoy and P. Merle, “Leveraging component-based software engineering with Fraclet,” *Annals of Telecommunications*, vol. 64, no. 1–2, pp. 65–79, 2009, doi: 10.1007/s12243-008-0072-z.
- [69] E. Burmako, “Scala Macros: Let Our Powers Combine!,” in Proceedings of the 4th Workshop on Scala, in SCALA@ECOOP ’13. 2013, p. 3:1–3:10. doi: 10.1145/2489837.2489840.
- [70] S. Raemaekers, A. van Deursen, and J. Visser, “Semantic Versioning and Impact of Breaking Changes in the Maven Repository,” *Journal of Systems and Software*, vol. 129, pp. 140–158, 2017, doi: 10.1016/j.jss.2016.04.008.
- [71] P. Lam, J. Dietrich, and D. J. Pearce, “Putting the Semantics into Semantic Versioning,” in Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, in Onward! ’20. 2020, pp. 157–179. doi: 10.1145/3426428.3426922.
- [72] P. Abate, R. Di Cosmo, R. Treinen, and S. Zacchiroli, “Dependency Solving: A Separate Concern in Component Evolution Management,” *Journal of Systems and Software*, vol. 85, no. 10, pp. 2228–2240, 2012, doi: 10.1016/j.jss.2012.02.018.

- [73] L. Cardelli, “Structural Subtyping and the Notion of Power Type,” in Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’88. 1988, pp. 70–79. doi: 10.1145/73560.73566.
- [74] B. Meyer, “Applying ”Design by Contract”,” Computer, vol. 25, no. 10, pp. 40–51, 1992, doi: 10.1109/2.161279. 84
- [75] B. C. Pierce, “Bounded Quantification is Undecidable,” Information and Computation, vol. 112, no. 1, pp. 131–165, 1994, doi: 10.1006/inco.1994.1055.
- [76] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2017, pp. 185–200. doi: 10.1145/3062341.3062363.
- [77] M. M. Swift, B. N. Bershad, and H. M. Levy, “Improving the reliability of commodity operating systems,” in Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), ACM, 2003, pp. 207–222. doi: 10.1145/945445.945466.
- [78] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, “Recovering device drivers,” ACM Transactions on Computer Systems, vol. 24, no. 4, pp. 333–360, 2006, doi: 10.1145/1189256.1189257.
- [79] D. E. Porter, O. S. Hofmann, C. J. Rossbach, A. Benn, and E. Witchel, “Operating System Transactions,” in Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP), ACM, 2009, pp. 161–176. doi: 10.1145/1629575.1629591.
- [80] O. Kiselyov and C.-chieh Shan, “Delimited Continuations in Operating Systems,” in Modeling and Using Context (CONTEXT 2007), in Lecture Notes in Computer Science, vol. 4635. Springer, 2007, pp. 291–302. doi: 10.1007/978-3-540-74255-5_22.
- [81] E. Dolstra and A. Löb, “NixOS: a purely functional Linux distribution,” in Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (ICFP), ACM, 2008, pp. 367–378. doi: 10.1145/1411204.1411255.
- [82] ZIO, “ZIO: Type-safe, composable asynchronous and concurrent programming for Scala.” [Online]. Available: <https://zio.dev/>
- [83] Effect, “Effect: A TypeScript library for building robust applications.” [Online]. Available: <https://effect.website/>
- [84] G. Canti, “fp-ts: Functional programming in TypeScript.” [Online]. Available: <https://github.com/gcanti/fp-ts>
- [85] J. I. Brachthäuser, P. Schuster, and K. Ostermann, “Effects as capabilities: effect handlers and lightweight effect polymorphism,” Proc. ACM Program. Lang., vol. 4, no. OOPSLA, 2020, doi: 10.1145/3428194.

- [86] J. I. Brachthäuser, P. Schuster, E. Lee, and A. Boruch-Gruszecki, “Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA1, 2022, doi: 10.1145/3527320.
- [87] C. Heunen, R. Kaarsgaard, and M. Karvonen, “Reversible Effects as Inverse Arrows,” in *Proceedings of the Thirty-Fourth Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIV)*, in *Electronic Notes in Theoretical Computer Science*, vol. 341. 2018, pp. 179–199. doi: 10.1016/j.entcs.2018.11.009.
- [88] D. Orchard, V.-B. Liepelt, and H. Eades III, “Quantitative program reasoning with graded modal types,” *Proc. ACM Program. Lang.*, vol. 3, no. ICFP, 2019, doi: 10.1145/3341714.
- [89] R. Bianchini, F. Dagnino, P. Giannini, E. Zucca, and M. Servetto, “Coeffects for sharing and mutation,” *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, Oct. 2022, doi: 10.1145/3563319. 85
- [90] R. Bianchini, F. Dagnino, P. Giannini, and E. Zucca, “A Java-like calculus with heterogeneous coeffects,” *Theoretical Computer Science*, vol. 971, p. 114063, 2023, doi: <https://doi.org/10.1016/j.tcs.2023.114063>.
- [91] C. Torczon, E. Suárez Acevedo, S. Agrawal, J. Velez-Ginorio, and S. Weirich, “Effects and Coeffects in Call-by-Push-Value,” *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Oct. 2024, doi: 10.1145/3689750.
- [92] R. Hirschfeld, P. Costanza, and O. Nierstrasz, “Context-oriented Programming,” *Journal of Object Technology*, vol. 7, no. 3, pp. 125–151, 2008, doi: 10.5381/jot.2008.7.3.a4.
- [93] P. Costanza and R. Hirschfeld, “Language constructs for context-oriented programming: an overview of ContextL,” in *Proceedings of the 2005 Symposium on Dynamic Languages (DLS '05)*, ACM, 2005, pp. 1–10. doi: 10.1145/1146841.1146842.
- [94] G. Salvaneschi, C. Ghezzi, and M. Pradella, “Context-oriented programming: A software engineering perspective,” *Journal of Systems and Software*, vol. 85, no. 8, pp. 1801–1817, 2012, doi: 10.1016/j.jss.2012.03.024.
- [95] G. Kiczales et al., “Aspect-Oriented Programming,” in *ECOOP'97 —Object-Oriented Programming, 11th European Conference*, in *Lecture Notes in Computer Science*, vol. 1241. Springer, 1997, pp. 220–242. doi: 10.1007/BFb0053381.
- [96] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, and W. G. Griswold, “An Overview of AspectJ,” in *ECOOP 2001 —Object-Oriented Programming, 15th European Conference*, in *Lecture Notes in Computer Science*, vol. 2072. Springer, 2001, pp. 327–353. doi: 10.1007/3-540-45337-7_18.

- [97] A. Popovici, T. Gross, and G. Alonso, “Dynamic Weaving for Aspect-Oriented Programming,” in Proceedings of the 1st International Conference on Aspect-Oriented Software Development (AOSD 2002), ACM, 2002, pp. 141–147. doi: 10.1145/508386.508404.
- [98] J. Bonér, “What Are the Key Issues for Commercial AOP Use: How Does AspectWerkz Address Them?,” in Proceedings of the 3rd International Conference on Aspect-Oriented Software Development (AOSD 2004), ACM, 2004, pp. 5–6. doi: 10.1145/976270.976273.
- [99] M. Hicks, J. T. Moore, and S. Nettles, “Dynamic Software Updating,” in Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation, in PLDI ’01. 2001, pp. 13–23. doi: 10.1145/378795.378798.
- [100] G. Stoyale, M. Hicks, G. Bierman, P. Sewell, and I. Neamtiu, “Mutatis Mutandis: Safe and Predictable Dynamic Software Updating,” in Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’05. 2005, pp. 183–194. doi: 10.1145/1040305.1040321.
- [101] C. M. Hayden, K. Saur, E. K. Smith, and M. Hicks, “Kitsune: Efficient, General-Purpose Dynamic Software Updating for C,” *ACM Trans. Program. Lang. Syst.*, vol. 36, no. 4, 2014, doi: 10.1145/2629460.
- [102] M. Overeem, M. Spoor, and S. Jansen, “The Dark Side of Event Sourcing: Managing Data Conversion,” in IEEE 24th International Conference on Software Analysis, Evolution and Reengineering, in SANER ’17. 2017, pp. 193–204. doi: 10.1109/SANER.2017.7884621.
- [103] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA: Addison-Wesley, 1994. 86
- [104] D. Leijen, “Algebraic Effect Handlers with Resources and Deep Finalization,” technical report MSR-TR-2018-10, Apr. 2018. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/algebraic-effect-handlers-resources-deep-finalization/>
- [105] M. Fowler, “Event Sourcing.” 2005.
- [106] J. Lee, J. Ahn, and K. Yi, “React-tRace: A Semantics for Understanding React Hooks,” *Proc. ACM Program. Lang.*, vol. 9, no. OOPSLA2, pp. 471–498, 2025, doi: 10.1145/3763067.
- [107] N. Shavit and D. Touitou, “Software Transactional Memory,” in Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing, in PODC ’95. 1995, pp. 204–213. doi: 10.1145/224964.224987.
- [108] T. Harris, S. Marlow, S. Peyton Jones, and M. Herlihy, “Composable Memory Transactions,” in Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, in PPOPP ’05. 2005, pp. 48–60. doi: 10.1145/1065944.1065952.

- [109] M. Herlihy and J. E. B. Moss, “Transactional Memory: Architectural Support for Lock-Free Data Structures,” in Proceedings of the 20th Annual International Symposium on Computer Architecture, in ISCA '93. 1993, pp. 289–300. doi: 10.1145/165123.165164.
- [110] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” IBM Journal of Research and Development, vol. 5, no. 3, pp. 183–191, 1961, doi: 10.1147/rd.53.0183.
- [111] C. H. Bennett, “Logical Reversibility of Computation,” IBM Journal of Research and Development, vol. 17, no. 6, pp. 525–532, 1973, doi: 10.1147/rd.176.0525.
- [112] T. Yokoyama and R. Glück, “A Reversible Programming Language and its Invertible Self-Interpreter,” in Proceedings of the 2007 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-Based Program Manipulation, in PEPM '07. 2007, pp. 144–153. doi: 10.1145/1244381.1244404.
- [113] V. Danos and J. Krivine, “Reversible Communicating Systems,” in CONCUR 2004 —Concurrency Theory, 15th International Conference, in Lecture Notes in Computer Science, vol. 3170. Springer, 2004, pp. 292–307. doi: 10.1007/978-3-540-28644-8_19.
- [114] I. Phillips and I. Ulidowski, “Reversing Algebraic Process Calculi,” in Foundations of Software Science and Computation Structures, 9th International Conference (FOSSACS 2006), in Lecture Notes in Computer Science, vol. 3921. Springer, 2006, pp. 246–260. doi: 10.1007/11690634_17.
- [115] P. Wadler, “Linear Types Can Change the World!,” in Programming Concepts and Methods: Proceedings of the IFIP Working Group 2.2/2.3 Working Conference, North-Holland, 1990, pp. 561–581. [Online]. Available: <https://homepages.inf.ed.ac.uk/wadler/papers/linear/linear.ps>
- [116] A. Lenharth, V. S. Adve, and S. T. King, “Recovery domains: an organizing principle for recoverable operating systems,” in Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 49–60. doi: 10.1145/1508244.1508251.
- [117] C. Walls, Spring in Action, 6th ed. Manning Publications, 2022. [Online]. Available: <https://www.manning.com/books/spring-in-action-sixth-edition> 87
- [118] C. Escoffier, R. S. Hall, and P. Lalanda, “iPOJO: an Extensible Service-Oriented Component Framework,” in IEEE International Conference on Services Computing, 2007, pp. 474–481. doi: 10.1109/SCC.2007.74.
- [119] H. Cervantes and R. S. Hall, “Autonomous Adaptation to Dynamic Availability Using a Service-Oriented Component Model,” in Proceedings of the 26th International Conference on Software Engineering, in ICSE '04. 2004, pp. 614–623. doi: 10.1109/ICSE.2004.1317483.

- [120] C. Elliott and P. Hudak, “Functional Reactive Animation,” in Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming, in ICFP '97. 1997, pp. 263–273. doi: 10.1145/258948.258973.
- [121] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in Programming Languages and Systems (ESOP 2006), in Lecture Notes in Computer Science, vol. 3924. Springer, 2006, pp. 294–308. doi: 10.1007/11693024_20.
- [122] I. Maier and M. Odersky, “Deprecating the Observer Pattern with Scala.React,” technical report EPFL-REPORT-176887, 2012. [Online]. Available: <https://infoscience.epfl.ch/record/176887>
- [123] E. Bainomugisha, A. L. Carreton, T. Van Cutsem, W. De Meuter, and others, “A Survey on Reactive Programming,” ACM Comput. Surv., vol. 45, no. 4, 2013, doi: 10.1145/2501654.2501666.
- [124] A. Margara and G. Salvaneschi, “On the Semantics of Distributed Reactive Programming: The Cost of Consistency,” IEEE Trans. Software Eng., vol. 44, no. 7, pp. 689–711, 2018, doi: 10.1109/TSE.2018.2833109. 88